ARHITECTURA SISTEMELOR DE CALCUL · ASC

Computer Systems Architecture

Theory & Worked Solutions

The complete theory of every chapter — definitions, rules, tables, commented NASM code, common mistakes and summaries — followed by detailed solutions to every exercise of the companion **Workbook**, with the same numbering (Chapter N / Exercise N.M).

x86 (IA-32), 32-bit protected mode · NASM syntax · toolchain NASM + ALINK + OllyDbg
13 chapters · 156 exercises

Year 1 · Computer Science (English) · UBB FMI Cluj-Napoca

Cuprins

Computer Systems Architecture — Theory & Solutions

Click any line to jump to that section. Page numbers match the page footers and the PDF viewer.

Before you start: tools, skeleton and golden rules 1

Notation 3

PART I — DATA AND THE MACHINE

Chapter 1 — Data Representation and Number Bases 4

1.1 Why number bases matter 4
1.2 Positional number systems 4
1.3 Any base to decimal 5
1.4 Decimal to any base: repeated division 5
1.5 Binary, hexadecimal and octal: groups of bits 6
1.6 Between two non-decimal bases 6
1.7 Data sizes on x86 (IA-32) 7
1.8 Writing constants in NASM 7
1.9 Characters: ASCII 7
1.10 Chapter summary 8
Solutions — Chapter 1 9
Exercise 1.1 — To decimal 9
Exercise 1.2 — Decimal to binary and hexadecimal 9
Exercise 1.3 — Binary and hexadecimal by nibbles 9
Exercise 1.4 — Hex digits of the x86 sizes 10
Exercise 1.5 — Octal and hexadecimal through bits 10
Exercise 1.6 — Base 9 to base 7 11
Exercise 1.7 — Strings as bytes 11
Exercise 1.8 — Minimum number of bits 12
Exercise 1.9 — Which constants are valid? 12
Exercise 1.10 — How many values can n digits hold? 13
Exercise 1.11 — Same number, different representations 13
Exercise 1.12 — Character arithmetic in NASM 14

Chapter 2 — Signed and Unsigned Numbers, Two's Complement 15

2.1 Representation versus interpretation 15
2.2 Two's complement 15
2.3 The 2^n shortcut 16
2.4 Ranges and the overlap 16
2.5 Representing a negative number 16
2.6 Sign extension and zero extension 17
2.7 Chapter summary 17
Solutions — Chapter 2 18
Exercise 2.1 — Both interpretations 18
Exercise 2.2 — Negative numbers on a byte 18
Exercise 2.3 — Three methods for two's complement 18
Exercise 2.4 — Reading a register 19
Exercise 2.5 — Negative numbers on a word and on a dword 19
Exercise 2.6 — When a byte is not enough 20
Exercise 2.7 — Sign and zero extension 20
Exercise 2.8 — Minimum number of bits (signed) 21
Exercise 2.9 — True or false: instruction classes 21
Exercise 2.10 — One addition, two meanings 22

Cuprins (continued)

Exercise 2.11 — Taking the low byte of a word 22
Exercise 2.12 — Numbers equal to their own complement 23

Chapter 3 — x86 Architecture: Registers, Flags, Segments 24
3.1 The big picture: EU and BIU 24
3.2 General-purpose registers 24
3.3 Segment registers and EIP 25
3.4 The EFLAGS register 25
3.5 Computing the flags by hand 26
3.6 Chapter summary 27
Solutions — Chapter 3 28
Exercise 3.1 — Which part of EAX changes? 28
Exercise 3.2 — Parts of a register 28
Exercise 3.3 — Registers with fixed roles 28
Exercise 3.4 — Flags after 7Fh + 1 29
Exercise 3.5 — Flags after 80h + 80h 29
Exercise 3.6 — Trace EAX through writes to its parts 30
Exercise 3.7 — inc versus add 30
Exercise 3.8 — Two memory operands 31
Exercise 3.9 — Parity and auxiliary carry 31
Exercise 3.10 — A full flag trace 32
Exercise 3.11 — CS:EIP and how EIP changes 32
Exercise 3.12 — Flags of subtraction and comparison 33

PART II — PROGRAMMING IN NASM

Chapter 4 — Assembly Basics: Program Structure, Data Definition, Directives, Operators 34
4.1 The program skeleton 34
4.2 The elements an assembler works with 34
4.3 Name versus [name] 35
4.4 Data definition directives 36
4.5 The location counter $ and lengths 37
4.6 EQU constants 37
4.7 Operators (evaluated at assembly time) 37
4.8 Worked case: a full data segment 38
4.9 Chapter summary 39
Solutions — Chapter 4 40
Exercise 4.1 — Memory content of a data segment 40
Exercise 4.2 — Find the errors 40
Exercise 4.3 — Lengths with $ 41
Exercise 4.4 — Reading an element of an array 41
Exercise 4.5 — Operators 41
Exercise 4.6 — Little endian views 42
Exercise 4.7 — EQU in brackets 42
Exercise 4.8 — Another full data segment 43
Exercise 4.9 — Where is a size specifier needed? 43
Exercise 4.10 — Precedence and truncation 44
Exercise 4.11 — A complete program with lengths 44
Exercise 4.12 — Reading a memory dump 45

Chapter 5 — Arithmetic: ADD/SUB, MUL/DIV, Type Conversions, Expressions 47
5.1 Rules for two-operand instructions 47
5.2 Addition and subtraction 47
5.3 Numbers wider than one register 48

Cuprins (continued)

5.4 Multiplication: MUL and IMUL 48
5.5 Division: DIV and IDIV 49
5.6 Conversions: making a value wider 50
5.7 Classification of conversions 51
5.8 The method for any expression 51
5.9 Chapter summary 52
Solutions — Chapter 5 53
Exercise 5.1 — (a − b) + (c − d), words, unsigned 53
Exercise 5.2 — a · b − c, mixed sizes, unsigned 53
Exercise 5.3 — Where does the product go? 54
Exercise 5.4 — A division that crashes 54
Exercise 5.5 — (a + b) / c, careful with the division 54
Exercise 5.6 — a / b + c · d, signed 55
Exercise 5.7 — Signs of quotient and remainder 56
Exercise 5.8 — A chain of conversions 56
Exercise 5.9 — Adding and subtracting qwords 57
Exercise 5.10 — A long signed expression 57
Exercise 5.11 — A long unsigned expression 58
Exercise 5.12 — Bug hunt: (b·c − a) / (a + 4) + d 59

Chapter 6 — Overflow: CF versus OF 61
6.1 The two flags 61
6.2 Quick rules for OF and CF 61
6.3 The four cases 62
6.4 Multiplication and division 62
6.5 Analysing a sequence of instructions 63
6.6 Chapter summary 64
Solutions — Chapter 6 65
Exercise 6.1 — 0BCh + 0DEh 65
Exercise 6.2 — 50 + 100 65
Exercise 6.3 — −50 − 100 66
Exercise 6.4 — 8000h − 1 on a word 66
Exercise 6.5 — 0FFh · 2 with mul 67
Exercise 6.6 — −100 · 3 with imul 67
Exercise 6.7 — 1000 / 3 with div 68
Exercise 6.8 — Operators, then a subtraction 68
Exercise 6.9 — Overflow on words 69
Exercise 6.10 — All four combinations, for add and for sub 69
Exercise 6.11 — div versus idiv on a negative dividend 70
Exercise 6.12 — A complete line-by-line analysis 71

Chapter 7 — Bitwise Instructions, Shifts and Rotates, Building Values from Bits 72
7.1 Logical instructions 72
7.2 Masks: the core trick 72
7.3 Shifts 73
7.4 Rotates 74
7.5 Shift operators versus shift instructions 74
7.6 Building a value from bits of other values 74
7.7 Chapter summary 75
Solutions — Chapter 7 76
Exercise 7.1 — Shifts and rotates of 96h 76
Exercise 7.2 — Check a single bit 76
Exercise 7.3 — Clear and set groups of bits 77
Exercise 7.4 — and, or, xor, not on the same byte 77

Cuprins (continued)

Exercise 7.5 — shr versus sar 77
Exercise 7.6 — Rotating through the carry 78
Exercise 7.7 — Swapping parts with one instruction 79
Exercise 7.8 — Count the 1 bits 79
Exercise 7.9 — Build a dword from a byte and a word 80
Exercise 7.10 — Build a byte from a word 81
Exercise 7.11 — Multiply by 10 with shifts 81
Exercise 7.12 — Reverse the bits of a byte 82

PART III — MEMORY, CONTROL FLOW AND STRINGS

Chapter 8 — Addresses, Addressing Modes, Offset Computation 83
8.1 Address and address specification 83
8.2 The offset formula 83
8.3 Addressing modes 84
8.4 Implicit segment registers 84
8.5 Pointer arithmetic at assembly time 85
8.6 LEA: computing an address 85
8.7 Chapter summary 85
Solutions — Chapter 8 86
Exercise 8.1 — Classify the addressing modes 86
Exercise 8.2 — Which segment register? 86
Exercise 8.3 — Valid or invalid? 86
Exercise 8.4 — An element of a dword array 87
Exercise 8.5 — v+4 versus [v+4] 87
Exercise 8.6 — LEA as a calculator 88
Exercise 8.7 — Offsets and overlapping reads 88
Exercise 8.8 — Sum of an array, two ways 89
Exercise 8.9 — A matrix stored by rows 90
Exercise 8.10 — Pointer arithmetic 90
Exercise 8.11 — Base or index: how NASM decides 91
Exercise 8.12 — Walk an array of words backwards 91

Chapter 9 — Comparisons, Jumps, Loops (if / while / for in Assembly) 93
9.1 CMP + conditional jump = IF 93
9.2 The jump table 93
9.3 Why signed and unsigned conditions differ 93
9.4 JMP and jump distances 94
9.5 The LOOP instructions 94
9.6 Translating if / else 95
9.7 Translating while, for and do-while 95
9.8 Chapter summary 96
Solutions — Chapter 9 97
Exercise 9.1 — How many times? (1) 97
Exercise 9.2 — How many times? (2) 97
Exercise 9.3 — Which jumps are taken? 97
Exercise 9.4 — Minimum of two numbers 98
Exercise 9.5 — Maximum of a string of signed bytes 98
Exercise 9.6 — Count the even words 99
Exercise 9.7 — Sum of the elements greater than 10 100
Exercise 9.8 — Factorial with a while loop 100
Exercise 9.9 — loope: skip the leading zeros 101
Exercise 9.10 — A loop whose body is too long 102
Exercise 9.11 — Is the number prime? 102

Cuprins (continued)

Exercise 9.12 — Bubble sort 103

Chapter 10 — Strings and String Instructions 105
10.1 Setting up a string operation 105
10.2 The string instructions 105
10.3 Repeat prefixes 106
10.4 XLAT: table lookup 106
10.5 Typical problems 106
10.6 Chapter summary 109
Solutions — Chapter 10 110
Exercise 10.1 — Trace the string registers 110
Exercise 10.2 — Copy a string 110
Exercise 10.3 — Uppercase copy 111
Exercise 10.4 — Compare two strings 112
Exercise 10.5 — Sum of the bytes of each dword 112
Exercise 10.6 — Keep only the positive elements 113
Exercise 10.7 — Concatenate low and high bytes 114
Exercise 10.8 — A byte as two hex characters 115
Exercise 10.9 — Reverse a string with string instructions 116
Exercise 10.10 — Split by parity 116
Exercise 10.11 — Find a character and its position 117
Exercise 10.12 — Remove the spaces in place 118

PART IV — PROCEDURES AND MODULES

Chapter 11 — The Stack, CALL/RET, cdecl, printf/scanf, Files 119
11.1 The stack 119
11.2 CALL and RET 120
11.3 The cdecl convention 121
11.4 printf and scanf 121
11.5 Files 122
11.6 Chapter summary 123
Solutions — Chapter 11 124
Exercise 11.1 — What does this sequence do? 124
Exercise 11.2 — Follow ESP 124
Exercise 11.3 — Address or value? 125
Exercise 11.4 — Joining and splitting through the stack 125
Exercise 11.5 — Read two numbers, print (a+b)·(a−b) 125
Exercise 11.6 — Read until 0, print the maximum 126
Exercise 11.7 — Write 1…10 to a file 127
Exercise 11.8 — A procedure with three parameters 128
Exercise 11.9 — Print a byte signed and unsigned 129
Exercise 11.10 — Count the digits in a file 129
Exercise 11.11 — A GCD procedure 131
Exercise 11.12 — Bug hunt in a printing loop 132

Chapter 12 — Multimodule Programming: ASM + ASM and ASM + C 134
12.1 Why several modules? 134
12.2 ASM + ASM 134
12.3 ASM + C 135
12.4 A function with a stack frame and saved registers 136
12.5 Call code, entry code, exit code 136
12.6 Chapter summary 137
Solutions — Chapter 12 138

Cuprins (continued)

Exercise 12.1 — global or extern? 138
Exercise 12.2 — Brackets or not? 138
Exercise 12.3 — Names and formats 138
Exercise 12.4 — Why save EBX but not ECX? 139
Exercise 12.5 — max in assembly, called from C 139
Exercise 12.6 — Sum of the digits of a C string 140
Exercise 12.7 — Use count_vowels 141
Exercise 12.8 — ASM + ASM with scanf and printf 141
Exercise 12.9 — Identify the pieces of a call 142
Exercise 12.10 — Sum of an int array 143
Exercise 12.11 — A variable shared by two modules 144
Exercise 12.12 — cdecl versus stdcall 144

PART V — INSIDE THE INSTRUCTION

Chapter 13 — Machine Instruction Representation (Internal Format, ModR/M, SIB, Prefixes) 146
13.1 The internal format 146
13.2 The ModR/M byte 146
13.3 The SIB byte 147
13.4 Decoded examples 148
13.5 Prefixes 148
13.6 Chapter summary 149
Solutions — Chapter 13 150
Exercise 13.1 — Decode a ModR/M byte 150
Exercise 13.2 — Fields of a direct-address instruction 150
Exercise 13.3 — Why does [esp] need a SIB? 150
Exercise 13.4 — Implicit prefixes 151
Exercise 13.5 — The four prefix categories 151
Exercise 13.6 — Decode with a displacement 152
Exercise 13.7 — Decode with a SIB byte 152
Exercise 13.8 — Encode two instructions 152
Exercise 13.9 — Byte-sized moves 153
Exercise 13.10 — Instruction lengths 153
Exercise 13.11 — A 32-bit displacement 153
Exercise 13.12 — All four prefixes at once 154

Before you start: tools, skeleton and golden rules

This manual has one chapter for every lesson of the course, in the course order. Every chapter has two parts:

1. **Theory** — definitions, rules, tables, commented NASM code, worked examples, common mistakes and a summary.
2. **Solutions** — every exercise of the Workbook, with the same number, solved in four steps: *Statement* → *Theory used* → *Step by step* → *Final result*.

All numbers in this manual (conversions, results, flags) were checked by computer, and every program was assembled with NASM and executed instruction by instruction to confirm the register and memory values written in its comments.

The toolchain

Tool	Role	Command
NASM	assembler: `.asm` → object file `.obj`	`nasm -fobj prog.asm`
ALINK	linker: `.obj` → Windows executable `.exe`	`alink -oPE -subsys console -entry start prog.obj`
OllyDbg	debugger: run step by step, watch registers, flags, memory	open `prog.exe`, F8 = step over, F7 = step into

For programs that combine assembly with C (Chapter 12): `nasm -f win32 m.asm`, then `gcc -m32 m.obj main.c -o main.exe` (MinGW) or a Visual Studio project.

The program skeleton

```
bits 32                         ; assemble for the 32-bit architecture
global start                    ; the entry point 'start' is visible to the linker
extern exit                     ; exit is defined elsewhere...
import exit msvcrt.dll          ; ...namely in msvcrt.dll (the C runtime)

segment data use32 class=data
    a db 10                     ; variables go here

segment code use32 class=code
start:
    mov al, [a]                 ; instructions go here; AL = 10
    push dword 0                ; exit(0): push the parameter
    call [exit]                 ; and call the C function -> the program ends
```

When a solution shows only the data and code parts, the missing lines are exactly these.

Golden rules

1. `a` is an **address**, `[a]` is the **value** at that address. NASM does not remember variable sizes: when no register fixes the size, write `byte`, `word` or `dword`.
2. Both operands of an instruction have the **same size**; at most **one** operand is in memory.
3. `mul` / `div` use `AL` / `AX` / `EAX` (and `AH` / `DX` / `EDX`) implicitly; the dividend is **twice** the size of the divisor; the quotient must fit, or the program stops with an exception.
4. Signed → `imul`, `idiv`, `cbw`, `cwd`, `cwde`, `cdq`, `movsx`, `jg` / `jl`. Unsigned → `mul`, `div`, `mov ah, 0`, `movzx`, `ja` / `jb`.

5. Memory is **little endian**: the lowest byte of a value is at the lowest address.
6. `loop` uses `ECX`; guard it with `jecxz`; never change `ECX` inside the loop body.
7. C functions: push parameters right to left, the result comes back in `EAX`, clean the stack yourself (`add esp, 4*n`); `EAX`, `ECX`, `EDX` are destroyed by the call.

Notation

Notation	Meaning
`1011b`, `1011`	a number written in binary (suffix **b**); in tables 1 is blue and 0 is orange
`7Fh`, `0x7F`	a number written in hexadecimal (suffix **h** or prefix **0x**)
`(212)3`	the digits 212 read in base 3
`a`	a variable name = its **address** (offset)
`[a]`	the **value** stored in memory at address `a`
`byte [a]`, `word [a]`, `dword [a]`	the value at `a` read as 1, 2 or 4 bytes
`DX:AX`, `EDX:EAX`	a pair of registers holding one value: the left one is the **high** part
`AL`, `AH`, `AX`, `EAX`	the low byte, high byte of AX, low word, and the whole 32-bit register
`CF OF ZF SF PF AF`	Carry, Overflow, Zero, Sign, Parity, Auxiliary-carry flags
`DF`	Direction flag (string instructions)
`$`	the location counter (current offset in the segment)
signed / *unsigned*	the two interpretations of the same bits
`→`	"becomes" / "gives"
☐	tick the box when you have solved the exercise

PART I

Data and the Machine

Number bases, signed and unsigned values, the x86 registers and flags.

CHAPTER 1

Data Representation and Number Bases

Bits, bytes and the number bases a programmer uses every day: binary, hexadecimal, octal and decimal, the conversions between them, the x86 data sizes, NASM constants and ASCII characters.

1.1 Why number bases matter

Everything inside the processor — instructions, numbers, characters, addresses — is a sequence of **bits** (binary digits, 0 or 1). A high-level language hides the bits; assembly language does not. When you write `mov al, 0B4h` you choose every one of the 8 bits of `AL`, and when OllyDbg shows you `EAX = 0000012C` you must recognise the number 300.

So an assembly programmer needs three skills:

- read and write numbers in **binary** (base 2) — the way the machine stores them;
- read and write numbers in **hexadecimal** (base 16) — a compact way to write binary (1 hex digit = 4 bits);
- switch between these and **decimal** (base 10) quickly and without mistakes.

> **A NUMBER IS NOT ITS REPRESENTATION**
>
> `128`, `80h` and `1000 0000b` are **the same number**, written in three bases. The value stored in `EAX` simply *is*; the base matters only when a human reads it or writes it. There is no instruction that "converts EAX from binary to hexadecimal" — the bits do not change, only the way we write them on paper.

1.2 Positional number systems

In a **positional** system with **base** (radix) *r*, a number with the digits $a_{n-1} \ldots a_1\, a_0$ has the value

```
A = a(n-1)·r^(n-1) + ... + a1·r^1 + a0·r^0
```

Each digit is between 0 and r−1 and is worth more the further left it stands: its **weight** is a power of the base. With *n* digits you can write exactly $\mathbf{r^n}$ different values, from 0 to r^n-1.

Base	Digits	Name	Written in NASM as
2	0, 1	binary	`1011b`
8	0–7	octal	`17q` (rare)
10	0–9	decimal	`123`
16	0–9, A(10), B(11), C(12), D(13), E(14), F(15)	hexadecimal	`0B4h`, `0xB4`

Powers you should know by heart

2^n	value		16^n	value
2^0	**1**		16^0	**1**
2^4	16		16^1	16
2^7	128		16^2	256
2^8	256		16^3	4 096
2^{10}	1 024		16^4	65 536
2^{15}	32 768			
2^{16}	65 536			
2^{31}	2 147 483 648			
2^{32}	4 294 967 296			

WHY THESE POWERS

2^8, 2^{16}, 2^{32} are the number of values of a byte, a word and a doubleword; 2^7, 2^{15}, 2^{31} are the limits of the signed ranges (Chapter 2). Since $16 = 2^4$, every power of 16 is also a power of 2: $16^2 = 2^8$, $16^4 = 2^{16}$.

1.3 Any base to decimal

Multiply every digit by its weight and add the results. Write the weights above the digits if it helps.

```
(1111000)2 = 1·64 + 1·32 + 1·16 + 1·8 + 0·4 + 0·2 + 0·1 = 120
(5C2)16    = 5·256 + 12·16 + 2·1 = 1280 + 192 + 2      = 1474
(212)3     = 2·9 + 1·3 + 2·1                           = 23
```

The same formula works for the digits after the point; their weights are negative powers of the base (r^{-1}, r^{-2}, ...):

```
(1011.101)2 = 8 + 0 + 2 + 1 + 1·0.5 + 0·0.25 + 1·0.125 = 11.625
```

BINARY IN PRACTICE

For binary you only add the weights of the 1 bits. Write the weights 128 64 32 16 8 4 2 1 above a byte and add those that stand above a 1.

1.4 Decimal to any base: repeated division

Divide the number by the base, write down the remainder, and continue with the quotient until the quotient is 0. The remainders, **read from the last one to the first**, are the digits of the result.

```
186 : 2 = 93 remainder 0          195 : 16 = 12 remainder 3
 93 : 2 = 46 remainder 1           12 : 16 =  0 remainder 12 (C)
 46 : 2 = 23 remainder 0          => 195 = C3h
 23 : 2 = 11 remainder 1
 11 : 2 =  5 remainder 1
  5 : 2 =  2 remainder 1
  2 : 2 =  1 remainder 0
  1 : 2 =  0 remainder 1
=> 186 = 1011 1010b
```

Why does it work? The last digit of a number in base *r* is exactly its remainder modulo *r*; dividing by *r* removes that digit and shifts the others one position to the right. So the first remainder is the **lowest** digit — that is why we read them backwards.

READING THE REMAINDERS IN THE WRONG ORDER

The first remainder is the **rightmost** digit. Reading 186 top-down would give 0101 1101b = 93 — a different number. Always check: convert the result back to decimal.

A FASTER WAY FOR BINARY

Subtract the largest power of 2 that fits, mark a 1 in its position, and repeat: 186 = 128 + 32 + 16 + 8 + 2 → bits 7, 5, 4, 3, 1 are 1 → `1011 1010`b.

1.5 Binary, hexadecimal and octal: groups of bits

Because 16 = 2^4, one hexadecimal digit corresponds to **exactly 4 bits** (a **nibble**). Converting between binary and hexadecimal needs no arithmetic at all: group the bits by 4 **from the right**, pad the leftmost group with zeros, and translate each group.

```
110 1010 1000b  ->  0110 1010 1000  ->  6 A 8  ->  6A8h
FA97h           ->  1111 1010 1001 0111b
```

Memorise the 16 nibbles:

hex	bin	hex	bin	hex	bin	hex	bin
0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

In the same way, since 8 = 2^3, one **octal** digit corresponds to **3 bits**. Octal ↔ hexadecimal goes through binary: write 3 bits for each octal digit, then regroup by 4 from the right.

```
(357)8 -> 011 101 111 -> 0 1110 1111 -> EFh       (check: 3·64 + 5·8 + 7 = 239 = EFh)
```

GROUPING FROM THE LEFT

Groups are always formed **from the right** (from the least significant bit). Grouping 110 1010 1000 from the left as 1101 0101 000 gives a wrong result.

1.6 Between two non-decimal bases

For two bases that are not related by powers of 2 (for example 9 and 7), go **through decimal**: first base 9 → decimal with the weights, then decimal → base 7 by repeated division.

```
(125)9 = 1·81 + 2·9 + 5 = 104
104 : 7 = 14 remainder 6
 14 : 7 =  2 remainder 0
  2 : 7 =  0 remainder 2      => (125)9 = (206)7
check: 2·49 + 0·7 + 6 = 104
```

The exception is 2 ↔ 8 ↔ 16: these go directly through bits (previous section).

1.7 Data sizes on x86 (IA-32)

The x86 processor works with a few fixed sizes. Their names are used everywhere — in instructions, in NASM directives and in this manual.

Name	Bits	Bytes	NASM define	Unsigned range	Signed range
byte	8	**1**	`db`	0 … 255	−128 … 127
word	16	2	`dw`	0 … 65 535	−32 768 … 32 767
doubleword (dword)	32	4	`dd`	0 … 4 294 967 295	−2 147 483 648 … 2 147 483 647
quadword (qword)	64	8	`dq`	0 … $2^{64} - 1$	-2^{63} … $2^{63} - 1$

- Bits are numbered from **0** (least significant, on the right) to **n−1** (most significant, on the left).
- A byte has two nibbles: bits 0–3 (**low nibble**) and bits 4–7 (**high nibble**). So a byte is always written with **2** hex digits, a word with **4**, a dword with **8**.
- Memory on x86 is **byte addressable**: every address designates exactly one byte. A word occupies two consecutive addresses, a dword four.

The signed ranges are explained in Chapter 2.

1.8 Writing constants in NASM

NASM accepts constants in several bases. The suffix (or prefix) tells it which base you mean.

```
segment code use32 class=code
start:
    mov al, 101b            ; binary (suffix b)         -> AL = 5
    mov bx, 34ABh           ; hexadecimal, suffix h     -> BX = 34ABh
    mov dx, 0ABCDh          ; a hex constant must START with a digit -> leading 0
    mov ecx, 0x1F           ; hexadecimal with 0x prefix -> ECX = 31
    mov ah, 'a'             ; character constant = its ASCII code -> AH = 97 = 61h
    mov esi, -114           ; decimal is the default    -> ESI = FFFFFF8Eh
```

`MOV AX, ABCDH`

Without the leading 0, NASM reads `ABCDh` as a **label name** (it starts with a letter) and reports an error (undefined symbol). Always write `0ABCDh`.

A CONSTANT THAT DOES NOT FIT

`mov al, 256` — AL has 8 bits and holds at most 255. NASM gives a warning and keeps only the low 8 bits (256 = 100h → 00h), so AL becomes 0. Always check that the constant fits the destination.

1.9 Characters: ASCII

A character is stored as **one byte** holding its **ASCII code**. A string is simply a sequence of such bytes.

characters	codes (decimal)	codes (hex)
`'0'` … `'9'`	48 … 57	30h … 39h
`'A'` … `'Z'`	65 … 90	41h … 5Ah
`'a'` … `'z'`	97 … 122	61h … 7Ah
space	32	20h
line feed / carriage return	10 / 13	0Ah / 0Dh

Useful tricks that follow from the table:

- `'a' - 'A' = 32`: lowercase → uppercase is `sub al, 32` (or, more readable, `sub al, 'a'-'A'`); uppercase → lowercase is `add al, 32`.
- digit character → its value: `sub al, '0'` ('7' = 37h → 7). Value → character: `add al, '0'`.
- the characters `'0'`…`'9'` are **not** the numbers 0…9: `'5'` is the byte 35h = 53.

```
segment code use32 class=code
start:
    mov al, 'g'             ; AL = 67h = 103
    sub al, 'a'-'A'         ; AL = 47h = 'G'
    mov bl, '7'             ; BL = 37h
    sub bl, '0'             ; BL = 7 (the value of the digit)
```

BINARY VIEW OF THE TRICK

32 = `0010 0000`b is bit 5. Lowercase and uppercase letters differ **only in bit 5**: 'g' = `0110 0111`, 'G' = `0100 0111`.

1.10 Chapter summary

IN SHORT

- A value and its written form are different things; bases only matter for humans.
- Base *r*, *n* digits → r^n values; value = Σ digit · weight.
- Any base → decimal: weights. Decimal → base *r*: repeated division, remainders read **bottom-up**.
- 1 hex digit = 4 bits, 1 octal digit = 3 bits; group **from the right**.
- byte/word/dword/qword = 8/16/32/64 bits = 2/4/8/16 hex digits; memory is byte addressable.
- NASM: `101b`, `0ABCDh` (leading 0!), `0x1F`, `'a'`; the constant must fit the destination.
- ASCII: '0' = 30h, 'A' = 41h, 'a' = 61h; 'a' − 'A' = 32; digit value = char − '0'.

Solutions — Chapter 1

Exercise 1.1 — To decimal BASIC

STATEMENT

Convert to decimal: `1011 0110b`, `7Fh`, `0FFFFh`, `100h`, `(212)3`.

THEORY USED

Any base → decimal: sum of digit · weight (section 1.3).

STEP BY STEP

number	computation	decimal
1011 0110b	128 + 32 + 16 + 4 + 2	182
7Fh	7·16 + 15	127
0FFFFh	all 16 bits are 1 = $2^{16} - 1$	65 535
100h	1·256	256
(212)3	2·9 + 1·3 + 2	23

For `0FFFFh` there is a shortcut: a number made only of 1 bits on n positions equals $2^n - 1$.

FINAL RESULT

182, 127, 65 535, 256, 23.

Exercise 1.2 — Decimal to binary and hexadecimal BASIC

STATEMENT

Convert to binary and to hexadecimal: 200, 1000, 65, 4095.

THEORY USED

Decimal → binary (powers of 2 or repeated division), binary → hex by groups of 4 bits (1.4, 1.5).

STEP BY STEP

- 200 = 128 + 64 + 8 → `1100 1000`b → C8h.
- 1000 = 512 + 256 + 128 + 64 + 32 + 8 → `11 1110 1000`b → groups 0011 1110 1000 → 3E8h.
- 65 = 64 + 1 → `0100 0001`b → 41h (this is also the ASCII code of 'A').
- 4095 = 4096 − 1 = $2^{12} - 1$ → twelve 1 bits `1111 1111 1111`b → FFFh.

Check one by hex weights: 3E8h = 3·256 + 14·16 + 8 = 768 + 224 + 8 = 1000 ✓.

FINAL RESULT

200 = 1100 1000b = C8h; 1000 = 11 1110 1000b = 3E8h; 65 = 0100 0001b = 41h; 4095 = 1111 1111 1111b = FFFh.

Exercise 1.3 — Binary and hexadecimal by nibbles BASIC

STATEMENT

Write in binary: `3Ch`, `0B7h`, `1F0Ah`. Write in hexadecimal: `1101 0111b`, `1 0000 0001b`, `10 1011 1100b`.

THEORY USED

1 hex digit = 4 bits; group bits by 4 from the right (1.5).

STEP BY STEP

Hex → binary, one nibble per digit:

- 3Ch → 3 = 0011, C = 1100 → `0011 1100`b
- 0B7h → B = 1011, 7 = 0111 → `1011 0111`b
- 1F0Ah → 0001 1111 0000 1010 → `1 1111 0000 1010`b (the leading zeros can be dropped)

Binary → hex, group from the right and pad on the left:

- 1101 0111b → D 7 → D7h
- 1 0000 0001b → 0001 0000 0001 → 101h
- 10 1011 1100b → 0010 1011 1100 → 2BCh

FINAL RESULT

3Ch = 0011 1100b, 0B7h = 1011 0111b, 1F0Ah = 1 1111 0000 1010b; D7h, 101h, 2BCh.

Exercise 1.4 — Hex digits of the x86 sizes BASIC

STATEMENT

How many hex digits does a byte, a word, a dword and a qword have? Write the largest unsigned value of each size in hexadecimal and in decimal.

THEORY USED

1 hex digit = 4 bits; data sizes table (1.7); a value made of n one-bits is $2^n - 1$.

STEP BY STEP

Divide the number of bits by 4. The largest unsigned value has all bits 1, i.e. all hex digits F.

size	bits	hex digits	largest	decimal
byte	8	2	0FFh	255
word	16	4	0FFFFh	65 535
dword	32	8	0FFFFFFFFh	4 294 967 295
qword	64	16	0FFFFFFFFFFFFFFFFh	18 446 744 073 709 551 615

FINAL RESULT

2, 4, 8, 16 hex digits; largest values FFh = 255, FFFFh = 65 535, FFFFFFFFh = 4 294 967 295, FFFFFFFFFFFFFFFFh = $2^{64} - 1$.

Exercise 1.5 — Octal and hexadecimal through bits INTERMEDIATE

STATEMENT

Convert `(357)8` to hexadecimal and `2Ah` to octal, directly through bits (without decimal). Check both results in decimal.

THEORY USED

1 octal digit = 3 bits, 1 hex digit = 4 bits; regroup from the right (1.5).

STEP BY STEP

(357)8 → hex. 3 5 7 → 011 101 111 → `0 1110 1111` → regroup by 4 from the right: 1110 1111 → **EFh**. Check: 3·64 + 5·8 + 7 = 239 and EFh = 14·16 + 15 = 239 ✓.

2Ah → octal. 2 A → 0010 1010 → `101010` → regroup by 3 from the right: 101 010 → 5 2 → **(52)8**. Check: 2Ah = 42 and (52)8 = 5·8 + 2 = 42 ✓.

FINAL RESULT

(357)8 = EFh (= 239); 2Ah = (52)8 (= 42).

Exercise 1.6 — Base 9 to base 7 INTERMEDIATE

STATEMENT

Convert `(125)9` to base 7 and check the result. Then convert `(1011.101)2` and `(A.8)16` to decimal.

THEORY USED

Two non-decimal bases: go through decimal (1.6). Digits after the point have weights r^{-1}, r^{-2}, ... (1.3).

STEP BY STEP

(125)9 = 1·81 + 2·9 + 5 = 104. Repeated division by 7: 104 : 7 = 14 r **6**, 14 : 7 = 2 r **0**, 2 : 7 = 0 r **2**. Reading bottom-up: **(206)7**. Check: 2·49 + 0·7 + 6 = 104 ✓.

(1011.101)2 = 8 + 2 + 1 + 0.5 + 0.125 = **11.625**.

(A.8)16 = 10 + 8·16^{-1} = 10 + 0.5 = **10.5**.

FINAL RESULT

(125)9 = (206)7; (1011.101)2 = 11.625; (A.8)16 = 10.5.

Exercise 1.7 — Strings as bytes INTERMEDIATE

STATEMENT

Write the bytes (in hex) that represent the string `'Hi5'` and the string `'AsC 9'`. What does `AL` contain after `mov al, 'g'` followed by `sub al, 'a'-'A'` ?

THEORY USED

Each character is one byte holding its ASCII code; 'a' − 'A' = 32 (1.9).

STEP BY STEP

- 'H' = 48h, 'i' = 69h, '5' = 35h → **48h 69h 35h**.
- 'A' = 41h, 's' = 73h, 'C' = 43h, ' ' = 20h, '9' = 39h → **41h 73h 43h 20h 39h**.
- 'g' = 67h = 103; 'a' − 'A' = 97 − 65 = 32; 103 − 32 = 71 = 47h = **'G'**.

Note that '5' and '9' are stored as 35h and 39h, not as 5 and 9.

FINAL RESULT

'Hi5' = 48 69 35; 'AsC 9' = 41 73 43 20 39; AL = 47h = 'G'.

Exercise 1.8 — Minimum number of bits INTERMEDIATE

STATEMENT

What is the minimum number of bits needed to write, unsigned, the values 255, 256, 300 and 1000? What is the smallest x86 data type that can hold each?

THEORY USED

With n bits the largest unsigned value is $2^n - 1$ (1.2, 1.7).

STEP BY STEP

Find the smallest n with $2^n >$ value:

value	power interval	bits	smallest type
255	$2^7 = 128 \le 255 < 256 = 2^8$	8	byte
256	$2^8 = 256 \le 256 < 512$	9	word
300	$256 \le 300 < 512$	9 (`1 0010 1100`)	word
1000	$512 \le 1000 < 1024$	10	word

FINAL RESULT

255 → 8 bits (byte); 256 → 9 bits (word); 300 → 9 bits (word); 1000 → 10 bits (word).

Exercise 1.9 — Which constants are valid? INTERMEDIATE

STATEMENT

For each line say whether NASM accepts it and, if yes, what the destination register contains (in hex):

```
mov ax, ABCDh
mov ax, 0ABCDh
mov al, 256
mov eax, 0x1F
mov bl, 'a'
mov cl, 1010b
mov dx, 0FFh
```

THEORY USED

Constants in NASM (1.8): hex constants must start with a digit; the constant must fit the destination.

STEP BY STEP

line	verdict	result
`mov ax, ABCDh`	error: `ABCDh` is read as an (undefined) label name	—
`mov ax, 0ABCDh`	valid	AX = ABCDh
`mov al, 256`	warning, only the low 8 bits are kept	AL = 00h
`mov eax, 0x1F`	valid	EAX = 0000001Fh
`mov bl, 'a'`	valid	BL = 61h
`mov cl, 1010b`	valid	CL = 0Ah
`mov dx, 0FFh`	valid (the constant is extended to 16 bits)	DX = 00FFh

FINAL RESULT

Only `mov ax, ABCDh` is an error and `mov al, 256` gives a warning (AL = 0); the others load ABCDh, 1Fh, 61h, 0Ah, 00FFh.

Exercise 1.10 — How many values can n digits hold? CHALLENGE

STATEMENT

(a) What is the largest number written with 5 binary digits, with 3 hexadecimal digits and with 4 octal digits? (b) How many different values can a word hold? (c) Show that 3 hex digits and 12 bits describe exactly the same set of values.

THEORY USED

With n digits in base r there are r^n values, the largest is $r^n - 1$ (1.2); $16 = 2^4$.

STEP BY STEP

(a) $2^5 - 1 =$ **31** = 11111b; $16^3 - 1 =$ **4095** = FFFh; $8^4 - 1 =$ **4095** = (7777)8.

(b) A word has 16 bits → $2^{16} =$ **65 536** values (0 … 65 535 unsigned).

(c) 3 hex digits give $16^3 = (2^4)^3 = 2^{12} = 4096$ values, the same count as 12 bits, and both start at 0, so both describe 0 … 4095. In fact each hex digit *is* a group of 4 bits — and in the same way 4 octal digits are 4 groups of 3 bits = 12 bits, which is why (a) found the same maximum 4095 twice.

FINAL RESULT

31, 4095, 4095; a word holds 65 536 values; $16^3 = 2^{12} = 4096$, so both cover 0 … 4095.

Exercise 1.11 — Same number, different representations CHALLENGE

STATEMENT

Which of these denote the same number: `128`, `80h`, `1000 0000b`, `(200)8`, `(11112)3`, `(400)5`? Explain how you decide.

THEORY USED

Convert everything to decimal with the weights (1.3); a number does not depend on its representation (1.1).

STEP BY STEP

- 80h = 8·16 = 128;
- 1000 0000b = 2^7 = 128;
- (200)8 = 2·64 = 128 (also: 010 000 000b = 1000 0000b);
- (11112)3 = 81 + 27 + 9 + 3 + 2 = **122**;
- (400)5 = 4·25 = 100.

So 128, 80h, 1000 0000b and (200)8 are four representations of the same number; (11112)3 = 122 and (400)5 = 100 are different numbers.

FINAL RESULT

128 = 80h = 1000 0000b = (200)8; (11112)3 = 122 and (400)5 = 100 differ.

Exercise 1.12 — Character arithmetic in NASM CHALLENGE

STATEMENT

`AL` holds the code of a lowercase letter and `BL` holds the code of a decimal digit character. (a) Give two different instructions that turn `AL` into the matching uppercase letter. (b) Turn `BL` into the value of the digit. (c) For `AL = 'q'` and `BL = '8'` give both registers in hex after your instructions. (d) Why can `and al, 0DFh` also be used in (a)?

THEORY USED

ASCII table and tricks (1.9); 'a' − 'A' = 32 = bit 5.

STEP BY STEP

(a) `sub al, 32` or, equivalently and more readable, `sub al, 'a'-'A'`.

(b) `sub bl, '0'` (or `sub bl, 30h`).

```
segment code use32 class=code
start:
    mov al, 'q'             ; AL = 71h
    mov bl, '8'             ; BL = 38h
    sub al, 'a'-'A'         ; AL = 51h = 'Q'
    sub bl, '0'             ; BL = 08h
    mov cl, 'q'
    and cl, 0DFh            ; CL = 51h too: bit 5 cleared
```

(c) 'q' = 71h → 71h − 20h = **51h** = 'Q'; '8' = 38h → 38h − 30h = **08h**.

(d) 0DFh = **1101 1111**b has a 0 only in bit 5. Lowercase letters have bit 5 = 1 and the matching uppercase letters differ from them only in that bit, so clearing bit 5 subtracts exactly 32.

FINAL RESULT

(a) `sub al, 32` / `sub al, 'a'-'A'`; (b) `sub bl, '0'`; (c) AL = 51h ('Q'), BL = 08h; (d) 0DFh clears bit 5, which is worth 32.

CHAPTER 2

Signed and Unsigned Numbers, Two's Complement

The same bits can mean two different numbers. This chapter explains the unsigned and the signed (two's complement) interpretation, the ranges of each size, and how to make a value wider without changing it.

2.1 Representation versus interpretation

A register or a memory location only holds **bits**. The same 8 bits can be read in two ways:

- **unsigned interpretation** — all bits are magnitude: `1110 1111`b = 239;
- **signed interpretation** — the most significant bit (**MSB**) is the **sign bit**, and negative numbers are stored in **two's complement**: `1110 1111`b = −17.

The processor does **not** know which interpretation you have in mind. The **instruction** you choose decides:

Class	Instructions	What they assume
don't care	`mov`, `add`, `sub`, `adc`, `sbb`, `inc`, `dec`, `cmp`, `and`, `or`, `xor`, `not`	the same bits come out either way
unsigned	`mul`, `div`, `movzx`, `ja` / `jb` / `jae` / `jbe`	the operands are unsigned
signed	`imul`, `idiv`, `cbw`, `cwd`, `cwde`, `cdq`, `movsx`, `jg` / `jl` / `jge` / `jle`	the operands are signed

> **BE CONSISTENT**
>
> In an "unsigned" problem use only the first two classes; in a "signed" problem only the first and the third. Mixing (for example `mul` on signed values) silently gives wrong results.

Why can `add` be "don't care"? Because adding two bit patterns gives the same result bits whether you read them as signed or unsigned — two's complement was designed exactly for that. Only the **meaning** of the result, and the flags CF/OF that describe it, differ (Chapter 6).

2.2 Two's complement

On *n* bits, the two's complement representation of a negative number −A is the binary form of **$2^n - A$**. On a byte (n = 8): −1 → 256 − 1 = 255 = FFh; −109 → 256 − 109 = 147 = 93h.

Three ways to get the two's complement of a bit pattern

Take `1001 0011`b (= 147 unsigned):

1. **Subtract from 2^n** (1 followed by n zeros): 1 0000 0000 − 1001 0011 = `0110 1101`.
2. **Invert all bits and add 1**: `0110 1100` + 1 = `0110 1101`.
3. **Fastest on paper**: from the right, keep all bits up to and including the **first 1**, invert all the bits to its left: 1001 0011 → `0110 1101`.

So `0110 1101`b = 6Dh = 109, and `1001 0011`b interpreted as signed is **−109**.

> **TWO'S COMPLEMENT IS "NEGATION"**
>
> Taking the two's complement of a pattern gives the pattern of the opposite number. Doing it twice gives the original back. The instruction `neg` does exactly this (Chapter 5).

FORGETTING THE "+1"

Only inverting the bits gives the **one's complement**: `0110 1100` = 108, not 109. One's complement is what `not` computes; two's complement is `not` + 1.

2.3 The 2^n shortcut

A pattern whose MSB is 1 has an unsigned value and a signed value that differ by exactly 2^n:

```
signed value   = unsigned value - 2^n      (when the MSB is 1)
unsigned value = signed value + 2^n        (when the value is negative)
```

This is the quickest way to switch interpretations:

pattern	size	unsigned	signed
FFh	byte	255	255 − 256 = −1
80h	byte	128	128 − 256 = −128
C8h	byte	200	200 − 256 = −56
93h	byte	147	147 − 256 = −109
FF6Dh	word	65 389	65 389 − 65 536 = −147

Why 2^n? The MSB of an *n*-bit pattern has weight $+2^{n-1}$ in the unsigned reading and -2^{n-1} in the signed reading; the difference is $2 \cdot 2^{n-1} = 2^n$.

2.4 Ranges and the overlap

On *n* bits:

- unsigned: $[0, 2^n - 1]$
- signed: $[-2^{n-1}, 2^{n-1} - 1]$

size	unsigned	signed
byte	0 … 255	−128 … 127
word	0 … 65 535	−32 768 … 32 767
dword	0 … 4 294 967 295	−2 147 483 648 … 2 147 483 647

Both ranges contain the same number of values (2^n). Patterns that start with 0 (00h…7Fh on a byte) mean the **same** positive number in both interpretations. Only patterns that start with 1 (80h…FFh) differ: unsigned they are 128…255, signed they are −128…−1.

```
byte pattern:  00h ........ 7Fh | 80h ......... FFh
unsigned:        0 ........ 127 | 128 ......... 255
signed:          0 ........ 127 | -128 ........  -1
```

THE SIGNED RANGE IS NOT SYMMETRIC

There is one more negative value than positive values: −128 exists on a byte, +128 does not. That is why the two's complement of 80h is 80h again (see Exercise 2.12).

2.5 Representing a negative number

To write −A on a given size:

1. Check that −A is inside the signed range of that size.
2. Write A in binary on that size.
3. Take the two's complement (any of the three methods), or compute 2^n − A directly.

Examples.

- −109 on a byte: in range; 109 = `0110 1101` → two's complement `1001 0011` = **93h**.
- −147 on a byte: **not** in [−128, 127] → it needs a word. 147 = `0000 0000 1001 0011` → two's complement `1111 1111 0110 1101` = **FF6Dh**.

> **DOES −VAL FIT IN THE SAME SIZE AS VAL?**
> If VAL (positive) starts with 0 in binary on that size, −VAL fits in the same size. If VAL starts with 1 (for example 147 on a byte), −VAL needs the next size up. The only extra case is -2^{n-1} itself (−128 on a byte), which fits although 128 starts with 1.

Minimum number of bits

- unsigned *v* needs the smallest *n* with v ≤ 2^n − 1: 3 = 11b → 2 bits;
- signed *v* needs the smallest *n* with -2^{n-1} ≤ v ≤ 2^{n-1} − 1: −3 needs 3 bits (`101`b; the signed range on 3 bits is −4 ... 3), +3 also needs 3 bits signed (`011`b — a sign bit is required).

2.6 Sign extension and zero extension

Often a value must be made wider (byte → word, word → dword) — for example before adding a byte to a word. The new, wider value must mean the **same number**:

- **unsigned** → **zero extension**: fill the new high part with 0;
- **signed** → **sign extension**: fill the new high part with copies of the sign bit (0 for positive, 1 for negative).

```
-12 on a byte  = F4h   = 1111 0100b
-12 on a word  = FFF4h = 1111 1111 1111 0100b      (sign-extended)
244 on a word  = 00F4h = 0000 0000 1111 0100b      (zero-extended: same byte F4h, other meaning!)
```

The byte F4h is −12 signed and 244 unsigned; each extension keeps the meaning of **its** interpretation.

> **EXTENDING WITH THE WRONG METHOD**
> Zero-extending a negative byte turns it into a large positive number (F4h → 00F4h = 244 instead of −12). Choose the extension from the interpretation of the problem, not from the value you happen to see.

The instructions that perform these extensions (`cbw`, `cwd`, `cwde`, `cdq`, `movsx`, `movzx`) are in Chapter 5.

2.7 Chapter summary

> **IN SHORT**
> - Bits have no meaning by themselves; the instruction chooses signed or unsigned.
> - Two's complement of A on n bits = 2^n − A = invert + 1 = "keep up to the first 1, invert the rest".
> - MSB = 1: signed = unsigned − 2^n. MSB = 0: both readings are equal.
> - byte: 0...255 / −128...127; word: 0...65 535 / −32 768...32 767; dword: 0...2^{32}−1 / -2^{31}...2^{31}−1.
> - If −A is outside the signed range, use the next size.
> - Widen unsigned with zeros, signed with copies of the sign bit.

Solutions — Chapter 2

Exercise 2.1 — Both interpretations BASIC

STATEMENT

Give the unsigned and the signed interpretation (base 10) of: `7Fh`, `80h`, `0FEh`, `9Ch` (bytes), `8000h`, `0FFF0h` (words).

THEORY USED

Unsigned value by weights; signed = unsigned − 2^n when the MSB is 1 (2.3).

STEP BY STEP

pattern	MSB	unsigned	signed
7Fh	0	127	127 (same)
80h	1	128	128 − 256 = −128
0FEh	1	254	254 − 256 = −2
9Ch	1	156	156 − 256 = −100
8000h	1	32 768	32 768 − 65 536 = −32 768
0FFF0h	1	65 520	65 520 − 65 536 = −16

FINAL RESULT

7Fh: 127/127; 80h: 128/−128; FEh: 254/−2; 9Ch: 156/−100; 8000h: 32 768/−32 768; FFF0h: 65 520/−16.

Exercise 2.2 — Negative numbers on a byte BASIC

STATEMENT

Represent on a byte (in hexadecimal): −1, −128, −100, 100, −5.

THEORY USED

Negative −A on a byte = 256 − A (2.2).

STEP BY STEP

- −1 → 256 − 1 = 255 = **FFh**
- −128 → 256 − 128 = 128 = **80h**
- −100 → 256 − 100 = 156 = **9Ch**
- 100 → positive, written directly = **64h**
- −5 → 256 − 5 = 251 = **FBh**

FINAL RESULT

FFh, 80h, 9Ch, 64h, FBh.

Exercise 2.3 — Three methods for two's complement BASIC

STATEMENT

Compute the two's complement of `1001 0011b` and of `0101 1000b` with all three methods and say which

number each pattern and each result represents (signed).

THEORY USED

The three methods of section 2.2.

STEP BY STEP

1001 0011b

1. 1 0000 0000 − 1001 0011 = `0110 1101`
2. invert: `0110 1100`, +1 → `0110 1101`
3. keep up to the first 1 from the right (the last bit 1), invert the rest → `0110 1101`

1001 0011b = −109 (signed); its complement 0110 1101b = 6Dh = +109.

0101 1000b

1. 1 0000 0000 − 0101 1000 = `1010 1000`
2. invert: `1010 0111`, +1 → `1010 1000`
3. keep "1000" (up to the first 1 from the right), invert "0101" → 1010 → `1010 1000`

0101 1000b = 58h = +88; its complement 1010 1000b = A8h = 168 − 256 = −88.

FINAL RESULT

1001 0011b (−109) → 0110 1101b (+109); 0101 1000b (+88) → 1010 1000b (−88).

Exercise 2.4 — Reading a register BASIC

STATEMENT

`AL = 0C8h`. What is `AL` in signed and in unsigned base 10? And `AX = 0FF38h`?

THEORY USED

Unsigned value by weights; signed = unsigned − 2^n (2.3).

STEP BY STEP

- AL = C8h = 12·16 + 8 = **200** unsigned; MSB = 1 → signed 200 − 256 = **−56**.
- AX = FF38h = **65 336** unsigned; signed 65 336 − 65 536 = **−200**.

FINAL RESULT

AL: 200 unsigned, −56 signed. AX: 65 336 unsigned, −200 signed.

Exercise 2.5 — Negative numbers on a word and on a dword INTERMEDIATE

STATEMENT

Represent on a word: −200, −1, −129. Represent on a dword: −1, −200.

THEORY USED

−A on n bits = 2^n − A; sign extension (2.2, 2.6).

STEP BY STEP

On a word (2^{16} = 65 536):

- −200 → 65 536 − 200 = 65 336 = **FF38h**
- −1 → **FFFFh**

- −129 → 65 536 − 129 = 65 407 = **FF7Fh**

On a dword the value is the word sign-extended (the new high word is FFFFh):

- −1 → **FFFFFFFFh**
- −200 → **FFFFFF38h** (= 2^{32} − 200 = 4 294 967 096)

FINAL RESULT

Word: FF38h, FFFFh, FF7Fh. Dword: FFFFFFFFh, FFFFFF38h.

Exercise 2.6 — When a byte is not enough INTERMEDIATE

STATEMENT

Can −200 be represented on a byte? What does NASM store for `db -200`? Are `db 200` and `db -56` the same bytes?

THEORY USED

Signed byte range −128 ... 127; NASM keeps the low 8 bits of a constant that does not fit (1.8, 2.4).

STEP BY STEP

−200 is outside [−128, 127], so it **cannot** be a signed byte. On a word it is FF38h; for `db -200` NASM warns and keeps only the low byte **38h** (= 56) — a different number.

`db 200` stores C8h; `db -56` stores 256 − 56 = 200 = C8h. They are the **same byte**: 200 and −56 are the two interpretations of C8h.

FINAL RESULT

No (range −128...127). `db -200` stores 38h = 56. `db 200` and `db -56` both store C8h.

Exercise 2.7 — Sign and zero extension INTERMEDIATE

STATEMENT

Sign-extend and zero-extend `0A5h` to a word and to a dword. Do the same for `7Ch`. Give each result in hex and its value.

THEORY USED

Zero extension fills with 0; sign extension fills with copies of the MSB (2.6).

STEP BY STEP

0A5h = `1010 0101`b: MSB = 1 (unsigned 165, signed −91).

	word	dword	value
sign extension	FFA5h	FFFFFFA5h	−91
zero extension	00A5h	000000A5h	165

7Ch = `0111 1100`b: MSB = 0 (124 in both readings), so both extensions add zeros: **007Ch** and **0000007Ch** = 124.

FINAL RESULT

A5h → FFA5h / FFFFFFA5h (−91) signed, 00A5h / 000000A5h (165) unsigned; 7Ch → 007Ch / 0000007Ch both ways.

Exercise 2.8 — Minimum number of bits (signed) INTERMEDIATE

STATEMENT

What is the minimum number of bits needed to represent, **signed**, each of: −3, −8, −9, 8, 127, −128? Write −9 in binary on that many bits.

THEORY USED

Signed range on n bits: $-2^{n-1} \dots 2^{n-1} - 1$ (2.5).

STEP BY STEP

value	smallest n	signed range on n bits
−3	3	−4 … 3
−8	4	−8 … 7
−9	5	−16 … 15
8	5	−16 … 15 (on 4 bits the maximum is 7)
127	8	−128 … 127
−128	8	−128 … 127

−9 on 5 bits: $2^5 - 9 = 23 =$ **10111**b.

FINAL RESULT

3, 4, 5, 5, 8, 8 bits; −9 = 10111b on 5 bits.

Exercise 2.9 — True or false: instruction classes INTERMEDIATE

STATEMENT

True or false? Explain each. (a) `mov` interprets its operands as signed. (b) `add` gives different result bits for signed and unsigned operands. (c) `mul` treats its operands as unsigned. (d) `cmp` decides by itself whether a comparison is signed. (e) `movsx` should be used to widen an unsigned byte.

THEORY USED

Instruction classes (2.1).

STEP BY STEP

(a) **False** — `mov` only copies bits; it has no interpretation at all.

(b) **False** — `add` produces the same bits for both interpretations; only CF and OF describe them differently.

(c) **True** — `mul` is the unsigned multiplication (`imul` is the signed one).

(d) **False** — `cmp` is a subtraction that only sets flags; the **conditional jump** that follows (`ja` vs `jg` …) decides the interpretation.

(e) **False** — `movsx` does sign extension; an unsigned byte is widened with `movzx`.

FINAL RESULT

(a) F, (b) F, (c) T, (d) F, (e) F.

Exercise 2.10 — One addition, two meanings CHALLENGE

STATEMENT

Add the bytes `0C8h` and `64h`. Give the result byte, then interpret operands and result both as unsigned and as signed. Are both readings correct? Explain why the same `add` works for both.

THEORY USED

`add` is in the don't-care class (2.1); signed = unsigned − 256 (2.3).

STEP BY STEP

C8h + 64h = 12Ch → the byte keeps **2Ch** (the carry 1 does not fit).

	C8h	64h	result 2Ch
unsigned	200	100	44
signed	−56	100	44

- Signed: −56 + 100 = 44 ✓ correct.
- Unsigned: 200 + 100 = 300 does not fit in a byte; 2Ch = 44 = 300 − 256 × (the lost carry is the 256).

Both readings use the same bits because adding 2^n to an operand does not change the result modulo 2^n: the signed value −56 and the unsigned value 200 differ exactly by 256. The processor computes modulo 256 and reports in CF (unsigned) and OF (signed) whether the true result fit (Chapter 6).

FINAL RESULT

Result byte 2Ch = 44. Signed: −56 + 100 = 44, correct. Unsigned: 200 + 100 = 300 does not fit (CF = 1).

Exercise 2.11 — Taking the low byte of a word CHALLENGE

STATEMENT

A word holds `FF6Dh`. Give both interpretations. Then only its low byte `6Dh` is kept. What are the values now? Explain when taking the low byte of a signed word keeps its value.

THEORY USED

Signed = unsigned − 2^n (2.3); the "does −VAL fit" rule (2.5).

STEP BY STEP

FF6Dh = 65 389 unsigned, 65 389 − 65 536 = **−147** signed.

The low byte 6Dh = 109 in both interpretations (MSB 0). Neither −147 nor 65 389 survived: −147 does not fit in a signed byte, so the narrowing lost information.

Taking the low byte keeps the signed value exactly when the word is the **sign extension** of that byte, i.e. when the high byte is all copies of bit 7 of the low byte (00h with a low byte 00h...7Fh, or FFh with a low byte 80h...FFh). Here the high byte is FFh but bit 7 of 6Dh is 0, so the value changes.

FINAL RESULT

FF6Dh = 65 389 / −147; the byte 6Dh = 109 / 109. Narrowing is safe only if the word is the sign extension of its low byte.

Exercise 2.12 — Numbers equal to their own complement CHALLENGE

STATEMENT

Compute the two's complement of `80h` and of `00h` on a byte. What do you notice? What does this say about the signed range?

THEORY USED

Two's complement = invert + 1 (2.2); the range is not symmetric (2.4).

STEP BY STEP

- 80h = `1000 0000` → invert `0111 1111` → +1 → `1000 0000` = **80h**.
- 00h = `0000 0000` → invert `1111 1111` → +1 → 1 0000 0000 → on 8 bits **00h**.

Both patterns are their own complement. For 0 this is correct (−0 = 0). For 80h = −128 the true opposite +128 does not exist on a signed byte: the signed range −128 ... 127 has one more negative value than positive values, so negating −128 overflows and gives −128 back.

FINAL RESULT

Both give themselves (80h → 80h, 00h → 00h). −128 has no positive counterpart on a byte.

CHAPTER 3

x86 Architecture: Registers, Flags, Segments

What is inside an IA-32 processor from the programmer's point of view: the two units, the general registers and their parts, the segment registers, EIP and the flags register.

3.1 The big picture: EU and BIU

The IA-32 processor has two main parts that work in parallel:

- **EU** (Execution Unit) — contains the **ALU** (arithmetic and logic unit) and the **general registers**; it executes the instructions.
- **BIU** (Bus Interface Unit) — talks to memory: it **fetches** the next instructions and **brings or stores** the memory operands. It manages the segment registers and `EIP`.



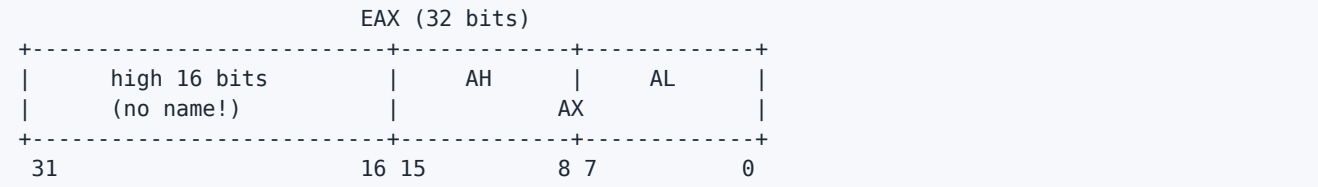
AT MOST ONE MEMORY OPERAND

The BIU can bring only **one** memory operand for an instruction. That is the reason behind the rule that an instruction cannot have two operands in memory: `add [a], [b]` does not exist; you write `mov al, [b]` and then `add [a], al`.

3.2 General-purpose registers

The 8 general registers have 32 bits. Some of their parts have names of their own:

32-bit	16-bit low	8-bit parts	Usual role
`EAX`	`AX`	`AH`, `AL`	accumulator: results of `mul` / `div`, return value of functions
`EBX`	`BX`	`BH`, `BL`	base: addressing memory
`ECX`	`CX`	`CH`, `CL`	counter: `loop`, shift counts (`CL`)
`EDX`	`DX`	`DH`, `DL`	data: high part of `mul` / `div`
`ESI`	`SI`	–	source index (strings)
`EDI`	`DI`	–	destination index (strings)
`EBP`	`BP`	–	base pointer (stack frames)
`ESP`	`SP`	–	stack pointer (top of the stack) – do not use it for data



WRITING A PART CHANGES ONLY THAT PART

`mov al, 5` changes **only** bits 0–7 of `EAX`; bits 8–31 keep their old value. `mov ax, 5` changes bits 0–15; `mov ah, 5` changes bits 8–15.

```
segment code use32 class=code
start:
    mov eax, 12345678h      ; EAX = 12345678h
    mov al, 0FFh            ; EAX = 123456FFh  (only bits 0-7 changed)
    mov ah, 0               ; EAX = 123400FFh  (only bits 8-15 changed)
    mov ax, 1               ; EAX = 12340001h  (bits 0-15 changed)
    mov ebx, eax
    shr ebx, 16             ; BX = 1234h: the high word of EAX moved down
```

THE HIGH WORD OF EAX HAS NO NAME

There is no register for bits 16–31 of `EAX`. To get them, shift (`shr eax, 16`, Chapter 7) or go through the stack or memory (Chapter 11).

The roles in the table are **conventions** and, for some instructions, **requirements**: `mul`/`div` always use `EAX`/`EDX`, `loop` always uses `ECX`, a variable shift count must be in `CL`, string instructions use `ESI`/`EDI`.

3.3 Segment registers and EIP

The six **segment registers** have 16 bits: `CS` (code), `DS` (data), `SS` (stack), `ES` (extra), and `FS`, `GS`. In 32-bit protected mode they hold **selectors** (not addresses) set by the operating system. Our programs never change them.

At any moment only one segment of each type is active. The pair **CS:EIP** is the full (FAR) address of the instruction being executed: `CS` selects the code segment, `EIP` is the offset inside it.

EIP (instruction pointer) holds the offset of the **next** instruction to execute. It is managed only by the BIU:

- after every instruction it advances by the length of that instruction;
- jumps, calls and returns load a new value into it.

You cannot write to it directly: `mov eip, eax` is a syntax error (`eip` is not an operand NASM knows).

3.4 The EFLAGS register

Flags are single bits in the `EFLAGS` register. Arithmetic and logic instructions **set** them; conditional jumps **read** them.

Flag	Name	Set to 1 when…
CF	Carry	an unsigned result does not fit (carry out of / borrow into the MSB)
OF	Overflow	a signed result does not fit (the sign of the result is wrong)
ZF	Zero	the result is 0
SF	Sign	the MSB of the result is 1 (the result is negative if read as signed)
PF	Parity	the **low byte** of the result has an **even** number of 1 bits
AF	Auxiliary carry	there is a carry from bit 3 to bit 4 (used for BCD arithmetic)
DF	Direction	control flag: 0 = string instructions go forward, 1 = backward
IF	Interrupt	control flag: interrupts enabled
TF	Trap	control flag: step-by-step debugging

Instructions that work directly with flags: `clc` / `stc` / `cmc` (CF = 0 / 1 / not CF), `cld` / `std` (DF = 0 / 1), `cli` / `sti` (IF), `pushf` / `popf`, `lahf` / `sahf`.

Which instructions change which flags

Instructions	Flags
`add`, `sub`, `adc`, `sbb`, `cmp`, `neg`	all of CF, OF, ZF, SF, PF, AF
`inc`, `dec`	the same **except CF** (CF unchanged)
`and`, `or`, `xor`, `test`	CF = OF = 0; ZF, SF, PF by the result
`mul`, `imul`	CF = OF = 1 if the high half of the result is significant
`mov`, `push`, `pop`, `lea`, `xchg`, `cbw` / `cwd` / `cwde` / `cdq`, `jmp`, `loop`, `not`	none
`div`, `idiv`	undefined (do not rely on them)

> **READING FLAGS IN OLLYDBG**
>
> OllyDbg shows the flags as C, P, A, Z, S, T, D, O next to the registers. After every arithmetic instruction, check them against your own prediction — it is the best way to learn them.

3.5 Computing the flags by hand

For an addition on *n* bits, do the operation in binary and then read each flag:

1. **Result**: keep the low *n* bits.
2. **CF**: was there a carry out of the MSB? (unsigned: is the true sum ≥ 2^n?)
3. **OF**: did two operands with the same sign give a result with the other sign? (Chapter 6)
4. **ZF**: is the kept result 0?
5. **SF**: the MSB of the kept result.
6. **PF**: count the 1 bits of the **low byte** of the result; even → PF = 1.
7. **AF**: add only the low nibbles; is the sum ≥ 10h?

Worked case: flags after `add`

```
segment code use32 class=code
start:
    mov al, 0F0h            ; 240 unsigned / -16 signed
    add al, 20h             ; AL = 10h   CF=1 OF=0 ZF=0 SF=0 PF=0 AF=0
```

F0h + 20h = 110h → AL keeps **10h**.

- CF = 1: unsigned 240 + 32 = 272 > 255.
- OF = 0: signed −16 + 32 = 16, correct and in range.
- ZF = 0 (10h ≠ 0), SF = 0 (MSB of `0001 0000` is 0).
- PF = 0: 10h has one bit 1 — odd.
- AF = 0: low nibbles 0 + 0 = 0, no carry from bit 3.

> **PARITY OF THE WHOLE RESULT**
>
> PF looks only at the **low byte** of the result, even for a 32-bit operation, and it is 1 for an **even** count (zero 1-bits counts as even).

3.6 Chapter summary

IN SHORT

- EU executes (ALU + registers); BIU fetches instructions and memory operands → at most one memory operand.
- EAX, EBX, ECX, EDX have 16-bit (AX…) and 8-bit (AH/AL…) parts; ESI, EDI, EBP, ESP only a 16-bit part.
- Writing a part of a register leaves the other bits unchanged; the high word of EAX has no name.
- CS:EIP = address of the current instruction; EIP cannot be written with `mov`.
- CF unsigned, OF signed, ZF zero, SF sign bit, PF even number of 1s in the low byte, AF carry 3→4.
- `inc` / `dec` keep CF; logic instructions clear CF and OF; `mov`, `lea`, `push`, `pop`, `loop` change no flags.

Solutions — Chapter 3

Exercise 3.1 — Which part of EAX changes? BASIC

STATEMENT

Which bits of `EAX` does each instruction change: `mov ah, 5`, `mov ax, 5`, `mov al, 5`, `mov eax, 5`?

THEORY USED

Register parts (3.2): writing a part changes only that part.

STEP BY STEP

instruction	bits changed	bits kept
`mov ah, 5`	8–15	0–7 and 16–31
`mov ax, 5`	0–15	16–31
`mov al, 5`	0–7	8–31
`mov eax, 5`	0–31	none

FINAL RESULT

8–15, 0–15, 0–7, all 32 bits respectively.

Exercise 3.2 — Parts of a register BASIC

STATEMENT

`EAX = 12345678h`. What are `AX`, `AH`, `AL`? Write instructions that put `1234h` into `BX`.

THEORY USED

Register parts (3.2); the high word has no name.

STEP BY STEP

AX = low 16 bits = **5678h**; AH = bits 8–15 = **56h**; AL = bits 0–7 = **78h**.

`1234h` is the high word, which has no name, so shift it down:

```
segment code use32 class=code
start:
    mov eax, 12345678h
    mov ebx, eax            ; EBX = 12345678h
    shr ebx, 16             ; EBX = 00001234h -> BX = 1234h
```

FINAL RESULT

AX = 5678h, AH = 56h, AL = 78h; `mov ebx, eax` / `shr ebx, 16` → BX = 1234h.

Exercise 3.3 — Registers with fixed roles BASIC

STATEMENT

Which register (or part) is used implicitly: (a) as the counter of `loop`; (b) as a variable shift count; (c) for the high half of a `mul` by a word; (d) as the source and destination index of string instructions; (e) as the top of the stack; (f) for the return value of a C function?

THEORY USED

Roles of the general registers (3.2).

STEP BY STEP

(a) `ECX`; (b) `CL`; (c) `DX` (the result is in `DX:AX`); (d) `ESI` (source) and `EDI` (destination); (e) `ESP`; (f) `EAX`.

FINAL RESULT

ECX, CL, DX, ESI/EDI, ESP, EAX.

Exercise 3.4 — Flags after 7Fh + 1 BASIC

STATEMENT

Give CF, OF, ZF, SF, PF and AF after `mov al, 7Fh` / `add al, 1`.

THEORY USED

Computing the flags by hand (3.5).

STEP BY STEP

`0111 1111` + 1 = `1000 0000` = 80h.

- CF = 0: unsigned 127 + 1 = 128 ≤ 255 fits.
- OF = 1: signed 127 + 1 = 128 does not fit in [−128, 127]; the result reads −128 (positive + positive = negative).
- ZF = 0; SF = 1 (MSB of 80h).
- PF = 0: 80h has one bit 1 (odd).
- AF = 1: low nibbles F + 1 = 10h → carry from bit 3.

FINAL RESULT

AL = 80h; CF = 0, OF = 1, ZF = 0, SF = 1, PF = 0, AF = 1.

Exercise 3.5 — Flags after 80h + 80h INTERMEDIATE

STATEMENT

Give AL and the flags CF, OF, ZF, SF, PF after `mov al, 80h` / `add al, 80h`.

THEORY USED

Computing the flags by hand (3.5).

STEP BY STEP

80h + 80h = 100h → AL = **00h**.

- CF = 1: unsigned 128 + 128 = 256 > 255.
- OF = 1: signed −128 + (−128) = −256 does not fit (negative + negative gave 0, not negative).
- ZF = 1 (the kept result is 0); SF = 0.
- PF = 1: zero bits 1 is an even count.

FINAL RESULT

AL = 00h; CF = 1, OF = 1, ZF = 1, SF = 0, PF = 1.

Exercise 3.6 — Trace EAX through writes to its parts INTERMEDIATE

STATEMENT

Give `EAX` after each line:

```
mov eax, 0FFFFFFFFh
mov al, 0
mov ah, 12h
mov ax, 1
mov al, -1
```

THEORY USED

Writing a part changes only that part (3.2).

STEP BY STEP

```
segment code use32 class=code
start:
    mov eax, 0FFFFFFFFh     ; EAX = FFFFFFFFh
    mov al, 0               ; EAX = FFFFFF00h
    mov ah, 12h             ; EAX = FFFF1200h
    mov ax, 1               ; EAX = FFFF0001h
    mov al, -1              ; EAX = FFFF00FFh  (-1 on a byte = FFh)
```

FINAL RESULT

FFFFFFFFh → FFFFFF00h → FFFF1200h → FFFF0001h → FFFF00FFh.

Exercise 3.7 — inc versus add INTERMEDIATE

STATEMENT

Compare the two sequences. Give AL, CF and ZF at the end of each and explain the difference.

```
stc                 ; sequence A        stc             ; sequence B
mov al, 0FFh                            mov al, 0FFh
inc al                                  add al, 1
```

THEORY USED

`inc` / `dec` do not change CF (3.4).

STEP BY STEP

Both compute FFh + 1 = 100h → AL = **00h**, ZF = **1**.

- A: `inc` leaves CF unchanged; it was set by `stc` → **CF = 1** (but only because of `stc`, not because of the wrap-around).
- B: `add` sets CF from the operation: 255 + 1 = 256 does not fit → **CF = 1**.

To see the difference, replace `stc` by `clc`: then A ends with CF = 0 (the wrap-around is invisible in CF) while B still gives CF = 1.

```
segment code use32 class=code
start:
    clc
    mov al, 0FFh
    inc al                  ; AL = 00h, ZF = 1, CF = 0 (unchanged)
    clc
    mov bl, 0FFh
    add bl, 1               ; BL = 00h, ZF = 1, CF = 1
```

Consequence: you cannot detect an unsigned wrap-around after `inc` with `jc`; but inside an `adc` chain you may use `inc`/`dec` without destroying the carry.

FINAL RESULT

Both: AL = 0, ZF = 1. CF stays as before after `inc` (here 1 from `stc`), CF = 1 from the overflow after `add`.

Exercise 3.8 — Two memory operands INTERMEDIATE

STATEMENT

Why is `add [a], [b]` invalid? Rewrite it for two byte variables `a` and `b`, keeping the result in `a`.

THEORY USED

The BIU brings one memory operand per instruction (3.1).

STEP BY STEP

Both operands would come from memory, but the BIU can bring only one memory operand per instruction. Pass the second value through a register:

```
segment data use32 class=data
    a db 10
    b db 25
segment code use32 class=code
start:
    mov al, [b]             ; AL = 25
    add [a], al             ; a = 35
```

FINAL RESULT

Two memory operands are impossible; use `mov al, [b]` then `add [a], al` (a = 35).

Exercise 3.9 — Parity and auxiliary carry INTERMEDIATE

STATEMENT

(a) Give PF for the results `10h`, `9Ah`, `0FFh`, `00h`, `1234h` (word). (b) Give AF for `0Fh + 01h`, `18h + 08h` and `22h + 11h`.

THEORY USED

PF: even number of 1s in the low byte; AF: carry from bit 3 to bit 4 (3.4, 3.5).

STEP BY STEP

(a)

result	low byte	number of 1s	PF
10h	0001 0000	1	0
9Ah	1001 1010	4	1
0FFh	1111 1111	8	1
00h	0000 0000	0	1
1234h	34h = 0011 0100	3	0

(b) Add the low nibbles: F + 1 = 10h → **AF = 1**; 8 + 8 = 10h → **AF = 1**; 2 + 1 = 3 → **AF = 0**.

FINAL RESULT

PF: 0, 1, 1, 1, 0. AF: 1, 1, 0.

Exercise 3.10 — A full flag trace CHALLENGE

STATEMENT

Give AL and the flags CF, OF, ZF, SF after each instruction:

```
mov al, 0F0h
add al, 20h
sub al, 11h
inc al
and al, 0Fh
```

THEORY USED

Flags of `add` / `sub` (3.4, 3.5); `inc` keeps CF; `and` clears CF and OF; `mov` changes no flags.

STEP BY STEP

```
segment code use32 class=code
start:
    mov al, 0F0h            ; AL = F0h  (flags unchanged)
    add al, 20h             ; AL = 10h  CF=1 OF=0 ZF=0 SF=0
    sub al, 11h             ; AL = FFh  CF=1 OF=0 ZF=0 SF=1
    inc al                  ; AL = 00h  CF=1 OF=0 ZF=1 SF=0
    and al, 0Fh             ; AL = 00h  CF=0 OF=0 ZF=1 SF=0
```

- `add`: 240 + 32 = 272 → carry (CF = 1); −16 + 32 = 16 fine (OF = 0).
- `sub`: 10h − 11h: unsigned 16 − 17 needs a borrow → CF = 1; signed 16 − 17 = −1 fits → OF = 0; FFh has MSB 1 → SF = 1.
- `inc`: FFh + 1 = 00h → ZF = 1, SF = 0; signed −1 + 1 = 0 → OF = 0; **CF stays 1** (from `sub`).
- `and`: 00h AND 0Fh = 00h; logic instructions force CF = OF = 0; ZF = 1.

FINAL RESULT

AL: F0h → 10h → FFh → 00h → 00h; final CF = 0, OF = 0, ZF = 1, SF = 0.

Exercise 3.11 — CS:EIP and how EIP changes CHALLENGE

STATEMENT

(a) What does `CS:EIP` contain at run time? (b) Why is `mov eip, eax` rejected? (c) List three ways in which `EIP` changes during execution.

THEORY USED

Segment registers and EIP (3.3).

STEP BY STEP

(a) The full (FAR) address of the instruction being executed: `CS` holds the **selector** of the code segment, `EIP` the **offset** of the instruction inside it.

(b) `EIP` is handled only by the BIU and is not an operand of any data-transfer instruction; NASM does not even know the name `eip` → syntax error.

(c) It advances automatically by the length of each executed instruction; a jump (`jmp`, conditional jumps, `loop`) loads the target offset; `call` loads the procedure address (after pushing the return address) and `ret` loads the popped return address.

FINAL RESULT

(a) selector : offset of the current instruction; (b) EIP is not writable with `mov`; (c) sequential advance, jumps, call/ret.

Exercise 3.12 — Flags of subtraction and comparison CHALLENGE

STATEMENT

Give the result (if any) and CF, ZF, SF, OF after: (a) `mov al, 0` / `sub al, 1`; (b) `mov al, 5` / `cmp al, 5`; (c) `mov al, 3` / `cmp al, 0FDh`.

THEORY USED

`sub` and `cmp` set all arithmetic flags; `cmp` does not store the result (3.4); CF on subtraction = borrow.

STEP BY STEP

```
segment code use32 class=code
start:
    mov al, 0
    sub al, 1               ; AL = FFh  CF=1 ZF=0 SF=1 OF=0
    mov bl, 5
    cmp bl, 5               ; BL = 5 (unchanged)  CF=0 ZF=1 SF=0 OF=0
    mov cl, 3
    cmp cl, 0FDh            ; 3 - 253: CF=1 ZF=0 SF=0 OF=0
```

(a) 0 − 1: unsigned needs a borrow → CF = 1; result FFh = −1 signed, which is correct → OF = 0; SF = 1.

(b) 5 − 5 = 0 → ZF = 1, no borrow → CF = 0; `cmp` does not change AL.

(c) 3 − FDh: unsigned 3 − 253 needs a borrow → CF = 1. The difference byte is 06h (SF = 0, ZF = 0). Signed it is 3 − (−3) = 6, correct → OF = 0.

FINAL RESULT

(a) AL = FFh, CF = 1, ZF = 0, SF = 1, OF = 0; (b) CF = 0, ZF = 1, SF = 0, OF = 0; (c) CF = 1, ZF = 0, SF = 0, OF = 0.

PART II

Programming in NASM

Program structure, data definition, arithmetic, overflow and bit manipulation.

CHAPTER 4

Assembly Basics: Program Structure, Data Definition, Directives, Operators

How a NASM program is built: the skeleton, the elements the assembler works with, defining data in memory (little endian!), the location counter, EQU constants and the operators evaluated at assembly time.

4.1 The program skeleton

Every program in this course has the same frame. Learn it by heart — you will write it hundreds of times.

```
bits 32                     ; assemble for the 32-bit architecture
global start                ; make the label 'start' visible = the entry point
extern exit                 ; exit is defined somewhere else...
import exit msvcrt.dll      ; ...namely in msvcrt.dll (the C runtime library)

segment data use32 class=data
    ; variables go here
    a db 10

segment code use32 class=code
start:
    ; instructions go here
    mov al, [a]             ; AL = 10
    push dword 0            ; parameter of exit: status code 0
    call [exit]             ; exit(0) - ends the program
```

- `segment data …` and `segment code …` open the data segment and the code segment.
- `start:` is a **label**: the address of the first instruction. `global start` tells the linker that execution begins there (`-entry start`).
- `extern` + `import` declare a function that comes from a DLL; `call [exit]` calls it (the brackets are explained in Chapter 12).
- Without the final `exit` call the processor would continue with whatever bytes follow the code and the program would crash.

Building and running

```
nasm -fobj prog.asm                                  -> prog.obj   (assembler)
alink -oPE -subsys console -entry start prog.obj     -> prog.exe   (linker)
```

Open `prog.exe` in **OllyDbg**: the code window shows the address, the **machine-code bytes** and the instruction; the register window shows the registers and flags; the dump window shows memory. F8 executes one instruction (stepping over calls), F7 steps into a call, F9 runs. To see your variables, follow the address of `a` in the dump.

4.2 The elements an assembler works with

A NASM source file is made of four kinds of elements:

1. **Labels** — names for addresses: *data labels* (variables, e.g. `a`) and *code labels* (jump targets, e.g. `start`, `again`).
2. **Instructions** — mnemonics (`mov`, `add`, ...) for which the assembler **generates machine bytes** that the CPU executes at run time.
3. **Directives** — orders for the **assembler** itself; they generate no instruction: `segment`, `db` / `dw` / `dd` / `dq`, `resb` ..., `times`, `equ`, `global`, `extern`, `import`, `bits`.
4. The **location counter** `$` — the offset where the current line starts = the number of bytes generated so far in the current segment. `$$` is the start of the current segment.

The format of a source line is

```
[label[:]]  [prefixes]  [mnemonic]  [operands]   [; comment]
```

Every part is optional. Rules for names:

- labels may contain letters, digits and `_ $ # @ ~ . ?`, and must start with a letter, `_` or `?`;
- **user names are case sensitive** (`Abc` ≠ `abc`); keywords, mnemonics and register names are not (`MOV EAX` = `mov eax`).

> **ASSEMBLY TIME VERSUS RUN TIME**
>
> Directives and operators are processed when NASM **assembles** the file; instructions are executed by the CPU when the program **runs**. Keeping the two moments apart explains most of the rules in this chapter.

4.3 Name versus [name]

A variable name is an **address** (an offset in the data segment). Square brackets mean "the memory **at** that address".

```
segment data use32 class=data
    a dd 5
segment code use32 class=code
start:
    mov eax, a              ; EAX = the ADDRESS (offset) of a
    mov ebx, [a]            ; EBX = the VALUE stored at a (4 bytes) = 5
    lea ecx, [a]            ; ECX = the address of a (same as mov ecx, a)
```

> **NASM DOES NOT REMEMBER THE TYPE OF A VARIABLE**
>
> `a db 10` only reserves one byte at address `a`. `mov ax, [a]` will happily read **two** bytes (a and the next one). The size of a memory operand always comes from the **other operand** or from a **type operator**: `byte`, `word`, `dword`, `qword`.

```
mov [v], 0              ; ERROR: operation size not specified
mov byte [v], 0         ; OK
mul [c]                 ; ERROR - same reason
mul byte [c]            ; OK
push [x]                ; ERROR -> push dword [x]
mov [v], al             ; OK: AL fixes the size (byte)
```

4.4 Data definition directives

Directive	Element size	Reserve without value
`db`	byte (1)	`resb n`
`dw`	word (2)	`resw n`
`dd`	dword (4)	`resd n`
`dq`	qword (8)	`resq n`
`dt`	10 bytes	`rest n`

```
segment data use32 class=data
    a db 23                 ; 17
    b dw 23ABh              ; AB 23  (little endian!)
    c dd -101               ; 9B FF FF FF
    s db 'abc', 0           ; 61 62 63 00
    v dw 1, 2, 3            ; 01 00 02 00 03 00
    t times 4 db 0          ; 00 00 00 00
    u times 3 dw 1, 2       ; 01 00 02 00 repeated 3 times (12 bytes)
    buf resb 8              ; 8 bytes, no initial value (zero in practice)
segment code use32 class=code
start:
    mov ax, [b]             ; AX = 23ABh
```

`times n <definition>` repeats the definition *n* times. The values in a list are placed one after another, each with the element size of the directive.

Little endian (very important)

In memory, a value of more than one byte is stored with the **least significant byte at the lowest address**. In registers you always read it "normally".

```
x dd 12345678h       ; memory, increasing addresses:  78 56 34 12
y dw -2              ; FFFEh                       ->  FE FF
```

So, for `x` above: `word [x]` is the low word 5678h; `word [x+2]` is the high word 1234h; `byte [x+3]` is 12h.

> **READING MEMORY LEFT TO RIGHT AS A NUMBER**
>
> The dump `78 56 34 12` is **not** the number 78563412h. Reverse the bytes of each element to get its value: 12345678h.

Strings with dw and dd

Characters of a string constant are placed **in order**, and the last element is padded with zeros up to the element size:

```
segment data use32 class=data
    w1 dw 'ab'              ; 61 62        (one word: 6261h)
    w2 dw 'abc'             ; 61 62 63 00  (two words)
    d1 dd 'ab'              ; 61 62 00 00
    w3 dw 'a', 'b'          ; 61 00 62 00  (two separate words)
segment code use32 class=code
start:
    mov ax, [w1]            ; AX = 6261h
```

What you cannot write in a data definition

```
x db [a]                ; ERROR - no memory access at assembly time
y dd eax                ; ERROR - registers do not exist at assembly time
z db a                  ; ERROR (obj format) - an address does not fit in a byte
w dd !a                 ; ERROR - '!' needs a scalar, 'a' is an address
q dw times 4 -128       ; ERROR - TIMES must come before the directive
```

The difference of two addresses of the same segment (`a2-a5`) is a plain number (a **scalar**), so `~(a2-a5)` and `!(a2-a5)` are allowed.

4.5 The location counter $ and lengths

`$` is the offset of the beginning of the current line. Right after a definition, `$` minus the name of the definition is the number of bytes it generated — the classic way to compute lengths:

```
segment data use32 class=data
    s db 'hello'
    len equ $ - s           ; 5 (bytes)
    v dw 1, 2, 3, 4
    lv equ ($ - v) / 2      ; 4 (words)
    d dd 10, 20, 30
    ld equ ($ - d) / 4      ; 3 (dwords)
segment code use32 class=code
start:
    mov eax, len            ; EAX = 5
    mov ebx, lv             ; EBX = 4
    mov ecx, ld             ; ECX = 3
```

SOMETHING BETWEEN THE DATA AND THE EQU

`len equ $ - s` must come **immediately** after `s`. Any variable defined in between is counted too.

To get the number of **elements** instead of bytes, divide by the element size (2 for words, 4 for dwords).

4.6 EQU constants

`name equ expression` gives a name to a value computed at assembly time. It **reserves no memory**: wherever the name appears, NASM writes the number.

```
n equ 5
mov ecx, n              ; OK: ECX = 5
mov ecx, [n]            ; WRONG: reads memory at address 5 - not your memory, the program crashes
```

EQU names are useful for lengths (`len equ $-s`), for constants used in several places and for readability (`STDIN equ 0`).

4.7 Operators (evaluated at assembly time)

Operators work on constants known at assembly time; instructions work at run time. The `+` in `[ebx+4]` is part of an address formula, while `add` is an instruction.

Operator	Meaning	Example
`+ - * /`	arithmetic (`/` unsigned), `//` signed division, `%` modulo	`(1000/4+1)`
`<< >>`	shift left / right	`01110111b << 3` = 3B8h
`~`	bitwise NOT	`~0F0h`
`& | ^`	bitwise AND, OR, XOR	`5|6` = 7
`!`	logical NOT (1 if the operand is 0, else 0) — scalars only	`!0` = 1
`byte word dword qword`	type operators	`mov byte [v], 0`

Precedence (from high to low): unary (`-` `~` `!`), then `* / // % %%`, then `+ -`, then `<< >>`, then `&`, then `^`, then `|`.

```
5|6 + 7&8  =  5 | ((6+7) & 8)  =  5 | (13 & 8)  =  5 | 8  =  13
```

Results are computed on 64 bits and then **truncated** to the destination size:

```
segment data use32 class=data
    a5 db 128 >> 1, -128 << 1   ; 40h, 00h  (-256 = ...FF00h -> low byte 00h)
    a6 db 5|6 + 7&8             ; 0Dh = 13
segment code use32 class=code
start:
    mov ax, 01110111b << 3      ; AX = 3B8h - computed by NASM, no shift instruction
```

OPERATORS ARE NOT INSTRUCTIONS

`mov ax, bx << 2` is an error: `<<` can only combine constants, and BX has no value at assembly time. To shift a register at run time use the instruction `shl` (Chapter 7).

4.8 Worked case: a full data segment

Starting at offset 0, write the memory content generated by:

```
segment data use32 class=data
    a1 db 'ab', 300-256     ; offset 0: 61 62 2C
    a2 dw a1-$, 1           ; offset 3: a1-$ = 0-3 = -3 -> FD FF, then 01 00
    a3 dd 5 >> 1            ; offset 7: 02 00 00 00
    a4 equ 10               ; no bytes
    a5 times 2 db a4*2      ; offset 11: 14 14
segment code use32 class=code
start:
    mov ax, [a2]            ; AX = FFFDh
```

Method: keep a running offset; for each line write the bytes in little endian and add their count to the offset. `$` inside `a2`'s line is 3 (the offset where that line starts).

Memory (offsets 0...12): `61 62 2C FD FF 01 00 02 00 00 00 14 14`.

4.9 Chapter summary

IN SHORT

- Skeleton: `bits 32`, `global start`, `extern` / `import exit`, data segment, code segment, `start:`, `push dword 0` + `call [exit]`.
- Build: `nasm -fobj`, `alink -oPE -subsys console -entry start`, debug in OllyDbg.
- Elements: labels, instructions (run time), directives (assembly time), location counter `$`.
- `a` = address, `[a]` = value; NASM does not remember sizes → `byte/word/dword`.
- `db dw dd dq dt`, `resb…`, `times n …`; strings padded to the element size.
- Little endian: lowest byte at the lowest address.
- `len equ $ - s` immediately after `s`; EQU reserves no memory (`mov ecx, n`, never `[n]`).
- Operators (`<< >> & | ^ ~ ! + - * / %`) work only on constants; results are truncated to the destination.

Solutions — Chapter 4

Exercise 4.1 — Memory content of a data segment BASIC

STATEMENT

Write the memory content (hex, starting at offset 0) generated by the lines below, and the value of `lx`.

```
x db 1, -1, 'A'
y dw 1234h, -2
z dd 'ab', 100h
lx equ y - x
```

THEORY USED

Data definition, little endian, strings padded to the element size (4.4); EQU reserves nothing (4.6).

STEP BY STEP

Offsets: `x` at 0 (3 bytes), `y` at 3 (2 words = 4 bytes), `z` at 7 (2 dwords = 8 bytes).

offset	bytes	explanation
0	01 FF 41	1, −1 = FFh, 'A' = 41h
3	34 12 FE FF	1234h little endian, −2 = FFFEh
7	61 62 00 00 00 01 00 00	'ab' padded to a dword, 100h = 00000100h

`lx = y − x = 3 − 0 = 3`; it generates no bytes.

```
segment data use32 class=data
    x db 1, -1, 'A'
    y dw 1234h, -2
    z dd 'ab', 100h
    lx equ y - x
segment code use32 class=code
start:
    mov eax, lx             ; EAX = 3
```

FINAL RESULT

`01 FF 41 34 12 FE FF 61 62 00 00 00 01 00 00`; lx = 3.

Exercise 4.2 — Find the errors BASIC

STATEMENT

What is wrong in each line? `a db [b]`, `b dd ebx`, `c dw times 2 5`, `d db 1000`.

THEORY USED

What cannot appear in a data definition (4.4); constants are truncated (4.7).

STEP BY STEP

- `a db [b]` — no memory access at assembly time: the content of `b` is not known when NASM assembles.
- `b dd ebx` — registers do not exist at assembly time.
- `c dw times 2 5` — `times` must come **before** the directive: `c times 2 dw 5`.
- `d db 1000` — 1000 does not fit in a byte; NASM warns and stores 1000 mod 256 = 232 = **E8h**.

FINAL RESULT

Memory access and registers are impossible in data definitions; write `c times 2 dw 5`; `db 1000` stores E8h with a warning.

Exercise 4.3 — Lengths with $ BASIC

STATEMENT

Given `s db 'abcdef'` followed by `n equ $-s`, what is `n`? What if `db 0` sits between them? What is `m` for `w dw 1, 2, 3` followed by `m equ ($-w)/2`?

THEORY USED

The location counter (4.5).

STEP BY STEP

- `$` after `s` is 6 bytes later → **n = 6**.
- With an extra `db 0` in between, `$` has advanced one more byte → **n = 7** (the extra byte is counted).
- `w` generates 3 words = 6 bytes → 6 / 2 = **m = 3** elements.

FINAL RESULT

n = 6; n = 7 with the extra byte; m = 3.

Exercise 4.4 — Reading an element of an array BASIC

STATEMENT

`v dw 10, 20, 30`. Write the instruction that puts the third element into `AX`. What would `mov ax, [v+2]` give?

THEORY USED

Element *i* (from 0) of a word array is at `v + 2·i` (4.4).

STEP BY STEP

Words take 2 bytes: offsets v+0 (10), v+2 (20), v+4 (30). The third element is at `v+4`:

```
segment data use32 class=data
    v dw 10, 20, 30
segment code use32 class=code
start:
    mov ax, [v+4]           ; AX = 30
    mov bx, [v+2]           ; BX = 20 (the second element)
```

FINAL RESULT

`mov ax, [v+4]` → AX = 30; `[v+2]` gives 20.

Exercise 4.5 — Operators INTERMEDIATE

STATEMENT

Compute (in hex): `0F0h & 3Ch`, `~0 & 0FFh`, `1 << 4 | 1`, `12 / 5`, `12 % 5`, `-7 // 2`, `0F0h ^ 0FFh`.

THEORY USED

Operators and precedence (4.7).

STEP BY STEP

- 0F0h & 3Ch = **1111** 0000 & 0011 **1100** = 0011 0000 = **30h**
- ~0 & 0FFh = (all ones) & FFh = **0FFh**
- 1 << 4 | 1: `<<` has higher precedence than `|` → 10h | 1 = **11h**
- 12 / 5 = **2** (integer division); 12 % 5 = **2**
- −7 // 2 = **−3** (signed division truncates toward zero) = FFFF...FDh
- 0F0h ^ 0FFh = **0Fh**

FINAL RESULT

30h, 0FFh, 11h, 2, 2, −3, 0Fh.

Exercise 4.6 — Little endian views INTERMEDIATE

STATEMENT

`a dd 11223344h`. What are `byte [a+1]`, `word [a+1]`, `word [a+2]` and `dword [a]`?

THEORY USED

Little endian (4.4).

STEP BY STEP

Memory from `a` upwards: **44 33 22 11**.

- `byte [a+1]` = 33h
- `word [a+1]` = bytes 33, 22 → **2233h**
- `word [a+2]` = bytes 22, 11 → **1122h**
- `dword [a]` = 11223344h

```
segment data use32 class=data
    a dd 11223344h
segment code use32 class=code
start:
    mov al, [a+1]           ; AL = 33h
    mov bx, [a+1]           ; BX = 2233h
    mov cx, [a+2]           ; CX = 1122h
```

FINAL RESULT

33h, 2233h, 1122h, 11223344h.

Exercise 4.7 — EQU in brackets INTERMEDIATE

STATEMENT

Why does `mov eax, [n]` not work when `n equ 5`? What is the correct instruction?

THEORY USED

EQU constants reserve no memory (4.6).

STEP BY STEP

`n` is not a variable; it is the number 5. `[n]` means "the value at address 5", which is not memory of the program → access violation at run time. Use the number itself: `mov eax, n` (EAX = 5).

FINAL RESULT

`[n]` reads address 5 and crashes; write `mov eax, n`.

Exercise 4.8 — Another full data segment INTERMEDIATE

STATEMENT

Starting at offset 0, write the bytes generated and give the offset of every label:

```
a1 dw 'xy', 'z'
a2 times 3 db 0FFh
a3 dd a2 - a1
a4 db -1, 255, 1 << 7
a5 dw $ - a1
```

THEORY USED

Data definition, strings with dw, `$`, little endian (4.4, 4.5, 4.8).

STEP BY STEP

label	offset	bytes	explanation
a1	0	78 79 7A 00	'xy' = one word, 'z' = one word padded with 00
a2	4	FF FF FF	times 3
a3	7	04 00 00 00	a2 − a1 = 4 − 0 = 4
a4	11	FF FF 80	−1 = FFh, 255 = FFh, 1<<7 = 80h
a5	14	0E 00	`$` = 14 at this line, 14 − 0 = 14 = 0Eh

```
segment data use32 class=data
    a1 dw 'xy', 'z'
    a2 times 3 db 0FFh
    a3 dd a2 - a1
    a4 db -1, 255, 1 << 7
    a5 dw $ - a1
segment code use32 class=code
start:
    mov eax, [a3]           ; EAX = 4
    mov bx, [a5]            ; BX = 14
```

FINAL RESULT

`78 79 7A 00 FF FF FF 04 00 00 00 FF FF 80 0E 00` (16 bytes); offsets a1 = 0, a2 = 4, a3 = 7, a4 = 11, a5 = 14.

Exercise 4.9 — Where is a size specifier needed? INTERMEDIATE

STATEMENT

For each instruction, say whether it is valid as written; if not, correct it (`v` is a byte variable, `x` a dword variable, `c` a byte variable): `mov [v], 0` · `inc [v]` · `mov [v], al` · `push [x]` · `mul [c]` · `mov eax, [x]` · `add [x], 5`

THEORY USED

NASM does not remember sizes (4.3).

STEP BY STEP

instruction	valid?	correction
`mov [v], 0`	no — size unknown	`mov byte [v], 0`
`inc [v]`	no	`inc byte [v]`
`mov [v], al`	yes — AL fixes the size	—
`push [x]`	no	`push dword [x]`
`mul [c]`	no	`mul byte [c]`
`mov eax, [x]`	yes — EAX fixes the size	—
`add [x], 5`	no	`add dword [x], 5`

The rule: when no register operand is present, a memory operand needs `byte`, `word` or `dword`.

FINAL RESULT

Valid: `mov [v], al`, `mov eax, [x]`. All others need a size: byte/byte/dword/byte/dword.

Exercise 4.10 — Precedence and truncation CHALLENGE

STATEMENT

Compute the bytes generated by `t db 5|6 + 7&8, 2+3*4<<1, -128<<1, ~0F0h, 300 % 256` and explain each.

THEORY USED

Precedence and truncation of operators (4.7).

STEP BY STEP

- `5|6 + 7&8` = 5 | ((6+7) & 8) = 5 | 8 = 13 = **0Dh**
- `2+3*4<<1`: `*` first → 12; `+` → 14; `<<` has lower precedence than `+` → 14 << 1 = 28 = **1Ch**
- `-128<<1` = −256 = ...FF00h → low byte **00h**
- `~0F0h` = ...FFFF0Fh → low byte **0Fh**
- `300 % 256` = 44 = **2Ch**

```
segment data use32 class=data
    t db 5|6 + 7&8, 2+3*4<<1, -128<<1, ~0F0h, 300 % 256
segment code use32 class=code
start:
    mov eax, [t]            ; EAX = 0F001C0Dh (bytes 0D 1C 00 0F, little endian)
```

FINAL RESULT

`0D 1C 00 0F 2C`.

Exercise 4.11 — A complete program with lengths CHALLENGE

STATEMENT

Write a complete program that defines: the string `s = 'ASC 2026'` and its length `ls`; the word array `v` with the values 1...5 and its number of elements `nv`; a reserved area `r` of 10 dwords; and two dword variables `lens`, `cnt`. The code must store `ls` into `lens` and `nv` into `cnt`, then exit.

THEORY USED

Skeleton (4.1); `$` and EQU (4.5, 4.6); `resd` (4.4); size of a memory destination (4.3).

STEP BY STEP

```
bits 32
global start
extern exit
import exit msvcrt.dll

segment data use32 class=data
    s db 'ASC 2026'
    ls equ $ - s            ; 8 bytes
    v dw 1, 2, 3, 4, 5
    nv equ ($ - v) / 2      ; 5 words
    r resd 10               ; 40 bytes reserved
    lens dd 0
    cnt dd 0

segment code use32 class=code
start:
    mov dword [lens], ls    ; lens = 8   (the constant has no size -> dword)
    mov dword [cnt], nv     ; cnt = 5
    push dword 0
    call [exit]
```

Each EQU is written right after its definition, so `$` measures exactly that definition.

FINAL RESULT

lens = 8, cnt = 5 (program above).

Exercise 4.12 — Reading a memory dump CHALLENGE

STATEMENT

The memory starting at label `x` contains the bytes `78 56 34 12 FE FF`. What are `dword [x]`, `word [x+4]` (signed and unsigned), `byte [x+1]` and `word [x+3]`?

THEORY USED

Little endian (4.4); signed interpretation (Chapter 2).

STEP BY STEP

- `dword [x]` = bytes 78 56 34 12 reversed = **12345678h**
- `word [x+4]` = FE FF → **FFFEh** = 65 534 unsigned, **−2** signed
- `byte [x+1]` = **56h**
- `word [x+3]` = bytes 12 FE → **FE12h**

```
segment data use32 class=data
    x db 78h, 56h, 34h, 12h, 0FEh, 0FFh
segment code use32 class=code
start:
    mov eax, [x]            ; EAX = 12345678h
    mov bx, [x+4]           ; BX = FFFEh
    mov cl, [x+1]           ; CL = 56h
    mov dx, [x+3]           ; DX = FE12h
```

FINAL RESULT

12345678h; FFFEh = 65 534 / −2; 56h; FE12h.

CHAPTER 5

Arithmetic: ADD/SUB, MUL/DIV, Type Conversions, Expressions

The arithmetic instructions of x86, numbers wider than one register, multiplication and division with their implicit operands, widening conversions, and a reliable method for computing any expression with mixed sizes.

5.1 Rules for two-operand instructions

Instructions such as `mov`, `add`, `sub`, `cmp`, `and` have a **destination** and a **source**: `add d, s` means d = d + s. Three rules always hold:

1. Both operands have the **same size** (byte + byte, word + word, dword + dword).
2. At most **one** operand is in memory.
3. The destination cannot be a constant.

```
add ax, bx              ; OK
add [a], [b]            ; ERROR - two memory operands
add ax, bl              ; ERROR - different sizes (16 and 8 bits)
add 5, al               ; ERROR - the destination is a constant
add eax, [a]            ; OK: reads a DWORD at a, whatever a was declared as!
```

"ADD EAX, [A]" WITH A BYTE VARIABLE

If `a db 3`, then `add eax, [a]` still reads 4 bytes starting at `a` — the 3 and the next three bytes of memory. To add the byte `a` to EAX you must first widen it (section 5.6): `movzx ebx, byte [a]` then `add eax, ebx`.

5.2 Addition and subtraction

Instruction	Effect	Flags
`add d, s`	d = d + s	all
`sub d, s`	d = d − s	all
`adc d, s`	d = d + s + CF	all
`sbb d, s`	d = d − s − CF	all
`inc d` / `dec d`	d = d ± 1	all except CF
`neg d`	d = 0 − d (two's complement)	all
`cmp d, s`	computes d − s, keeps only the flags	all

These instructions belong to the "don't care" class (Chapter 2): the same instruction serves signed and unsigned values; CF describes the unsigned result and OF the signed one (Chapter 6).

```
segment code use32 class=code
start:
    mov al, 10
    add al, 25              ; AL = 35
    sub al, 40              ; AL = FBh  (-5 signed; CF = 1: unsigned 35 < 40)
    neg al                  ; AL = 05h
    mov bl, 7
    cmp bl, 7               ; BL unchanged, ZF = 1
    dec bl                  ; BL = 6
```

5.3 Numbers wider than one register

A qword (64 bits) does not fit in one 32-bit register; we keep it in a **pair** such as `EDX:EAX` (EDX = high dword). To add two such numbers, add the **low** parts with `add`, then the **high** parts with `adc`, which also adds the carry produced by the low parts. For subtraction use `sub`, then `sbb`.

```
segment data use32 class=data
    x dq 00000001FFFFFFFFh
segment code use32 class=code
start:
    mov eax, 1
    mov edx, 0              ; EDX:EAX = 1
    add eax, dword [x]      ; low dwords:  1 + FFFFFFFFh = 1 00000000h -> EAX = 0, CF = 1
    adc edx, dword [x+4]    ; high dwords: 0 + 1 + CF    -> EDX = 2
                            ; EDX:EAX = 00000002 00000000h
```

`dword [x]` is the low dword of `x` and `dword [x+4]` the high dword (little endian). The same idea works for a dword kept in `DX:AX` (16-bit halves): `add ax, …` then `adc dx, …`.

> **USING ADD FOR THE HIGH PART**
>
> `add edx, [x+4]` would forget the carry from the low part; the result would be wrong whenever the low addition overflows. The high part always needs `adc` (or `sbb`).

5.4 Multiplication: MUL and IMUL

`mul` (unsigned) and `imul` (signed) have **one explicit operand** — a register or a memory location, never a constant. The other factor and the destination are **implicit** and depend on the size of the operand:

Operand size	Computes	Result in
byte	`AL · op`	`AX`
word	`AX · op`	`DX:AX` (DX = high word)
dword	`EAX · op`	`EDX:EAX`

The result is always **twice** the size of the factors, so a multiplication can never lose bits.

```
segment data use32 class=data
    c db 3
segment code use32 class=code
start:
    mov al, 7
    mov bl, 2
    mul bl                  ; AX = 14
    mul byte [c]            ; AX = AL*3 = 42  (size needed for a memory operand!)
    mov ax, 1000
    mov bx, 70
    mul bx                  ; DX:AX = 70000 = 0001 1170h -> DX = 0001h, AX = 1170h
```

```
mul 5                   ; ERROR - no immediate operand for mul
mul [c]                 ; ERROR - size not specified
```

Why DX:AX and not EAX?

For compatibility with the 16-bit 8086, which had no EAX. When you need the word-multiplication result in one register, combine the halves:

```
segment code use32 class=code
start:
    mov ax, 1000
    mov bx, 70
    mul bx                  ; DX:AX = 70000
    push dx                 ; method 1: through the stack (Chapter 11)
    push ax
    pop ecx                 ; ECX = 70000
    movzx eax, ax           ; method 2: EAX = low word, high word cleared
    shl edx, 16             ; the old DX moves to bits 16-31 of EDX
    or eax, edx             ; EAX = 70000
```

Two- and three-operand IMUL

`imul` also has the forms `imul reg, src` (reg = reg · src) and `imul reg, src, constant`. Their result is **truncated** to the size of the destination, so it can lose bits. Use them only when you know the product fits; otherwise use the one-operand form.

```
segment code use32 class=code
start:
    mov ebx, -6
    mov eax, 7
    imul eax, ebx           ; EAX = -42
    imul ecx, ebx, 10       ; ECX = -60
```

5.5 Division: DIV and IDIV

`div` (unsigned) and `idiv` (signed) have one explicit operand: the **divisor**.

Divisor size	Dividend	Quotient	Remainder
byte	`AX`	`AL`	`AH`
word	`DX:AX`	`AX`	`DX`
dword	`EDX:EAX`	`EAX`	`EDX`

> **THE DIVIDEND IS TWICE THE SIZE OF THE DIVISOR**
>
> Before `div bx` the whole dividend must be in `DX:AX` — so `DX` must be prepared even if the number "fits in AX": `mov dx, 0` for unsigned values, `cwd` for signed ones.

If the quotient does not fit in `AL` / `AX` / `EAX`, or the divisor is 0, the processor raises an exception and the program is stopped ("Integer overflow" / "Integer division by zero").

```
mov ax, 1000
mov bl, 2
div bl                  ; 1000 / 2 = 500 does not fit in AL -> the program crashes
```

```
segment code use32 class=code
start:
    mov ax, 1000
    mov dx, 0               ; DX:AX = 1000 (unsigned extension)
    mov bx, 2
    div bx                  ; AX = 500, DX = 0
    mov ax, -7
    cwd                     ; DX:AX = -7
    mov cx, 2
    idiv cx                 ; AX = -3 (FFFDh), DX = -1 (FFFFh)
```

`idiv` truncates the quotient **toward zero**: −7 / 2 = −3, and the remainder has the sign of the **dividend** (−7 = 2 · (−3) + (−1)).

GARBAGE IN THE HIGH PART

After an earlier `mul`, `DX` / `EDX` usually hold a non-zero value. Dividing without resetting them makes the dividend enormous and the quotient does not fit → crash. Always set `AH` / `DX` / `EDX` right before the division.

5.6 Conversions: making a value wider

Before combining operands of different sizes, the smaller one must be **widened** without changing its value: zero extension for unsigned values, sign extension for signed ones (Chapter 2).

From → to	Unsigned (zero extension)	Signed (sign extension)
`AL` → `AX`	`mov ah, 0`	`cbw`
`AX` → `DX:AX`	`mov dx, 0`	`cwd`
`AX` → `EAX`	`movzx eax, ax`	`cwde` (or `movsx eax, ax`)
`EAX` → `EDX:EAX`	`mov edx, 0`	`cdq`
any reg/mem → larger reg	`movzx r, src`	`movsx r, src`

`cbw`, `cwd`, `cwde`, `cdq` work only on AL/AX/EAX and have no operands. `movzx` and `movsx` accept any register as destination and a register or memory source (with an explicit size):

```
segment data use32 class=data
    a db -3
    b dw 0FFFEh
segment code use32 class=code
start:
    movsx ebx, byte [a]     ; EBX = FFFFFFFDh = -3
    movzx ecx, word [b]     ; ECX = 0000FFFEh = 65534
    mov al, [a]
    cbw                     ; AX  = FFFDh
    cwde                    ; EAX = FFFFFFFDh
    cdq                     ; EDX = FFFFFFFFh (EDX:EAX = -3 on 64 bits)
```

```
movzx eax, [v]          ; ERROR - the size of [v] is not specified
```

CWDE VERSUS CWD

Both turn a word into a dword. **cwde** keeps the result in one register (E**AX**); **cwd** splits it in two (**DX**:AX), which is what a word division needs.

5.7 Classification of conversions

The course classifies the ways of changing the size of a value:

- **Destructive** conversions change the register: `cbw`, `cwd`, `cwde`, `cdq`, `movzx`, `movsx`, `mov ah, 0`, `mov dx, 0`, `mov edx, 0`.
- **Non-destructive**: the type operators `byte` / `word` / `dword` / `qword [x]` — memory is only *looked at* with another size.
- **Signed**: `cbw`, `cwd`, `cwde`, `cdq`, `movsx`. **Unsigned**: `movzx`, `mov ah, 0`, ...
- **By enlargement**: the destructive ones above. **By narrowing**: taking a smaller part (`byte [x]` of a word, `AL` of `EAX`) — this can **lose information**.

5.8 The method for any expression

1. Write down the size of every variable. Decide: signed (→ `imul`, `idiv`, `cbw`...) or unsigned (→ `mul`, `div`, `mov ah, 0`...).
2. Go from the innermost parentheses outwards, following operator precedence.
3. Before every operation, bring both operands to the **same size** (widen the smaller one).
4. Before `mul`: one factor must be in `AL` / `AX` / `EAX`. Before `div`: the dividend must be in `AX` / `DX:AX` / `EDX:EAX`.
5. Keep intermediate results in free registers (`BX`, `CX`, `ESI`, `EDI`...) or on the stack — `mul` / `div` destroy `EAX` and `EDX`.
6. For qwords use pairs `EDX:EAX` / `ECX:EBX` and `add` / `adc`, `sub` / `sbb`.

Worked case 1 (unsigned, bytes): x = (a + b) · c / d

```
segment data use32 class=data
    a db 3
    b db 4
    c db 2
    d db 3
    x db 0
segment code use32 class=code
start:
    mov al, [a]             ; AL = 3
    add al, [b]             ; AL = 7
    mul byte [c]            ; AX = 14
    div byte [d]            ; AL = 4 (quotient), AH = 2 (remainder)
    mov [x], al             ; x = 4
```

Worked case 2 (signed, mixed sizes): x = (a · b) / d − c, a, c, d bytes, b word

```
segment data use32 class=data
    a db -3
    b dw 4
    c db 2
    d db 3
    x dw 0
segment code use32 class=code
start:
    mov al, [a]
    cbw                     ; AX = -3 (byte -> word, b is a word)
    imul word [b]           ; DX:AX = -12
    mov bx, ax              ; save the low part (DX is kept)
    mov al, [d]
    cbw
    mov cx, ax              ; CX = 3 (the divisor must be a word: dividend is DX:AX)
    mov ax, bx              ; DX:AX = -12 again
    idiv cx                 ; AX = -4
    mov bx, ax
    mov al, [c]
    cbw                     ; AX = 2
    sub bx, ax              ; BX = -6
    mov [x], bx             ; x = -6
```

Notice how every value was brought to a **word** before being combined with a word, and how `BX` / `CX` kept the intermediate results away from the registers that `cbw` and `idiv` overwrite.

5.9 Chapter summary

IN SHORT

- Same sizes, at most one memory operand, destination not a constant.
- Wide numbers: `add` + `adc`, `sub` + `sbb`.
- `mul` / `imul` op: byte → AX, word → DX:AX, dword → EDX:EAX; never a constant operand.
- `div` / `idiv` op: dividend AX / DX:AX / EDX:EAX; quotient AL/AX/EAX, remainder AH/DX/EDX; prepare the high part; quotient must fit.
- `idiv` truncates toward zero; remainder has the sign of the dividend.
- Widen: unsigned `mov ah/dx/edx, 0`, `movzx`; signed `cbw`, `cwd`, `cwde`, `cdq`, `movsx`.
- Expressions: sizes → signed/unsigned → inner parentheses first → equalise sizes → watch EAX/EDX.

Solutions — Chapter 5

Exercise 5.1 — (a − b) + (c − d), words, unsigned BASIC

STATEMENT

Compute `(a − b) + (c − d)` for the unsigned words `a = 50`, `b = 20`, `c = 7`, `d = 2`. Write the program and give the result register.

THEORY USED

`sub`, `add` with operands of the same size (5.1, 5.2).

STEP BY STEP

All operands are words, so no conversion is needed.

```
segment data use32 class=data
    a dw 50
    b dw 20
    c dw 7
    d dw 2
segment code use32 class=code
start:
    mov ax, [a]
    sub ax, [b]             ; AX = 30
    mov bx, [c]
    sub bx, [d]             ; BX = 5
    add ax, bx              ; AX = 35
```

FINAL RESULT

AX = 35 (0023h).

Exercise 5.2 — a · b − c, mixed sizes, unsigned BASIC

STATEMENT

Compute `a · b − c` with `a`, `b` unsigned bytes and `c` an unsigned word (`a = 20`, `b = 15`, `c = 100`).

THEORY USED

`mul` byte → AX (5.4); the product is already a word, the size of `c`.

STEP BY STEP

```
segment data use32 class=data
    a db 20
    b db 15
    c dw 100
segment code use32 class=code
start:
    mov al, [a]
    mul byte [b]            ; AX = 300
    sub ax, [c]             ; AX = 200
```

FINAL RESULT

AX = 200 (00C8h).

Exercise 5.3 — Where does the product go? BASIC

STATEMENT

Give the registers that hold the result, and their values in hex, after each sequence: (a) `mov al, 7` / `mov bl, 2` / `mul bl` (b) `mov ax, 1000` / `mov bx, 70` / `mul bx` (c) `mov eax, 100000` / `mov ebx, 100000` / `mul ebx`

THEORY USED

Implicit operands of `mul` (5.4).

STEP BY STEP

(a) byte operand → AX = 7 · 2 = 14 = **000Eh**.

(b) word operand → DX:AX = 70 000 = 0001 1170h → **DX = 0001h, AX = 1170h**.

(c) dword operand → EDX:EAX = 10^{10} = 2 540B E400h → **EDX = 00000002h, EAX = 540BE400h**.

The product never loses bits: it is placed in a destination twice as large.

FINAL RESULT

(a) AX = 000Eh; (b) DX = 0001h, AX = 1170h; (c) EDX = 2, EAX = 540BE400h.

Exercise 5.4 — A division that crashes BASIC

STATEMENT

What happens at `mov ax, 300` / `mov bl, 1` / `div bl`? Fix it.

THEORY USED

The quotient of a byte division must fit in AL (5.5).

STEP BY STEP

The dividend is AX = 300, the divisor BL = 1, the quotient 300 must go into **AL**, which holds at most 255 → division overflow → the program is stopped with an exception.

Fix: divide on words, so the quotient goes into AX:

```
segment code use32 class=code
start:
    mov ax, 300
    mov dx, 0               ; DX:AX = 300
    mov bx, 1
    div bx                  ; AX = 300, DX = 0
```

FINAL RESULT

Exception (quotient 300 does not fit in AL); with a word divisor AX = 300, DX = 0.

Exercise 5.5 — (a + b) / c, careful with the division INTERMEDIATE

STATEMENT

Compute `(a + b) / c` with `a`, `b` unsigned words and `c` an unsigned byte (`a = 1000`, `b = 200`, `c = 7`). Give quotient and remainder.

THEORY USED

Preparing the dividend and choosing the divisor size (5.5, 5.6).

STEP BY STEP

The sum is a word, so the division should be **word / word** with the dividend in DX:AX. Dividing by the byte would put the quotient in AL — 1200 / 7 = 171 happens to fit, but 1200 / 2 = 600 would crash. Widening the divisor to a word is the safe way.

```
segment data use32 class=data
    a dw 1000
    b dw 200
    c db 7
segment code use32 class=code
start:
    mov ax, [a]
    add ax, [b]             ; AX = 1200
    mov dx, 0               ; DX:AX = 1200 (unsigned extension)
    mov bl, [c]
    mov bh, 0               ; BX = 7
    div bx                  ; AX = 171, DX = 3
```

Check: 7 · 171 + 3 = 1200 ✓.

FINAL RESULT

Quotient AX = 171, remainder DX = 3.

Exercise 5.6 — a / b + c · d, signed INTERMEDIATE

STATEMENT

Compute `a / b + c · d` (signed) for `a = −100000` (dword), `b = 300` (word), `c = −4`, `d = 25` (bytes).

THEORY USED

`idiv` word: dividend DX:AX (5.5); `imul` byte → AX (5.4); little endian halves of a dword.

STEP BY STEP

A dword divided by a word: put `a` in DX:AX — its low word in AX, its high word in DX. The quotient is a word, the product c·d is a word, so the addition is on words.

```
segment data use32 class=data
    a dd -100000
    b dw 300
    c db -4
    d db 25
segment code use32 class=code
start:
    mov ax, word [a]        ; low word of a
    mov dx, word [a+2]      ; high word of a  -> DX:AX = a
    idiv word [b]           ; AX = a/b = -333  (remainder DX = -100)
    mov bx, ax              ; BX = -333
    mov al, [c]
    imul byte [d]           ; AX = c*d = -100
    add ax, bx              ; AX = -433 (FE4Fh)
```

Check: −100 000 / 300 = −333.3… → −333 (toward zero); −333 + (−100) = −433.

FINAL RESULT

AX = −433 = FE4Fh.

Exercise 5.7 — Signs of quotient and remainder INTERMEDIATE

STATEMENT

Give the quotient and the remainder of `idiv` for −7 / 2, 7 / −2 and −7 / −2 (bytes, dividend in AX).

THEORY USED

`idiv` truncates toward zero; the remainder has the sign of the dividend (5.5).

STEP BY STEP

Use dividend = divisor · quotient + remainder with the quotient truncated toward zero:

division	quotient (AL)	remainder (AH)	check
−7 / 2	−3 (FDh)	−1 (FFh)	2 · (−3) − 1 = −7
7 / −2	−3 (FDh)	1 (01h)	(−2) · (−3) + 1 = 7
−7 / −2	3 (03h)	−1 (FFh)	(−2) · 3 − 1 = −7

```
segment code use32 class=code
start:
    mov ax, -7
    mov bl, 2
    idiv bl                 ; AL = FDh (-3), AH = FFh (-1)
    mov cx, ax              ; keep it
    mov ax, 7
    mov bl, -2
    idiv bl                 ; AL = FDh (-3), AH = 01h
```

FINAL RESULT

(−3, −1), (−3, 1), (3, −1).

Exercise 5.8 — A chain of conversions INTERMEDIATE

STATEMENT

`AL = 0F4h`. Give AX after `cbw`, then EAX after `cwde`, then EDX after `cdq`. What would `movzx ebx, al` give instead?

THEORY USED

Sign and zero extension instructions (5.6).

STEP BY STEP

F4h = −12 signed, 244 unsigned.

```
segment code use32 class=code
start:
    mov al, 0F4h
    movzx ebx, al           ; EBX = 000000F4h = 244
    cbw                     ; AX  = FFF4h       = -12
    cwde                    ; EAX = FFFFFFF4h   = -12
    cdq                     ; EDX = FFFFFFFFh   (EDX:EAX = -12)
```

FINAL RESULT

AX = FFF4h, EAX = FFFFFFF4h, EDX = FFFFFFFFh; `movzx` gives EBX = 000000F4h.

Exercise 5.9 — Adding and subtracting qwords INTERMEDIATE

STATEMENT

For the qwords `x = 1FFFFFFFFh` and `y = 0FFFFFFFFh` compute `s = x + y` and `t = x − y` with 32-bit registers.

THEORY USED

`add` / `adc` and `sub` / `sbb` on register pairs (5.3); little endian halves.

STEP BY STEP

```
segment data use32 class=data
    x dq 1FFFFFFFFh
    y dq 0FFFFFFFFh
    s dq 0
    t dq 0
segment code use32 class=code
start:
    mov eax, [x]
    mov edx, [x+4]          ; EDX:EAX = x
    add eax, [y]            ; FFFFFFFFh + FFFFFFFFh: EAX = FFFFFFFEh, CF = 1
    adc edx, [y+4]          ; 1 + 0 + 1:             EDX = 2
    mov [s], eax
    mov [s+4], edx          ; s = 2FFFFFFFEh
    mov eax, [x]
    mov edx, [x+4]
    sub eax, [y]            ; EAX = 0, CF = 0 (no borrow)
    sbb edx, [y+4]          ; EDX = 1
    mov [t], eax
    mov [t+4], edx          ; t = 100000000h
```

FINAL RESULT

s = 2 FFFFFFFEh (12 884 901 886), t = 1 00000000h (4 294 967 296).

Exercise 5.10 — A long signed expression CHALLENGE

STATEMENT

Compute (signed) `x + (2 − a·b) / (a·3) − a + c` with `a` byte, `b` word, `c` dword, `x` qword. Use `a = −5`, `b = 300`, `c = −100000`, `x = −7000000000` and store the result in the qword `r`.

THEORY USED

The method (5.8): conversions (5.6), `imul` / `idiv` implicit operands (5.4, 5.5), qword arithmetic (5.3).

STEP BY STEP

Plan: a·b is a dword (DX:AX); 2 − a·b on DX:AX; a·3 is a word; the word division gives a word quotient; then everything is widened to a qword for the additions with `x` and `c`.

```
segment data use32 class=data
    a db -5
    b dw 300
    c dd -100000
    x dq -7000000000
```

```
    r dq 0
segment code use32 class=code
start:
    mov al, [a]
    cbw                     ; AX = a
    imul word [b]           ; DX:AX = a*b = -1500
    mov bx, ax
    mov cx, dx              ; CX:BX = a*b
    mov ax, 2
    cwd                     ; DX:AX = 2
    sub ax, bx
    sbb dx, cx              ; DX:AX = 2 - a*b = 1502
    push dx
    push ax                 ; keep it safe
    mov al, [a]
    mov bl, 3
    imul bl                 ; AX = a*3 = -15
    mov bx, ax
    pop ax
    pop dx                  ; DX:AX = 1502
    idiv bx                 ; AX = 1502 / -15 = -100
    cwde
    cdq                     ; EDX:EAX = -100 on a qword
    add eax, dword [x]
    adc edx, dword [x+4]    ; + x
    mov ebx, eax
    mov ecx, edx            ; ECX:EBX = x - 100
    mov al, [a]
    cbw
    cwde
    cdq                     ; EDX:EAX = a
    sub ebx, eax
    sbb ecx, edx            ; - a
    mov eax, [c]
    cdq                     ; EDX:EAX = c
    add ebx, eax
    adc ecx, edx            ; + c
    mov [r], ebx
    mov [r+4], ecx          ; r = -7000100095
```

Check: 2 − (−1500) = 1502; 1502 / (−15) = −100 (toward zero); −7 000 000 000 − 100 + 5 − 100 000 = −7 000 100 095.

FINAL RESULT

r = −7 000 100 095 (FFFFFFFE5EC2F301h).

Exercise 5.11 — A long unsigned expression CHALLENGE

STATEMENT

Compute (unsigned) `x − (a·b + c) / (a + 1)` with `a` byte, `b` word, `c` dword, `x` qword, for `a = 200`, `b = 1000`, `c = 70000`, `x = 5000000000`. Store the result in the qword `r`.

THEORY USED

Unsigned versions of the conversions (`mov ah, 0`, `movzx`, `mov edx, 0`) (5.6); `div` dword (5.5); `sub`/`sbb` (5.3).

STEP BY STEP

- a·b: widen a to a word (`mov ah, 0`), `mul word [b]` → DX:AX; move it into EAX.
- c: the sum may exceed 32 bits in general, so keep the carry in EDX: EDX:EAX = a·b + c.

- / (a + 1): divisor as a dword in ECX; `div ecx` divides EDX:EAX.
- x – quotient: qword subtraction with `sub` / `sbb ecx, 0`.

```
segment data use32 class=data
    a db 200
    b dw 1000
    c dd 70000
    x dq 5000000000
    r dq 0
segment code use32 class=code
start:
    mov al, [a]
    mov ah, 0               ; AX = a (unsigned)
    mul word [b]            ; DX:AX = 200000
    push dx
    push ax
    pop eax                 ; EAX = a*b = 200000
    mov edx, 0
    add eax, [c]
    adc edx, 0              ; EDX:EAX = a*b + c = 270000
    movzx ecx, byte [a]
    inc ecx                 ; ECX = a + 1 = 201
    div ecx                 ; EAX = 1343, EDX = 57
    mov ebx, [x]
    mov ecx, [x+4]          ; ECX:EBX = x
    sub ebx, eax
    sbb ecx, 0              ; ECX:EBX = x - 1343
    mov [r], ebx
    mov [r+4], ecx          ; r = 4999998657
```

Check: 270 000 / 201 = 1343 remainder 57; 5 000 000 000 − 1343 = 4 999 998 657.

FINAL RESULT

r = 4 999 998 657.

Exercise 5.12 — Bug hunt: (b·c – a) / (a + 4) + d CHALLENGE

STATEMENT

The program below should compute, **signed**, `(b·c – a) / (a + 4) + d` in `EAX`, with `a` byte, `b` word, `c` byte, `d` dword (`a = –7`, `b = 300`, `c = –5`, `d = 1000`). Find every bug, explain it, and give the corrected program and the final value of `EAX`.

```
mov al, [c]
imul word [b]
push dx
push ax
pop ebx
mov al, a
cbw
cwde
sub ebx, eax
mov cl, [a]
add cl, 4
movzx ecx, cl
mov eax, ebx
cdq
div ecx
add eax, d
```

THEORY USED

Conversions (5.6), `imul` / `idiv` (5.4, 5.5), name versus [name] (Chapter 4).

STEP BY STEP

line	bug	fix
`imul word [b]`	the word multiplication uses **AX**, but only AL was loaded (AH is garbage)	`cbw` before `imul`
`mov al, a`	`a` is the **address**, not the value	`mov al, [a]`
`movzx ecx, cl`	a + 4 = −3 is **signed**; zero extension makes it 253	`movsx ecx, cl`
`div ecx`	unsigned division of a signed value	`idiv ecx`
`add eax, d`	adds the **address** of `d`	`add eax, [d]`

```
segment data use32 class=data
    a db -7
    b dw 300
    c db -5
    d dd 1000
segment code use32 class=code
start:
    mov al, [c]
    cbw                     ; AX = c = -5
    imul word [b]           ; DX:AX = b*c = -1500
    push dx
    push ax
    pop ebx                 ; EBX = -1500
    mov al, [a]
    cbw
    cwde                    ; EAX = a = -7
    sub ebx, eax            ; EBX = -1500 + 7 = -1493
    mov cl, [a]
    add cl, 4               ; CL = -3
    movsx ecx, cl           ; ECX = -3
    mov eax, ebx
    cdq                     ; EDX:EAX = -1493
    idiv ecx                ; EAX = 497, EDX = -2
    add eax, [d]            ; EAX = 1497
```

Check: −1493 / −3 = 497.67 → 497 (toward zero), remainder −1493 − (−3)·497 = −2; 497 + 1000 = 1497.

FINAL RESULT

Five bugs (missing `cbw`, `a` instead of `[a]`, `movzx` instead of `movsx`, `div` instead of `idiv`, `d` instead of `[d]`); EAX = 1497.

CHAPTER 6

Overflow: CF versus OF

What overflow means, why the processor needs two flags for it, quick rules to predict CF and OF for additions and subtractions, how multiplication and division behave, and a complete method for analysing a sequence of instructions.

6.1 The two flags

Overflow means that the result of an operation does **not fit** in the space reserved for it. Because the same bits can be read as unsigned or as signed (Chapter 2), "does not fit" has two meanings, and the processor reports both:

	CF — Carry Flag	OF — Overflow Flag
Interpretation	**unsigned**	**signed**
Set when	the true result does not fit as an unsigned number (carry out of the MSB on add, borrow on sub)	the true result does not fit in the signed range (the sign of the result is "wrong")
Byte range	0 … 255	−128 … 127

CF and OF are **independent**: all four combinations are possible. The processor always sets both; it is the programmer who knows which interpretation matters and therefore which flag to look at (`jc` for unsigned, `jo` for signed).

> **THINK IN TWO COLUMNS**
>
> For every operation write two columns — unsigned and signed — compute the true result in each, and check whether it fits the range of the destination. CF answers the unsigned column, OF the signed column.

6.2 Quick rules for OF and CF

Addition

OF = 1 only when you add two numbers with the **same sign** and obtain a result with the **other sign**:

- positive + positive = negative → OF = 1
- negative + negative = positive → OF = 1
- positive + negative → **never** overflows

CF = 1 when the unsigned sum is $\geq 2^n$ (there is a carry out of the MSB).

Subtraction a − b

OF = 1 when a and b have **different signs** and the result has the sign of **b**:

- positive − negative = negative → OF = 1
- negative − positive = positive → OF = 1

CF = 1 when, unsigned, **a < b** (a borrow was needed).

WHY THE SIGN RULES WORK

Adding a positive and a negative number gives a value between the two operands, so it always fits. Only two operands that pull in the same direction can leave the range — and when they do, the result wraps around to the opposite sign.

"THE RESULT IS NEGATIVE, SO OF = 1"

A negative result is not an overflow by itself: 10h − 20h = F0h = −16 is perfectly correct signed (OF = 0). OF asks whether the result is **wrong**, not whether it is negative.

6.3 The four cases

Instruction	Bits	Unsigned	Signed	CF	OF
F0h + 20h	10h	240 + 32 = 272 ×	−16 + 32 = 16 ✓	1	0
7Fh + 01h	80h	127 + 1 = 128 ✓	127 + 1 = 128 × (reads −128)	0	1
80h + 80h	00h	128 + 128 = 256 ×	−128 − 128 = −256 ×	1	1
10h + 20h	30h	16 + 32 = 48 ✓	16 + 32 = 48 ✓	0	0

```
segment code use32 class=code
start:
    mov al, 0F0h
    add al, 20h             ; AL = 10h  CF=1 OF=0
    mov bl, 7Fh
    add bl, 1               ; BL = 80h  CF=0 OF=1
    mov cl, 80h
    add cl, 80h             ; CL = 00h  CF=1 OF=1
    mov dl, 10h
    add dl, 20h             ; DL = 30h  CF=0 OF=0
```

6.4 Multiplication and division

mul / imul

A multiplication **cannot overflow**: its result has double size (Chapter 5). Instead, `mul` and `imul` set **CF = OF = 1** when the **high half** of the result is significant — i.e. the result does not fit in the low half alone — and CF = OF = 0 otherwise.

- `mov al, 100` / `mov bl, 3` / `mul bl` → AX = 012Ch (300); AH ≠ 0 → **CF = OF = 1**.
- `mov al, 10` / `mov bl, -3` / `imul bl` → AX = FFE2h (−30); −30 fits in AL as a signed byte (AH is only the sign extension) → **CF = OF = 0**.

```
segment code use32 class=code
start:
    mov al, 100
    mov bl, 3
    mul bl                  ; AX = 012Ch  CF=1 OF=1
    mov al, 10
    mov bl, -3
    imul bl                 ; AX = FFE2h  CF=0 OF=0
```

div / idiv

If the quotient does not fit in AL/AX/EAX, the processor raises a **run-time exception** ("division overflow", "integer overflow") and the program is terminated. Flags are not the mechanism here — after a division they

are undefined.

6.5 Analysing a sequence of instructions

A typical task: *"What is the result? Detail the effect of each line, give the signed and unsigned interpretation in base 10 and 16 of each value, and say whether there is an overflow, explaining its cause and effect."*

Template for every line: value in hex → unsigned decimal → signed decimal → CF/OF with the reason.

Worked case A

```
segment code use32 class=code
start:
    mov ah, 128 | 2          ; 1000 0000b | 0000 0010b = 82h
    mov bh, 90h >> 3         ; 1001 0000b >> 3 = 0001 0010b = 12h
    sub ah, bh               ; AH = 70h  CF=0 OF=1
```

value	hex	unsigned	signed
AH before	82h	130	−126
BH	12h	18	18
AH after	70h	112	112

- Unsigned: 130 − 18 = 112, correct, no borrow → **CF = 0**.
- Signed: −126 − 18 = −144 < −128 does not fit; we got +112 (negative − positive = positive) → **OF = 1**, signed overflow.
- `|` and `>>` are **operators**, computed by NASM at assembly time; only `sub` runs on the CPU.

Worked case B

```
mov ax, -100             ; AX = FF9Ch
mov bl, 2
div bl                   ; AL = AX / BL ?
```

`div` is unsigned: AX = FF9Ch = 65 436. 65 436 / 2 = 32 718 does not fit in AL (max 255) → **division overflow** → the program is stopped with a run-time exception. With `idiv bl` it would be fine: −100 / 2 = −50 fits in AL. The bug is using the unsigned instruction on a negative value.

Worked case C

```
segment code use32 class=code
start:
    mov dh, 62h              ; 98
    mov ch, 200              ; C8h = 200 unsigned / -56 signed
    sub dh, ch               ; DH = 9Ah  CF=1 OF=1 SF=1 ZF=0 PF=1
```

- Unsigned: 98 − 200 < 0 → borrow → **CF = 1**; 9Ah reads as 154.
- Signed: 98 − (−56) = 154 > 127 → **OF = 1**; 9Ah reads as −102.
- SF = 1, ZF = 0, PF = 1 (9Ah = `1001 1010` has four 1 bits).

6.6 Chapter summary

IN SHORT

- Overflow = the true result does not fit. CF: unsigned overflow; OF: signed overflow; independent.
- Add: OF only for same-sign operands giving the other sign; CF when the unsigned sum ≥ 2^n.
- Sub a − b: OF when signs differ and the result has the sign of b; CF when a < b unsigned.
- `mul` / `imul` : CF = OF = 1 if the high half is significant; never a real overflow.
- `div` / `idiv` : a quotient that does not fit → run-time exception.
- Analyse each line: hex → unsigned → signed → CF, OF with the reason; operators are computed by NASM.

Solutions — Chapter 6

Exercise 6.1 — 0BCh + 0DEh BASIC

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 0BCh
add al, 0DEh
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

BCh + DEh = 19Ah → AL = 9Ah.

- Unsigned: 188 + 222 = 410 > 255 → **CF = 1**.
- Signed: −68 + (−34) = −102, fits; 9Ah = −102 ✓ → **OF = 0**.

```
segment code use32 class=code
start:
    mov al, 0BCh
    add al, 0DEh
```

FINAL RESULT

AL = 9Ah = 154 unsigned = −102 signed; CF = 1, OF = 0 → unsigned overflow only.

Exercise 6.2 — 50 + 100 BASIC

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 50
add al, 100
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

50 + 100 = 150 = 96h.

- Unsigned: 150 ≤ 255 fits → **CF = 0**.
- Signed: positive + positive gave 96h = −106 (negative) → **OF = 1**.

```
segment code use32 class=code
start:
    mov al, 50
    add al, 100
```

FINAL RESULT

AL = 96h = 150 unsigned = −106 signed; CF = 0, OF = 1 → signed overflow only.

Exercise 6.3 — −50 − 100 BASIC

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, -50
sub al, 100
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

−50 = CEh, 100 = 64h; CEh − 64h = 6Ah.

- Unsigned: 206 − 100 = 106, no borrow → **CF = 0**.
- Signed: −50 − 100 = −150 < −128; we got +106 (negative − positive = positive) → **OF = 1**.

```
segment code use32 class=code
start:
    mov al, -50
    sub al, 100
```

FINAL RESULT

AL = 6Ah = 106 unsigned = 106 signed; CF = 0, OF = 1 → signed overflow.

Exercise 6.4 — 8000h − 1 on a word BASIC

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov ax, 8000h
sub ax, 1
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

8000h − 1 = 7FFFh.

- Unsigned: 32 768 − 1 = 32 767 → **CF = 0**.
- Signed: −32 768 − 1 = −32 769 does not fit in a word → **OF = 1**.

```
segment code use32 class=code
start:
    mov ax, 8000h
    sub ax, 1
```

FINAL RESULT

AX = 7FFFh = 32 767 unsigned = 32 767 signed; CF = 0, OF = 1 → signed overflow.

Exercise 6.5 — 0FFh · 2 with mul INTERMEDIATE

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, 0FFh
mov bl, 2
mul bl
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

255 · 2 = 510 = 01FEh in AX. AX holds it, so nothing is lost; but the high byte AH = 01h ≠ 0, so `mul` sets **CF = OF = 1** ("the result does not fit in AL alone").

```
segment code use32 class=code
start:
    mov al, 0FFh
    mov bl, 2
    mul bl
```

FINAL RESULT

AX = 01FEh = 510 unsigned = 510 signed; CF = OF = 1 (the result needs AH); no real overflow.

Exercise 6.6 — −100 · 3 with imul INTERMEDIATE

STATEMENT

For the sequence below give the final value (hex, unsigned, signed), CF, OF, and say whether there is an overflow.

```
mov al, -100
mov bl, 3
imul bl
```

THEORY USED

Quick rules for CF and OF (6.2); two-column method (6.1).

STEP BY STEP

−100 · 3 = −300 = FED4h in AX. −300 does not fit in AL as a signed byte (AH is not just the sign extension of AL) → **CF = OF = 1**. The word result in AX is correct.

```
segment code use32 class=code
start:
    mov al, -100
    mov bl, 3
    imul bl
```

FINAL RESULT

AX = FED4h = 65 236 unsigned = −300 signed; CF = OF = 1; the value in AX is correct.

Exercise 6.7 — 1000 / 3 with div INTERMEDIATE

STATEMENT

What happens at `mov ax, 1000` / `mov bl, 3` / `div bl`?

THEORY USED

Division overflow (6.4).

STEP BY STEP

1000 / 3 = 333 > 255 → the quotient does not fit in AL → **division overflow**, the program is stopped by a run-time exception. Flags play no role. Fix: divide on words (`mov dx, 0` / `mov bx, 3` / `div bx` → AX = 333, DX = 1).

```
segment code use32 class=code
start:
    mov ax, 1000
    mov dx, 0
    mov bx, 3
    div bx                  ; AX = 333, DX = 1
```

FINAL RESULT

Run-time exception (quotient 333 does not fit in AL); with a word divisor AX = 333, DX = 1.

Exercise 6.8 — Operators, then a subtraction INTERMEDIATE

STATEMENT

Detail every line (hex, unsigned, signed) and give CF and OF after the last one:

```
mov ch, 0C0h | 0Ah
mov dl, 0A0h >> 2
sub dl, ch
```

THEORY USED

Operators are evaluated by NASM (Chapter 4); rules for subtraction (6.2).

STEP BY STEP

- 0C0h | 0Ah = **1100 0000** | 0000 **1010** = **1100 1010** = **CAh** (202 / −54).
- 0A0h >> 2 = **1010 0000** >> 2 = 0010 **1000** = **28h** (40 / 40).
- 28h − CAh = **5Eh** (94 / 94).

value	hex	unsigned	signed
CH	CAh	202	−54
DL before	28h	40	40
DL after	5Eh	94	94

- Unsigned: 40 − 202 < 0 → borrow → **CF = 1** (the true result −162 is not representable; 5Eh = 94 is wrong).

- Signed: 40 − (−54) = 94, fits → **OF = 0** (5Eh = 94 is correct).

```
segment code use32 class=code
start:
    mov ch, 0C0h | 0Ah      ; CH = CAh
    mov dl, 0A0h >> 2       ; DL = 28h
    sub dl, ch              ; DL = 5Eh  CF=1 OF=0
```

FINAL RESULT

DL = 5Eh = 94 (both readings); CF = 1 (unsigned borrow), OF = 0 (signed result correct).

Exercise 6.9 — Overflow on words INTERMEDIATE

STATEMENT

Give the result and CF, OF for (a) `mov ax, 7FFFh` / `add ax, 7FFFh`; (b) `mov bx, 0` / `sub bx, 1`.

THEORY USED

Rules of 6.2 with n = 16 bits (ranges 0...65 535 and −32 768...32 767).

STEP BY STEP

(a) 7FFFh + 7FFFh = **FFFEh**. Unsigned 32 767 + 32 767 = 65 534 fits → CF = 0. Signed positive + positive gave FFFEh = −2 → **OF = 1**.

(b) 0 − 1 = **FFFFh**. Unsigned 0 < 1 → borrow → **CF = 1**. Signed 0 − 1 = −1, correct → OF = 0.

```
segment code use32 class=code
start:
    mov ax, 7FFFh
    add ax, 7FFFh           ; AX = FFFEh  CF=0 OF=1
    mov bx, 0
    sub bx, 1               ; BX = FFFFh  CF=1 OF=0
```

FINAL RESULT

(a) AX = FFFEh, CF = 0, OF = 1; (b) BX = FFFFh, CF = 1, OF = 0.

Exercise 6.10 — All four combinations, for add and for sub CHALLENGE

STATEMENT

Find byte operands that produce each of the four combinations (CF, OF) = (0,0), (0,1), (1,0), (1,1) — first with `add`, then with `sub`. Justify each.

THEORY USED

Rules of 6.2 and the table of 6.3.

STEP BY STEP

add (the table of 6.3):

operands	result	CF	OF	why
10h + 20h	30h	0	0	48 fits both ways
7Fh + 01h	80h	0	1	128 fits unsigned; positive + positive = negative
F0h + 20h	10h	1	0	272 > 255; −16 + 32 = 16 correct
80h + 80h	00h	1	1	256 > 255; negative + negative = 0

sub:

operands	result	CF	OF	why
30h − 10h	20h	0	0	48 − 16 = 32 both ways
80h − 01h	7Fh	0	1	128 ≥ 1; −128 − 1 = −129 does not fit
10h − 20h	F0h	1	0	16 < 32 borrow; 16 − 32 = −16 correct
7Fh − FFh	80h	1	1	127 < 255 borrow; 127 − (−1) = 128 does not fit

```
segment code use32 class=code
start:
    mov al, 80h
    sub al, 01h             ; AL = 7Fh  CF=0 OF=1
    mov bl, 7Fh
    sub bl, 0FFh            ; BL = 80h  CF=1 OF=1
```

FINAL RESULT

add: 10h+20h (0,0), 7Fh+01h (0,1), F0h+20h (1,0), 80h+80h (1,1); sub: 30h−10h (0,0), 80h−01h (0,1), 10h−20h (1,0), 7Fh−FFh (1,1).

Exercise 6.11 — div versus idiv on a negative dividend CHALLENGE

STATEMENT

`mov ax, -300` / `mov bl, 3`. What happens with `idiv bl`, and what with `div bl`? Explain.

THEORY USED

`div` is unsigned, `idiv` signed (Chapter 5); division overflow (6.4).

STEP BY STEP

AX = −300 = **FED4h**.

- `idiv bl`: signed −300 / 3 = −100, which fits in AL → AL = 9Ch (−100), AH = 00h (remainder 0). Correct.
- `div bl`: unsigned FED4h = 65 236; 65 236 / 3 = 21 745 does not fit in AL → **division overflow**, run-time exception.

```
segment code use32 class=code
start:
    mov ax, -300
    mov bl, 3
    idiv bl                 ; AL = 9Ch (-100), AH = 00h
```

The same bits in AX mean −300 to `idiv` but 65 236 to `div`: the choice of instruction decides the interpretation.

FINAL RESULT

`idiv`: AL = 9Ch = −100, AH = 0. `div`: 65 236 / 3 = 21 745 does not fit in AL → exception.

Exercise 6.12 — A complete line-by-line analysis CHALLENGE

STATEMENT

Detail the effect of each line (hex, unsigned, signed, CF, OF) and explain every overflow:

```
mov al, 9Ch
add al, 0E4h
mov bl, 7Fh
sub bl, al
```

THEORY USED

Template of 6.5; rules of 6.2.

STEP BY STEP

line	result	unsigned	signed	CF	OF
`mov al, 9Ch`	AL = 9Ch	156	−100	–	–
`add al, 0E4h`	AL = 80h	128	−128	**1**	**0**
`mov bl, 7Fh`	BL = 7Fh	127	127	(unchanged)	(unchanged)
`sub bl, al`	BL = FFh	255	−1	**1**	**1**

- `add`: 9Ch + E4h = 180h → 80h. Unsigned 156 + 228 = 384 > 255 → CF = 1 (unsigned overflow: 128 is wrong). Signed −100 + (−28) = −128, fits → OF = 0 (−128 is correct).
- `mov` changes no flags.
- `sub`: 7Fh − 80h = FFh. Unsigned 127 − 128 < 0 → borrow, CF = 1 (255 is wrong). Signed 127 − (−128) = 255 > 127 → OF = 1 (−1 is wrong): positive − negative = negative.

```
segment code use32 class=code
start:
    mov al, 9Ch
    add al, 0E4h            ; AL = 80h  CF=1 OF=0
    mov bl, 7Fh
    sub bl, al              ; BL = FFh  CF=1 OF=1
```

FINAL RESULT

AL = 80h (CF = 1, OF = 0), then BL = FFh = 255 / −1 with CF = 1 and OF = 1: both results are wrong in their interpretation.

CHAPTER 7

Bitwise Instructions, Shifts and Rotates, Building Values from Bits

Working with individual bits: the logical instructions and masks, shifts and rotates, the difference between shift operators and shift instructions, and a step-by-step method for building a value out of bits of other values.

7.1 Logical instructions

The logical instructions work **bit by bit**: bit *k* of the result depends only on bit *k* of the operands.

Instruction	Bit of the result	Typical use
`and d, s`	1 only if both bits are 1	isolate / clear bits (mask)
`or d, s`	1 if at least one bit is 1	set bits, combine pieces
`xor d, s`	1 if the bits differ	invert chosen bits; `xor eax, eax` = 0
`not d`	the inverted bit (one's complement)	invert everything
`test d, s`	like `and`, but only the flags are kept	"is bit k set?"

Truth tables (1 is blue, 0 is orange):

x	y	x and y	x or y	x xor y	not x
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Flags: `and`, `or`, `xor`, `test` set **CF = OF = 0** and ZF, SF, PF from the result. `not` changes **no** flags.

```
segment code use32 class=code
start:
    mov al, 0B6h            ; 1011 0110
    and al, 0Fh             ; AL = 06h (0000 0110)  CF=0 OF=0 ZF=0
    mov bl, 0B6h
    or bl, 0Fh              ; BL = BFh (1011 1111)
    mov cl, 0B6h
    xor cl, 0Fh             ; CL = B9h (1011 1001)
    mov dl, 0B6h
    not dl                  ; DL = 49h (0100 1001)
```

7.2 Masks: the core trick

A **mask** is a constant whose bits select the positions you want to work on.

Goal	Instruction	Mask has 1 where...
keep some bits, clear the rest	`and`	the bits to keep
set some bits to 1	`or`	the bits to set
invert some bits	`xor`	the bits to invert
check some bits	`test`	the bits to check (then `jz` / `jnz`)

```
and ax, 0000000011110000b   ; keep only bits 4-7, clear the rest
or  ax, 1000000000000001b   ; set bits 0 and 15
xor ax, 0000000000001111b   ; invert bits 0-3
test al, 00000001b          ; ZF = 1 if AL is even (bit 0 = 0)
```

To **clear** some bits and keep the others, `and` with a mask that has 0 in the bits to clear: `and ax, 0FFF0h` clears bits 0–3.

THREE WAYS TO MAKE A REGISTER 0

`mov al, 0`, `and al, 0` and `xor al, al` all give AL = 0. The difference: `mov` changes no flags, the other two set ZF = 1 and clear CF and OF.

WRITING THE MASK IN HEX WITHOUT CHECKING THE BITS

Bits 2–9 are **not** "0FFh shifted"; write the mask in binary first (`0000 0011 1111 1100`b) and only then convert to hex (03FCh). One wrong nibble silently changes the result.

7.3 Shifts

Instruction	Effect
`shl d, n` / `sal d, n`	shift left; zeros enter on the right; the last bit shifted out goes to CF. Multiplies by 2^n
`shr d, n`	**logical** shift right; zeros enter on the left. Unsigned division by 2^n
`sar d, n`	**arithmetic** shift right; copies of the sign bit enter on the left. Signed division by 2^n (rounds toward $-\infty$)

```
shl:   CF <- [b7 b6 b5 b4 b3 b2 b1 b0] <- 0
shr:   0  -> [b7 b6 b5 b4 b3 b2 b1 b0] -> CF
sar:   b7 -> [b7 b6 b5 b4 b3 b2 b1 b0] -> CF      (b7 is kept and copied)
```

The count *n* is either a **constant** or the register **CL** — no other register:

```
shl eax, 4              ; OK
mov cl, 3
shr bx, cl              ; OK
shr bx, dl              ; ERROR - only CL
```

```
segment code use32 class=code
start:
    mov al, 0B4h            ; 1011 0100
    shl al, 1               ; AL = 68h (0110 1000), CF = 1
    mov bl, 0B4h
    shr bl, 1               ; BL = 5Ah (0101 1010), CF = 0
    mov cl, 0B4h
    sar cl, 1               ; CL = DAh (1101 1010), CF = 0
```

SAR AND IDIV ROUND DIFFERENTLY

`sar` rounds toward $-\infty$: −7 sar 1 = −4. `idiv` rounds toward 0: −7 / 2 = −3. They agree for non-negative numbers and for exact divisions.

7.4 Rotates

Instruction	Effect
`rol d, n`	rotate left: the bit leaving on the left re-enters on the right (and is copied to CF)
`ror d, n`	rotate right: the bit leaving on the right re-enters on the left (and is copied to CF)
`rcl d, n`	rotate left **through CF**: CF is an extra bit in the circle
`rcr d, n`	rotate right through CF

```
rol:   +--[b7 ... b0]<-+        rcl:   +--[b7 ... b0]<--CF<-+
       +-------->-------+  (CF = copy)  +----------->--------+
```

Useful idioms:

- `rol ax, 8` swaps AH and AL; `ror eax, 16` swaps the two words of EAX;
- `rol al, 4` swaps the two nibbles of AL;
- `rcl` / `rcr` let a bit travel from one register to another through CF.

```
segment code use32 class=code
start:
    mov ax, 1234h
    rol ax, 8               ; AX = 3412h (AH and AL swapped)
    mov ebx, 12345678h
    ror ebx, 16             ; EBX = 56781234h
    mov cl, 3Ch
    rol cl, 4               ; CL = C3h (nibbles swapped)
```

7.5 Shift operators versus shift instructions

`<<`, `>>`, `&`, `|`, `~`, `^` in NASM are **operators**: they are computed when the program is **assembled**, and they work on constants only. `shl`, `shr`, `and`, ... are **instructions** executed at run time on whatever is in the operand.

```
segment code use32 class=code
start:
    mov ax, 01110111b << 3  ; AX = 3B8h - NASM computed it, no shift instruction is generated
    mov bx, 01110111b
    shl bx, 3               ; BX = 3B8h - computed by the CPU at run time
```

7.6 Building a value from bits of other values

A frequent task: *"Given A (and B), compute C such that bits x–y of C are bits u–v of A, bits ... are 1, ..."*. The method, for every piece of the destination:

1. **Load** the source into a scratch register (zero-extend with `movzx` to 32 bits).
2. **Isolate** the needed bits with `and` and a mask.
3. **Move** them to the destination position with `shl` / `shr` (or `rol` / `ror`). Shift amount = destination position − source position (left if positive, right if negative).
4. **Combine** into the result register with `or`.

Constant pieces: bits that must be 1 → `or` with a mask; bits that must be 0 → start the result from 0 (or `and` with the inverted mask). Bits that must be inverted → `not` before isolating (or `xor` with the mask).

Worked case

Given the word A, compute the dword B:

- bits 28–31 of B are 1;
- bits 24–25 and 26–27 of B are bits 8–9 of A;
- bits 20–23 of B are the inverted bits 0–3 of A;
- bits 16–19 of B are 0;
- bits 0–15 of B are the same as bits 16–31 of B.

```
segment data use32 class=data
    a dw 0A5C3h
    b dd 0
segment code use32 class=code
start:
    mov ebx, 0              ; the result is built in EBX
    or ebx, 0F0000000h      ; bits 28-31 = 1
    movzx eax, word [a]
    and eax, 0300h          ; isolate bits 8-9 of A
    shl eax, 16             ; 8 -> 24 : bits 24-25
    or ebx, eax
    shl eax, 2              ; 24 -> 26 : bits 26-27
    or ebx, eax
    movzx eax, word [a]
    not eax                 ; invert
    and eax, 0Fh            ; inverted bits 0-3
    shl eax, 20             ; -> bits 20-23
    or ebx, eax
                            ; bits 16-19 stay 0 (we started from 0)
    mov eax, ebx
    shr eax, 16             ; bits 16-31 of B -> positions 0-15
    or ebx, eax
    mov [b], ebx            ; for A = A5C3h: B = F5C0F5C0h
```

Check by hand: bits 8–9 of A5C3h (`1010 0101 1100 0011`) are 01 → bits 24–27 = 0101 = 5; inverted bits 0–3: not 0011 = 1100 = C; so the high word is F5C0h and B = F5C0F5C0h.

7.7 Chapter summary

IN SHORT

- `and` keeps/clears, `or` sets, `xor` inverts, `test` checks; they clear CF and OF. `not` changes no flags.
- Write masks in binary first, then convert to hex.
- `shl` $\times 2^n$; `shr` unsigned $\div 2^n$; `sar` signed $\div 2^n$ toward $-\infty$; last bit out → CF; count = constant or CL.
- `rol` / `ror` circular; `rcl` / `rcr` through CF; `rol ax, 8` swaps bytes, `rol al, 4` swaps nibbles.
- `<< >> & | ~ ^` are assembly-time operators on constants.
- Build values: load (movzx) → isolate (and) → move (shl/shr) → combine (or).

Solutions — Chapter 7

Exercise 7.1 — Shifts and rotates of 96h BASIC

STATEMENT

What do `AL` and `CF` contain after `mov al, 96h` followed by each of (separately): `shl al, 2`, `shr al, 3`, `sar al, 3`, `rol al, 4`?

THEORY USED

Shifts (7.3) and rotates (7.4); the last bit shifted out goes to CF.

STEP BY STEP

96h = `1001 0110`b.

instruction	AL (binary)	AL	CF = last bit out
`shl al, 2`	`0101 1000`	58h	bit 6 = 0
`shr al, 3`	`0001 0010`	12h	bit 2 = 1
`sar al, 3`	`1111 0010`	F2h	bit 2 = 1
`rol al, 4`	`0110 1001`	69h	last bit rotated = 1

FINAL RESULT

58h/CF 0; 12h/CF 1; F2h/CF 1; 69h/CF 1.

Exercise 7.2 — Check a single bit BASIC

STATEMENT

Write instructions that jump to the label `set` if bit 5 of `BL` is 1.

THEORY USED

`test` with a mask, then `jnz` (7.2).

STEP BY STEP

```
segment code use32 class=code
start:
    mov bl, 00100001b
    mov al, 0
    test bl, 00100000b      ; bit 5; BL is not modified
    jnz set                 ; ZF = 0 means the bit was 1
    jmp done
set:
    mov al, 1               ; reached for this BL
done:
```

`test` computes BL AND 20h without storing it: the result is non-zero exactly when bit 5 is 1.

FINAL RESULT

`test bl, 00100000b` / `jnz set`.

Exercise 7.3 — Clear and set groups of bits BASIC

STATEMENT

Make bits 0–3 of `AX` zero and bits 12–15 one, leaving the other bits unchanged. Give AX for an initial value of `5A5Ah`.

THEORY USED

`and` with a mask to clear, `or` with a mask to set (7.2).

STEP BY STEP

```
segment code use32 class=code
start:
    mov ax, 5A5Ah
    and ax, 0FFF0h          ; clear bits 0-3   -> AX = 5A50h
    or ax, 0F000h           ; set bits 12-15   -> AX = FA50h
```

FINAL RESULT

`and ax, 0FFF0h` / `or ax, 0F000h`; 5A5Ah → FA50h.

Exercise 7.4 — and, or, xor, not on the same byte BASIC

STATEMENT

`AL = 0B6h`. Give AL after each instruction applied separately: `and al, 0Fh`, `or al, 0Fh`, `xor al, 0Fh`, `not al`, `xor al, al`. Which of them changes the flags?

THEORY USED

Logical instructions and their flags (7.1).

STEP BY STEP

0B6h = `1011 0110`, 0Fh = `0000 1111`.

instruction	result	flags
`and al, 0Fh`	`0000 0110` = 06h	CF = OF = 0, ZF = 0, SF = 0
`or al, 0Fh`	`1011 1111` = BFh	CF = OF = 0, SF = 1
`xor al, 0Fh`	`1011 1001` = B9h	CF = OF = 0, SF = 1
`not al`	`0100 1001` = 49h	none changed
`xor al, al`	00h	CF = OF = 0, ZF = 1

FINAL RESULT

06h, BFh, B9h, 49h, 00h; all except `not` change the flags.

Exercise 7.5 — shr versus sar INTERMEDIATE

STATEMENT

(a) Give AL after `shr al, 2` and after `sar al, 2` for `AL = 0F0h`, and interpret both results. (b) Compare `sar` by 1 with `idiv` by 2 for −7.

THEORY USED

Logical versus arithmetic shift (7.3).

STEP BY STEP

(a) F0h = **1111** 0000: unsigned 240, signed −16.

- `shr al, 2` → 0011 **1100** = 3Ch = 60 = 240 / 4 (unsigned division).
- `sar al, 2` → **1111 1100** = FCh = −4 = −16 / 4 (signed division).

(b) −7 = F9h = **1111 1001**. `sar al, 1` → **1111 1100** = FCh = **−4** (rounds toward −∞), while `idiv` gives **−3** (rounds toward 0).

```
segment code use32 class=code
start:
    mov al, 0F0h
    shr al, 2               ; AL = 3Ch
    mov bl, 0F0h
    sar bl, 2               ; BL = FCh
    mov cl, -7
    sar cl, 1               ; CL = FCh (-4)
```

FINAL RESULT

(a) 3Ch = 60 and FCh = −4; (b) −7 sar 1 = −4, but −7 idiv 2 = −3.

Exercise 7.6 — Rotating through the carry INTERMEDIATE

STATEMENT

Give AL and CF after `mov al, 81h` / `stc` / `rcl al, 1`, and then after a following `rcr al, 1`.

THEORY USED

`rcl` / `rcr`: CF is the 9th bit of the circle (7.4).

STEP BY STEP

- `rcl al, 1`: every bit moves left; the old CF (1) enters bit 0; the old bit 7 (1) goes to CF. **1**000 000**1** → 0000 00**11** = **03h**, **CF = 1**.
- `rcr al, 1`: every bit moves right; CF (1) enters bit 7; bit 0 (1) goes to CF. 0000 00**11** → **1**000 000**1** = **81h**, **CF = 1** — the original state is restored.

```
segment code use32 class=code
start:
    mov al, 81h
    stc
    rcl al, 1               ; AL = 03h, CF = 1
    mov bl, al
    rcr al, 1               ; AL = 81h, CF = 1
```

FINAL RESULT

After `rcl`: AL = 03h, CF = 1. After `rcr`: AL = 81h, CF = 1.

Exercise 7.7 — Swapping parts with one instruction INTERMEDIATE

STATEMENT

With one instruction each: swap the nibbles of `AL` (3Ch → C3h); swap `AH` and `AL`; swap the two words of `EAX`.

THEORY USED

Rotates (7.4).

STEP BY STEP

```
segment code use32 class=code
start:
    mov al, 3Ch
    rol al, 4               ; AL = C3h   (ror al, 4 works too)
    mov bx, 1234h
    rol bx, 8               ; BX = 3412h
    mov ecx, 12345678h
    ror ecx, 16             ; ECX = 56781234h (rol ecx, 16 works too)
```

Rotating by half the width moves one half into the place of the other.

FINAL RESULT

`rol al, 4`; `rol ax, 8`; `ror eax, 16`.

Exercise 7.8 — Count the 1 bits INTERMEDIATE

STATEMENT

Count how many bits of 1 are in the dword `x` (use a loop with `shr` and `adc`). Use `x = 0F0F0F0Fh` and `x = 80000001h`.

THEORY USED

`shr` sends bit 0 into CF (7.3); `adc ebx, 0` adds CF (Chapter 5); `loop` (Chapter 9).

STEP BY STEP

Each `shr eax, 1` moves the lowest bit into CF; `adc ebx, 0` adds that CF to the counter. Repeat 32 times.

```
segment data use32 class=data
    x dd 0F0F0F0Fh
segment code use32 class=code
start:
    mov eax, [x]
    mov ebx, 0              ; counter
    mov ecx, 32
count:
    shr eax, 1              ; bit 0 -> CF
    adc ebx, 0              ; EBX += CF
    loop count              ; EBX = 16 for 0F0F0F0Fh
```

For 80000001h the program gives 2.

FINAL RESULT

16 for 0F0F0F0Fh, 2 for 80000001h.

Exercise 7.9 — Build a dword from a byte and a word CHALLENGE

STATEMENT

Given the byte A and the word B, compute the dword C:

- bits 0–7 of C are 1;
- bits 8–11 of C are bits 4–7 of A;
- bits 12–19 of C are bits 2–9 of B;
- bits 20–23 of C are bits 0–3 of A;
- bits 24–31 of C are the high byte of B.

Give C for `A = 0B7h`, `B = 0E2D5h`.

THEORY USED

The method of 7.6.

STEP BY STEP

piece of C	source	mask	shift
0–7 = 1	constant	`0FFh`	–
8–11	A bits 4–7	`0F0h`	left 4
12–19	B bits 2–9	`03FCh` = `0000 0011 1111 1100`	left 10
20–23	A bits 0–3	`0Fh`	left 20
24–31	B bits 8–15	`0FF00h`	left 16

```
segment data use32 class=data
    a db 0B7h
    b dw 0E2D5h
    c dd 0
segment code use32 class=code
start:
    mov ebx, 0FFh           ; bits 0-7 = 1
    movzx eax, byte [a]
    and eax, 0F0h           ; bits 4-7 of A
    shl eax, 4              ; -> 8-11
    or ebx, eax
    movzx eax, word [b]
    and eax, 03FCh          ; bits 2-9 of B
    shl eax, 10             ; 2 -> 12
    or ebx, eax
    movzx eax, byte [a]
    and eax, 0Fh            ; bits 0-3 of A
    shl eax, 20             ; -> 20-23
    or ebx, eax
    movzx eax, word [b]
    and eax, 0FF00h         ; high byte of B (bits 8-15)
    shl eax, 16             ; -> 24-31
    or ebx, eax
    mov [c], ebx            ; C = E27B5BFFh
```

Check: B7h → bits 4–7 = B, bits 0–3 = 7; E2D5h bits 2–9 = `10 1101 01` → B5h; high byte E2h. Assemble: E2 | 7 | B5 | B | FF → **E27B5BFFh**.

FINAL RESULT

C = E27B5BFFh.

Exercise 7.10 — Build a byte from a word CHALLENGE

STATEMENT

Given the word A, compute the byte B:

- bits 0–2 of B are bits 5–7 of A;
- bits 3–5 of B are 1;
- bits 6–7 of B are the inverted bits 0–1 of A.

Give B for `A = 0B5C3h`.

THEORY USED

The method of 7.6 with right shifts; `not` before isolating (7.6).

STEP BY STEP

- bits 5–7 → 0–2: mask `0E0h`, shift **right** 5.
- bits 3–5 = 1: `or` with `0011 1000` = 38h.
- inverted bits 0–1 → 6–7: `not`, mask `03h`, shift left 6.

```
segment data use32 class=data
    a dw 0B5C3h
    b db 0
segment code use32 class=code
start:
    mov bl, 0
    or bl, 38h              ; bits 3-5 = 1
    mov ax, [a]
    and ax, 0E0h            ; bits 5-7 of A
    shr ax, 5               ; -> 0-2
    or bl, al
    mov ax, [a]
    not ax
    and ax, 03h             ; inverted bits 0-1
    shl ax, 6               ; -> 6-7
    or bl, al
    mov [b], bl             ; B = 3Eh
```

A = B5C3h: low byte C3h = **1100 0011**; bits 5–7 = 110 → 6; bits 0–1 = 11 → inverted 00. So B = **00 111 110** = **3Eh**.

FINAL RESULT

B = 3Eh.

Exercise 7.11 — Multiply by 10 with shifts CHALLENGE

STATEMENT

Compute `EAX = 10 · x` for the dword `x` without `mul` / `imul`, using x·10 = x·8 + x·2. Try it with x = 1234.

THEORY USED

`shl` by n multiplies by 2^n (7.3).

STEP BY STEP

```
segment data use32 class=data
    x dd 1234
segment code use32 class=code
start:
    mov eax, [x]
    shl eax, 1              ; EAX = 2x = 2468
    mov ebx, eax
    shl ebx, 2              ; EBX = 8x = 9872
    add eax, ebx            ; EAX = 10x = 12340
```

Shifts and additions are cheap; compilers use this trick for multiplication by small constants. The method is correct as long as 10·x fits in 32 bits.

FINAL RESULT

EAX = 12 340.

Exercise 7.12 — Reverse the bits of a byte CHALLENGE

STATEMENT

Reverse the order of the bits of the byte `x` (bit 0 ↔ bit 7, bit 1 ↔ bit 6, ...) using a loop with `shr` and `rcl`. Give the result for `x = 0B4h` and for `x = 01h`.

THEORY USED

`shr` puts bit 0 in CF; `rcl` takes CF into bit 0 of another register (7.3, 7.4).

STEP BY STEP

Take bits out of AL from the right (`shr al, 1` → CF) and push them into BL from the right (`rcl bl, 1`). After 8 steps the first bit taken out has travelled to bit 7 of BL.

```
segment data use32 class=data
    x db 0B4h
    r db 0
segment code use32 class=code
start:
    mov al, [x]             ; 1011 0100
    mov bl, 0
    mov ecx, 8
again:
    shr al, 1               ; lowest bit of AL -> CF
    rcl bl, 1               ; CF -> lowest bit of BL, older bits move left
    loop again
    mov [r], bl             ; r = 2Dh (0010 1101)
```

For x = 01h the result is 80h.

FINAL RESULT

B4h → 2Dh; 01h → 80h.

PART III

Memory, Control Flow and Strings

Addressing, comparisons and jumps, loops, string instructions.

CHAPTER 8

Addresses, Addressing Modes, Offset Computation

How an instruction designates a memory location: FAR and NEAR addresses, the offset formula base + index·scale + constant, the addressing modes, the implicit segment registers, pointer arithmetic and LEA.

8.1 Address and address specification

- An **address** identifies one byte of memory. In x86 protected mode a full (**FAR**) address is **segment_selector : offset** (16 + 32 bits).
- An **address specification** is the way you *write* an address in an instruction: `[ebx + esi*4 + 8]`, `[v]`, `[es:edi]`, ...
- A **NEAR** address is only the offset (32 bits); the segment is implied (the current one). A **FAR** address contains selector + offset; it is needed only to go to another segment (`jmp far`, `call far`).

In our programs we work almost only with **offsets**; the processor automatically combines them with the right segment register (section 8.4).

8.2 The offset formula

Every memory operand has an offset computed as

```
offset = [ base ] + [ index × scale ] + [ constant ]
```

Part	Allowed
base	any 32-bit general register: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
index	any 32-bit general register **except ESP**
scale	1, 2, 4 or 8
constant	a number or a variable name (the **displacement**); may be negative

Any combination of the three parts may appear, but at most **one base** and **one index**.

```
segment data use32 class=data
    a dw 7
    v dd 10, 20, 30, 40
segment code use32 class=code
start:
    mov ax, [a]             ; constant only        -> direct addressing, AX = 7
    mov ebx, v
    mov ecx, [ebx]          ; base only            -> ECX = 10
    mov esi, 2
    mov edx, [v + esi*4]    ; constant + index*scale = v[2] -> EDX = 30
    mov edi, [ebx + esi*4 + 4]   ; base + index*scale + constant = v[3] -> EDI = 40
```

```
mov eax, [esp*2]        ; ERROR - ESP cannot be an index
mov eax, [eax+ebx+ecx]  ; ERROR - too many registers
mov eax, [eax*3]        ; OK! NASM writes it as [eax + eax*2]
mov eax, [ebx*5]        ; OK: [ebx + ebx*4]
```

ARRAYS

For an array whose elements have size *s*, element *i* (counting from 0) is at `v + i·s`. With the index in a register: `[v + esi*s]` for s = 1, 2, 4, 8 — this is exactly what the scale is for.

8.3 Addressing modes

Mode	The operand is...	Example	Computed
immediate	a constant inside the instruction	`mov eax, 5`	at assembly time
register	a register	`mov eax, ebx`	–
direct memory	in memory; offset = a constant only	`mov eax, [v]`	offset at assembly time, full address at load time
indirect memory	in memory; the offset uses registers	`mov eax, [ebx+4]`	at run time

The indirect variants named in the course:

- **base** (register indirect): `[ebx]`
- **base / index + displacement**: `[ebx+4]`, `[array + esi]`
- **base + index + displacement**: `[ebp+esi+8]`
- **scaled index**: `[ebx+esi*4+disp]`

An instruction may combine modes: `add [ebx+4], eax` has an indirect-memory destination and a register source.

8.4 Implicit segment registers

The processor chooses the segment register from the **base** register of the address:

- base = **EBP** or **ESP** → **SS** (the stack segment);
- any other base, or no base → **DS** (the data segment);
- string instructions: source **DS:ESI**, destination **ES:EDI** (fixed, Chapter 10);
- instruction fetch: **CS:EIP**.

Which register is the base?

When two registers appear, NASM decides:

- with no scale written, the **first** register is the base;
- a register written with `*scale` is the index;
- if both are written with `*1`, the **first** one is the index (the other is the base).

Specification	Base	Segment
`[ebx + ebp]`	EBX	DS
`[ebp + ebx]`	EBP	SS
`[ebx*1 + ebp*1]`	EBP	SS
`[ebp*2]`	EBP (NASM writes `[ebp+ebp]`)	SS

You can override the default with a **segment prefix**: `mov eax, [es:ebx]` (NASM also accepts `es:[ebx]`). In flat Windows programs all segments cover the same memory, so this rarely changes the result — but it does change the encoding of the instruction (Chapter 13).

8.5 Pointer arithmetic at assembly time

A variable name is a **pointer** (an offset). With pointers NASM allows:

- pointer + constant → pointer (`a + 4`)
- pointer − constant → pointer
- pointer − pointer (same segment) → **scalar** (`$ - a`, `b - a`)

Not allowed: pointer + pointer, pointer · constant, `!pointer`, `~pointer` (these need a scalar).

```
len equ $ - s           ; scalar
mov eax, s + 2          ; pointer: the address of s[2]
mov eax, [s + 2]        ; the value at s[2]
x dd a + b              ; ERROR - sum of two pointers
```

ADDRESS VERSUS VALUE, AGAIN

`mov eax, v+4` loads an **address** (the offset of the second dword of `v`); `mov eax, [v+4]` loads the **value** stored there. The brackets make all the difference.

8.6 LEA: computing an address

`lea reg, [address formula]` puts the computed **offset** (not the memory content) into `reg`. It does not access memory and does not change flags, so it is also a quick calculator for expressions of the form base + index·scale + constant.

```
segment data use32 class=data
    s db 'abcdef'
    t dw 0, 0, 0, 0, 0
segment code use32 class=code
start:
    lea esi, [s]            ; ESI = address of s (same as mov esi, s)
    mov ebx, 6
    lea eax, [ebx + ebx*4]  ; EAX = EBX*5 = 30
    mov ecx, 1
    lea edi, [t + ecx*2 + 6]; EDI = address of t[4]
```

8.7 Chapter summary

IN SHORT

- FAR address = selector : offset; NEAR = offset only; we work with offsets.
- offset = base + index·scale + constant; scale ∈ {1, 2, 4, 8}; ESP never an index; one base, one index.
- Modes: immediate, register, direct memory (constant offset), indirect memory (registers in the offset).
- Base EBP/ESP → SS, otherwise DS; strings DS:ESI / ES:EDI; code CS:EIP. Override with `[es:ebx]`.
- Pointer ± constant → pointer; pointer − pointer → scalar; pointer + pointer is invalid.
- `lea` computes the offset without reading memory and without touching the flags.

Solutions — Chapter 8

Exercise 8.1 — Classify the addressing modes BASIC

STATEMENT

Classify the addressing mode of each operand: `mov [ebx+4], eax`, `add ecx, 10`, `sub eax, [v]`, `inc dword [v+esi*4]`.

THEORY USED

Addressing modes (8.3).

STEP BY STEP

instruction	destination	source
`mov [ebx+4], eax`	indirect memory (base + displacement)	register
`add ecx, 10`	register	immediate
`sub eax, [v]`	register	direct memory
`inc dword [v+esi*4]`	indirect memory (index·scale + displacement)	–

FINAL RESULT

See the table: indirect/register; register/immediate; register/direct; indirect.

Exercise 8.2 — Which segment register? BASIC

STATEMENT

Which segment register is used for `[ebp+8]`, `[ebx+esi]`, `[esi+ebp]`, `[ebp*2]`, `[esp+4]`?

THEORY USED

Implicit segment registers and the choice of the base (8.4).

STEP BY STEP

specification	base	segment
`[ebp+8]`	EBP	SS
`[ebx+esi]`	EBX (first register)	DS
`[esi+ebp]`	ESI (first register)	DS
`[ebp*2]`	NASM encodes `[ebp+ebp]`, base EBP	SS
`[esp+4]`	ESP	SS

FINAL RESULT

SS, DS, DS, SS, SS.

Exercise 8.3 — Valid or invalid? BASIC

STATEMENT

Which of these are invalid, and why? `[eax+ebx+ecx]`, `[esp*2]`, `[ebx*3]`, `[ebx*5]`, `[a+b]` (`a`, `b`

variables), `[ebx-4]`.

THEORY USED

The offset formula (8.2); pointer arithmetic (8.5).

STEP BY STEP

- `[eax+ebx+ecx]` — **invalid**: at most one base and one index.
- `[esp*2]` — **invalid**: ESP cannot be an index.
- `[ebx*3]` — valid: NASM rewrites it as `[ebx+ebx*2]`.
- `[ebx*5]` — valid: `[ebx+ebx*4]`.
- `[a+b]` — **invalid**: the sum of two addresses is not a valid expression.
- `[ebx-4]` — valid: the constant may be negative.

FINAL RESULT

Invalid: `[eax+ebx+ecx]`, `[esp*2]`, `[a+b]`. Valid: `[ebx*3]`, `[ebx*5]`, `[ebx-4]`.

Exercise 8.4 — An element of a dword array BASIC

STATEMENT

`v dd 10, 20, 30, 40`. Put `v[2]` into `EAX` using `ESI = 2`.

THEORY USED

Scaled index for arrays (8.2).

STEP BY STEP

Dword elements have size 4, so v[2] is at v + 2·4:

```
segment data use32 class=data
    v dd 10, 20, 30, 40
segment code use32 class=code
start:
    mov esi, 2
    mov eax, [v + esi*4]    ; EAX = 30
```

FINAL RESULT

`mov eax, [v + esi*4]` → EAX = 30.

Exercise 8.5 — v+4 versus [v+4] INTERMEDIATE

STATEMENT

For `v dd 10, 20, 30, 40`, what is the difference between `mov eax, v+4` and `mov eax, [v+4]`? What does each load?

THEORY USED

Pointer arithmetic and brackets (8.5).

STEP BY STEP

- `mov eax, v+4` loads the **address** of the second element (offset of v plus 4) — an immediate operand.
- `mov eax, [v+4]` loads the **value** of the second element, 20 — a direct memory operand.

```
segment data use32 class=data
    v dd 10, 20, 30, 40
segment code use32 class=code
start:
    mov eax, v+4            ; EAX = address of v[1]
    mov ebx, [v+4]          ; EBX = 20
    mov ecx, [eax]          ; ECX = 20 too: EAX holds the address
```

FINAL RESULT

`v+4` = address of v[1]; `[v+4]` = 20.

Exercise 8.6 — LEA as a calculator INTERMEDIATE

STATEMENT

Compute `EAX` after `mov ebx, 10` / `lea eax, [ebx*8 + ebx + 1]`. Then write a single `lea` for each: EAX = EBX·3 + 7; EAX = ECX·9; EDX = EAX·4 + EBX.

THEORY USED

`lea` computes base + index·scale + constant (8.6).

STEP BY STEP

`[ebx*8 + ebx + 1]` = 10·8 + 10 + 1 = **91**.

- EAX = EBX·3 + 7 → `lea eax, [ebx + ebx*2 + 7]`
- EAX = ECX·9 → `lea eax, [ecx + ecx*8]`
- EDX = EAX·4 + EBX → `lea edx, [ebx + eax*4]`

```
segment code use32 class=code
start:
    mov ebx, 10
    lea eax, [ebx*8 + ebx + 1]  ; EAX = 91
    lea esi, [ebx + ebx*2 + 7]  ; ESI = 37
    mov ecx, 5
    lea edi, [ecx + ecx*8]      ; EDI = 45
    lea edx, [ebx + eax*4]      ; EDX = 10 + 91*4 = 374
```

FINAL RESULT

EAX = 91; `lea eax, [ebx+ebx*2+7]`, `lea eax, [ecx+ecx*8]`, `lea edx, [ebx+eax*4]`.

Exercise 8.7 — Offsets and overlapping reads INTERMEDIATE

STATEMENT

The data segment starts at offset 0:

```
a db 1, 2, 3
b dw 4, 5
c dd 6
```

(a) What is the offset of `c`? (b) What offset does `[b+2]` designate? (c) What are `word [a+1]` and `dword [b-1]`?

THEORY USED

Offsets accumulate (Chapter 4); little endian; NASM ignores declared types (8.5).

STEP BY STEP

Memory: offset 0: `01 02 03`, offset 3: `04 00 05 00`, offset 7: `06 00 00 00`.

(a) c is at 3 + 4 = **7**.

(b) b + 2 = 3 + 2 = **5** (the second word of b).

(c) `word [a+1]` = bytes at 1, 2 = 02 03 → **0302h**. `dword [b-1]` = bytes at 2...5 = 03 04 00 05 → **05000403h**.

```
segment data use32 class=data
    a db 1, 2, 3
    b dw 4, 5
    c dd 6
segment code use32 class=code
start:
    mov ax, [a+1]           ; AX = 0302h
    mov ebx, [b-1]          ; EBX = 05000403h
```

FINAL RESULT

(a) 7; (b) 5; (c) 0302h and 05000403h.

Exercise 8.8 — Sum of an array, two ways INTERMEDIATE

STATEMENT

Compute the sum of the dword array `v dd 5, 10, 15, 20, 25` twice: once with `[v + esi*4]` and an index, once with a pointer register `EBX` that advances by 4.

THEORY USED

Scaled index (8.2) and base register (8.3); `loop` (Chapter 9).

STEP BY STEP

```
segment data use32 class=data
    v dd 5, 10, 15, 20, 25
    n equ ($ - v) / 4
segment code use32 class=code
start:
    mov eax, 0
    mov esi, 0              ; index
    mov ecx, n
sum1:
    add eax, [v + esi*4]    ; EAX += v[esi]
    inc esi
    loop sum1               ; EAX = 75
    mov edx, 0
    mov ebx, v              ; pointer to v[0]
    mov ecx, n
sum2:
    add edx, [ebx]          ; EDX += *ebx
    add ebx, 4              ; next element
    loop sum2               ; EDX = 75
```

The first version counts elements and lets the scale do the multiplication; the second keeps an address and adds the element size itself.

FINAL RESULT

Both give 75.

Exercise 8.9 — A matrix stored by rows CHALLENGE

STATEMENT

A 3×4 matrix of bytes is stored row by row in `m`. Write the address formula of element (i, j) and load `m[2][1]` into `AL` with `i` in `ESI` and `j` in `EDI`.

THEORY USED

Offset formula (8.2), LEA (8.6).

STEP BY STEP

Each row has 4 bytes, so element (i, j) is at **m + 4·i + j**.

```
segment data use32 class=data
    m db 1, 2, 3, 4
      db 5, 6, 7, 8
      db 9, 10, 11, 12
segment code use32 class=code
start:
    mov esi, 2              ; i
    mov edi, 1              ; j
    lea ebx, [esi*4]        ; EBX = 4*i = 8
    mov al, [m + ebx + edi] ; AL = m[2][1] = 10
    mov cl, [m + esi*4 + edi] ; the same in one formula: CL = 10
```

`[m + esi*4 + edi]` fits the formula directly: constant m, index ESI with scale 4, base EDI.

FINAL RESULT

Address m + 4i + j; AL = 10.

Exercise 8.10 — Pointer arithmetic CHALLENGE

STATEMENT

With `a` and `b` variables in the data segment, which of the following are valid at assembly time, and is each a pointer or a scalar? `a+4`, `b-a`, `a+b`, `a*2`, `$-a`, `!a`, `~(b-a)`, `a-4`.

THEORY USED

Pointer arithmetic (8.5).

STEP BY STEP

expression	valid?	kind
`a+4`	yes	pointer
`b-a`	yes	scalar
`a+b`	no — pointer + pointer	–
`a*2`	no — pointer · constant	–
`$-a`	yes	scalar
`!a`	no — `!` needs a scalar	–
`~(b-a)`	yes	scalar
`a-4`	yes	pointer

FINAL RESULT

Valid: a+4 (pointer), b−a (scalar), $−a (scalar), ~(b−a) (scalar), a−4 (pointer). Invalid: a+b, a*2, !a.

Exercise 8.11 — Base or index: how NASM decides CHALLENGE

STATEMENT

Which register is the base, and which segment register is used, for `[ebx+ebp]`, `[ebp+ebx]`, `[ebx+ebp*1]`, `[ebx*1+ebp*1]`, `[ebp*1+ebx*1]`? What does `mov eax, [es:ebx]` change?

THEORY USED

Choice of the base and implicit segments (8.4).

STEP BY STEP

specification	base	index	segment	rule
`[ebx+ebp]`	EBX	EBP	DS	no scale: first is base
`[ebp+ebx]`	EBP	EBX	SS	no scale: first is base
`[ebx+ebp*1]`	EBX	EBP	DS	the scaled one is the index
`[ebx*1+ebp*1]`	EBP	EBX	SS	both scaled: first is the index
`[ebp*1+ebx*1]`	EBX	EBP	DS	both scaled: first is the index

`[es:ebx]` adds a **segment-override prefix**: the offset EBX is taken in ES instead of DS. The instruction gets one more byte (26h, Chapter 13).

FINAL RESULT

Bases: EBX, EBP, EBX, EBP, EBX → segments DS, SS, DS, SS, DS. `[es:ebx]` uses ES instead of DS.

Exercise 8.12 — Walk an array of words backwards CHALLENGE

STATEMENT

Copy the word array `s dw 1, 2, 3, 4, 5` into `d` in reverse order (d = 5, 4, 3, 2, 1), using one register as an index going up and another going down.

THEORY USED

Scaled index (8.2); offsets of the last element (8.5).

STEP BY STEP

Element i of s goes to position n − 1 − i of d.

```
segment data use32 class=data
    s dw 1, 2, 3, 4, 5
    n equ ($ - s) / 2
    d times n dw 0
segment code use32 class=code
start:
    mov esi, 0              ; i = 0 (up)
    mov edi, n - 1          ; n-1-i (down)
    mov ecx, n
copy:
    mov ax, [s + esi*2]     ; AX = s[i]
    mov [d + edi*2], ax     ; d[n-1-i] = s[i]
    inc esi
    dec edi
    loop copy               ; d = 5, 4, 3, 2, 1
```

FINAL RESULT

d = 5, 4, 3, 2, 1.

CHAPTER 9

Comparisons, Jumps, Loops (if / while / for in Assembly)

How decisions and repetition are written in assembly: CMP and the conditional jumps, signed versus unsigned conditions, jump distances, the LOOP family, and the translation of if, while, for and do-while.

9.1 CMP + conditional jump = IF

`cmp a, b` computes **a – b**, discards the result and only sets the flags (exactly like `sub`). The **conditional jump** written right after it reads the flags and jumps — or not.

```
cmp eax, ebx
jg bigger               ; jump if EAX > EBX (signed)
; ... code for EAX <= EBX
bigger:
```

A conditional jump does not have to follow a `cmp`: it reads whatever flags the previous instruction left (`test`, `sub`, `dec`, `and` ...). But the instructions between the flag-setting instruction and the jump must not change the flags (`mov` is safe, `add` is not).

9.2 The jump table

Relation	unsigned	signed	Condition checked
a = b	`je` / `jz`	`je` / `jz`	ZF = 1
a ≠ b	`jne` / `jnz`	`jne` / `jnz`	ZF = 0
a > b	`ja` (above) / `jnbe`	`jg` (greater) / `jnle`	unsigned: CF = 0 and ZF = 0 · signed: ZF = 0 and SF = OF
a ≥ b	`jae` / `jnb` / `jnc`	`jge` / `jnl`	CF = 0 · SF = OF
a < b	`jb` (below) / `jnae` / `jc`	`jl` (less) / `jnge`	CF = 1 · SF ≠ OF
a ≤ b	`jbe` / `jna`	`jle` / `jng`	CF = 1 or ZF = 1 · ZF = 1 or SF ≠ OF

Jumps on a single flag: `jc` / `jnc`, `jz` / `jnz`, `js` / `jns`, `jo` / `jno`, `jp` / `jnp` (`jpe` / `jpo`). Register check: `jecxz` (jump if ECX = 0), `jcxz` (if CX = 0).

MEMORY HOOK

Above/**B**elow = unsigned; **G**reater/**L**ess = signed. "A, B" for "bytes as they are"; "G, L" for numbers with a sign.

WHY "SF = OF" MEANS "GREATER OR EQUAL" FOR SIGNED VALUES

After `cmp a, b`, SF is the sign of the computed difference. If there was no signed overflow (OF = 0), that sign is correct: SF = 0 means a − b ≥ 0. If there was an overflow (OF = 1), the sign is inverted: SF = 1 then means a − b ≥ 0. In both cases a ≥ b exactly when SF = OF.

9.3 Why signed and unsigned conditions differ

Take `cmp al, bl` with AL = 0FFh and BL = 01h.

- Unsigned: 255 > 1 → `ja` jumps.
- Signed: −1 < 1 → `jg` does **not** jump (`jl` does).

```
segment code use32 class=code
start:
    mov al, 0FFh
    mov bl, 1
    mov cl, 0
    mov dl, 0
    cmp al, bl              ; CF=0 ZF=0 SF=1 OF=0
    jbe skip1
    mov cl, 1               ; executed: unsigned 255 > 1
skip1:
    cmp al, bl
    jle skip2
    mov dl, 1               ; NOT executed: signed -1 < 1
skip2:                      ; CL = 1, DL = 0
```

CHOOSING THE JUMP BY HABIT

The values in a register do not say whether they are signed. Decide from the problem statement; then use only `ja/jb/jae/jbe` (unsigned) or only `jg/jl/jge/jle` (signed).

9.4 JMP and jump distances

`jmp label` is the **unconditional** jump; it can reach any distance.

- A **short** jump is encoded with a 1-byte displacement: the target must be within −128 … +127 bytes of the next instruction.
- A **near** jump uses a 32-bit displacement: anywhere in the same segment.
- A **far** jump also changes CS (`jmp far …`); not used in our programs.

LOOP AND JECXZ ARE ONLY SHORT

In NASM, `loop` (and `jecxz`) exist only in the short form. If the label is more than 127 bytes away you get *"short jump is out of range"*. The conditional jumps (`jz`, `jne`, …) are automatically made near by NASM.

Two ways out of "short jump is out of range":

1. Replace `loop L` by `dec ecx` / `jnz L` — but `dec` changes ZF, SF, OF, PF, AF, while `loop` changes **no** flags.
2. Keep `loop` but let it jump to a nearby `jmp` that goes back to the start (Exercise 9.10).

9.5 The LOOP instructions

Instruction	Effect
`loop L`	ECX = ECX − 1; jump to L if ECX ≠ 0
`loope L` / `loopz L`	ECX = ECX − 1; jump if ECX ≠ 0 **and** ZF = 1
`loopne L` / `loopnz L`	ECX = ECX − 1; jump if ECX ≠ 0 **and** ZF = 0

None of them changes the flags; the decrement happens **before** the check.

ECX = 0 BEFORE THE LOOP

If ECX = 0 when `loop` executes, it becomes FFFFFFFFh ≠ 0 and the loop runs **4 294 967 296** times. Always guard a loop whose count may be 0 with `jecxz end` **before** entering it.

CHANGING ECX INSIDE THE BODY

The body must not modify ECX (no `dec ecx`, no `mov ecx, …`, and no C function calls, which destroy ECX). Otherwise the counter is wrong and the loop may never end.

9.6 Translating if / else

Jump **over** the code that must not run. The condition in assembly is usually the **negation** of the C condition, because we jump to skip.

```
segment data use32 class=data
    a dd -5
    b dd 3
    max dd 0
segment code use32 class=code
start:
    ; if (a > b) max = a; else max = b;   (signed dwords)
    mov eax, [a]
    cmp eax, [b]
    jg take_a               ; a > b -> keep EAX = a
    mov eax, [b]            ; else EAX = b
take_a:
    mov [max], eax          ; max = 3
```

General shape:

```
        cmp  ...                ; condition
        j<not condition>  else_part
        ...  then part ...
        jmp  end_if
else_part:
        ...  else part ...
end_if:
```

9.7 Translating while, for and do-while

while

```
segment data use32 class=data
    n dd 5
    s dd 0
segment code use32 class=code
start:
    ; while (n > 0) { s += n; n--; }
while_start:
    cmp dword [n], 0
    jle while_end           ; condition false -> leave
    mov eax, [n]
    add [s], eax
    dec dword [n]
    jmp while_start
while_end:                  ; s = 15, n = 0
```

for with loop

```
segment data use32 class=data
    v db 10, 20, 30, 40
    len equ $ - v
    sum dw 0
segment code use32 class=code
start:
    ; for (i = 0; i < len; i++) sum += v[i];  v = bytes, sum = word
    mov ecx, len
    mov esi, 0
    mov ax, 0
    jecxz done              ; guard: len may be 0
again:
    movzx bx, byte [v + esi]
    add ax, bx
    inc esi
    loop again
done:
    mov [sum], ax           ; sum = 100
```

do-while with a counter that is not ECX

```
segment data use32 class=data
    v db 1, 2, 3, 4
    len equ $ - v
segment code use32 class=code
start:
    mov esi, 0
    mov al, 0
repeat:
    add al, [v + esi]       ; ... work with element esi ...
    inc esi
    cmp esi, len
    jb repeat               ; AL = 10
```

A do-while executes the body at least once; it fits when the count can never be 0.

9.8 Chapter summary

IN SHORT

- `cmp a, b` = `sub` without storing; the following jump reads the flags.
- Unsigned: `ja jae jb jbe`; signed: `jg jge jl jle`; both: `je jne`.
- `loop` / `jecxz` are short (±127 bytes); conditional jumps are made near by NASM.
- `loop`: ECX−1, jump if ≠ 0, no flags changed. Guard with `jecxz`; never change ECX in the body.
- if/else: jump over the part that must not run (negated condition). while: check at the top. do-while: check at the bottom.

Solutions — Chapter 9

Exercise 9.1 — How many times? (1) BASIC

STATEMENT

How many times does the body run? `mov ecx, 5` / `L: dec ecx` / `loop L`.

THEORY USED

`loop` decrements ECX before checking it (9.5); never change ECX in the body.

STEP BY STEP

Follow ECX: 5 → `dec` 4 → `loop` 3 (jump) → `dec` 2 → `loop` 1 (jump) → `dec` 0 → `loop` FFFFFFFFh ≠ 0 (jump) → `dec` FFFFFFFEh → `loop` FFFFFFFDh ...

Each iteration subtracts 2, so after `loop` ECX is always **odd** and never 0: the loop is **infinite**.

FINAL RESULT

Infinitely many times — never change ECX inside a `loop` body.

Exercise 9.2 — How many times? (2) BASIC

STATEMENT

How many times does the body run? `mov ecx, 0` / `L: inc ebx` / `loop L`.

THEORY USED

`loop` with ECX = 0 (9.5).

STEP BY STEP

The body runs once, then `loop` makes ECX = FFFFFFFFh ≠ 0 and jumps. From there ECX goes down to 0, so the body runs 1 + (2^{32} − 1) = 2^{32} = **4 294 967 296** times. A `jecxz` before `L` would have skipped the loop.

FINAL RESULT

2^{32} = 4 294 967 296 times.

Exercise 9.3 — Which jumps are taken? BASIC

STATEMENT

After `mov al, 80h` / `cmp al, 7Fh`, which of `ja`, `jg`, `jb`, `jl` jump?

THEORY USED

The jump table and signed versus unsigned (9.2, 9.3).

STEP BY STEP

80h − 7Fh:

- unsigned: 128 > 127 → **`ja` jumps**, `jb` does not;
- signed: −128 < 127 → **`jl` jumps**, `jg` does not.

Flags: 80h − 7Fh = 01h; CF = 0 (no borrow), ZF = 0, SF = 0, OF = 1 (negative − positive gave positive). `ja`: CF = 0 and ZF = 0 ✓; `jl`: SF ≠ OF ✓.

FINAL RESULT

`ja` and `jl` jump; `jg` and `jb` do not.

Exercise 9.4 — Minimum of two numbers BASIC

STATEMENT

Store in `m` the minimum of the dwords `a = -5` and `b = 3`, first treating them as signed, then as unsigned. Why are the results different?

THEORY USED

if/else (9.6) with signed and unsigned jumps (9.2).

STEP BY STEP

```
segment data use32 class=data
    a dd -5
    b dd 3
    ms dd 0
    mu dd 0
segment code use32 class=code
start:
    mov eax, [a]
    cmp eax, [b]
    jl keep_s               ; signed: a < b -> keep a
    mov eax, [b]
keep_s:
    mov [ms], eax           ; ms = -5
    mov eax, [a]
    cmp eax, [b]
    jb keep_u               ; unsigned: FFFFFFFBh < 3 ? no
    mov eax, [b]
keep_u:
    mov [mu], eax           ; mu = 3
```

−5 = FFFFFFFBh. Signed it is smaller than 3; unsigned it is 4 294 967 291, larger than 3.

FINAL RESULT

Signed minimum −5; unsigned minimum 3.

Exercise 9.5 — Maximum of a string of signed bytes INTERMEDIATE

STATEMENT

Compute the maximum of the string of signed bytes `s db 5, -7, 20, -100, 13` into `max`.

THEORY USED

Loop with `loop` (9.5), signed comparison `jle` (9.2).

STEP BY STEP

Start with max = s[0] and compare it with the other len − 1 elements.

```
segment data use32 class=data
    s db 5, -7, 20, -100, 13
    len equ $ - s
    max db 0
segment code use32 class=code
start:
    mov esi, 0
    mov al, [s]             ; max = s[0]
    mov ecx, len - 1
    jecxz done
next:
    inc esi
    cmp [s + esi], al
    jle not_bigger          ; signed comparison
    mov al, [s + esi]
not_bigger:
    loop next
done:
    mov [max], al           ; max = 20
```

FINAL RESULT

max = 20.

Exercise 9.6 — Count the even words INTERMEDIATE

STATEMENT

Count the even numbers in the string of words `w dw 3, 8, 10, 7, 2` into the byte `cnt`.

THEORY USED

`test` on bit 0 (Chapter 7); loop with a pointer register (9.7).

STEP BY STEP

A number is even exactly when bit 0 is 0.

```
segment data use32 class=data
    w dw 3, 8, 10, 7, 2
    len equ ($ - w) / 2
    cnt db 0
segment code use32 class=code
start:
    mov esi, w
    mov ecx, len
    jecxz done
next:
    mov ax, [esi]
    test ax, 1              ; bit 0 = 0 -> even
    jnz odd
    inc byte [cnt]
odd:
    add esi, 2
    loop next
done:                       ; cnt = 3
```

FINAL RESULT

cnt = 3 (8, 10, 2).

Exercise 9.7 — Sum of the elements greater than 10 INTERMEDIATE

STATEMENT

For the unsigned bytes `v db 4, 12, 200, 10, 11, 9`, compute in the word `s` the sum of the elements strictly greater than 10.

THEORY USED

Unsigned condition `jbe` (9.2); widening before adding (Chapter 5).

STEP BY STEP

```
segment data use32 class=data
    v db 4, 12, 200, 10, 11, 9
    len equ $ - v
    s dw 0
segment code use32 class=code
start:
    mov esi, 0
    mov ecx, len
    jecxz done
next:
    mov al, [v + esi]
    cmp al, 10
    jbe skip                ; unsigned: <= 10 -> skip
    mov ah, 0               ; AX = element (zero extension)
    add [s], ax
skip:
    inc esi
    loop next
done:                       ; s = 12 + 200 + 11 = 223
```

FINAL RESULT

s = 223.

Exercise 9.8 — Factorial with a while loop INTERMEDIATE

STATEMENT

Compute `f = n!` for the byte `n = 6` into the dword `f`, using a while loop that stops when the counter reaches 1.

THEORY USED

while translation (9.7); `mul` dword (Chapter 5).

STEP BY STEP

```
segment data use32 class=data
    n db 6
    f dd 0
segment code use32 class=code
start:
    mov eax, 1              ; f = 1
    movzx ebx, byte [n]     ; k = n
while_start:
    cmp ebx, 1
    jbe while_end           ; while (k > 1)
    mul ebx                 ; EDX:EAX = f * k (EDX = 0 for these values)
    dec ebx
    jmp while_start
while_end:
    mov [f], eax            ; f = 720
```

For n = 0 or 1 the loop does not run and f = 1, which is correct.

FINAL RESULT

f = 720.

Exercise 9.9 — loope: skip the leading zeros INTERMEDIATE

STATEMENT

For `s db 0, 0, 0, 7, 0, 5`, use `loope` to find the index of the first non-zero byte (store it in `pos`, or `len` if all bytes are zero).

THEORY USED

`loope` jumps while ECX ≠ 0 and ZF = 1 (9.5).

STEP BY STEP

Compare each byte with 0; `loope` continues while the comparison says "equal" (ZF = 1).

```
segment data use32 class=data
    s db 0, 0, 0, 7, 0, 5
    len equ $ - s
    pos dd 0
segment code use32 class=code
start:
    mov esi, -1
    mov ecx, len
    jecxz all_zero
next:
    inc esi
    cmp byte [s + esi], 0
    loope next              ; continue while zero and elements remain
    jne found               ; ZF = 0: a non-zero byte stopped the loop
all_zero:
    mov esi, len
found:
    mov [pos], esi          ; pos = 3
```

After the loop, ZF still holds the result of the last `cmp` (`loope` does not change flags): ZF = 0 means a non-zero element was found at index ESI.

FINAL RESULT

pos = 3.

Exercise 9.10 — A loop whose body is too long CHALLENGE

STATEMENT

A `loop L` gives *short jump is out of range* because its body is longer than 127 bytes. Rewrite it so that it still uses `loop` (and therefore changes no flags). Show that it works with a body of 200 `nop`s that also increments `EAX`, executed 3 times.

THEORY USED

`loop` is only short; `jmp` can be near (9.4).

STEP BY STEP

Let `loop` jump to a nearby `jmp` that goes back to the start of the body:

```
segment code use32 class=code
start:
    mov eax, 0
    mov ecx, 3
L:
    inc eax
    times 200 nop           ; a long body (200 bytes)
    loop go_back            ; short: go_back is right below
    jmp done                ; ECX = 0: leave
go_back:
    jmp L                   ; near jump back to the start
done:                       ; EAX = 3
```

`loop` and `jmp` change no flags, so the flags produced inside the body are preserved — which `dec ecx` / `jnz L` would not guarantee.

FINAL RESULT

`loop go_back` / `jmp done` / `go_back: jmp L` — EAX = 3 after 3 iterations.

Exercise 9.11 — Is the number prime? CHALLENGE

STATEMENT

Set the byte `prime` to 1 if the dword `n` (n ≥ 2) is prime, else to 0, by trying all divisors d = 2, 3, ... while d·d ≤ n. Check with n = 97 and n = 91.

THEORY USED

while loop (9.7); `div` and the remainder in EDX (Chapter 5).

STEP BY STEP

```
segment data use32 class=data
    n dd 97
    prime db 1
segment code use32 class=code
start:
    mov ebx, 2              ; d = 2
try:
    mov eax, ebx
    mul ebx                 ; EAX = d*d
    cmp eax, [n]
    ja done                 ; d*d > n -> no divisor found: prime
    mov eax, [n]
    mov edx, 0
    div ebx                 ; EDX = n mod d
    cmp edx, 0
    je not_prime
    inc ebx
    jmp try
not_prime:
    mov byte [prime], 0
done:                       ; prime = 1 for 97
```

For n = 91 the loop finds d = 7 (91 = 7 · 13) and sets prime = 0.

FINAL RESULT

97 → prime = 1; 91 → prime = 0.

Exercise 9.12 — Bubble sort CHALLENGE

STATEMENT

Sort the string of signed bytes `s db 5, -3, 12, 0, -8, 7` in ascending order with bubble sort (repeat passes until a pass makes no swap).

THEORY USED

Nested loops, signed comparison `jle` (9.2), do-while (9.7).

STEP BY STEP

```
segment data use32 class=data
    s db 5, -3, 12, 0, -8, 7
    len equ $ - s
segment code use32 class=code
start:
pass:
    mov dl, 0               ; swapped = false
    mov esi, 0
    mov ecx, len - 1        ; compare s[i], s[i+1] for i = 0 .. len-2
cmp_next:
    mov al, [s + esi]
    mov bl, [s + esi + 1]
    cmp al, bl
    jle in_order            ; signed: already s[i] <= s[i+1]
    mov [s + esi], bl       ; swap
    mov [s + esi + 1], al
    mov dl, 1               ; swapped = true
in_order:
    inc esi
    loop cmp_next
    cmp dl, 1
    je pass                 ; do ... while (swapped)
                            ; s = -8, -3, 0, 5, 7, 12
```

FINAL RESULT

s = −8, −3, 0, 5, 7, 12.

CHAPTER 10

Strings and String Instructions

The instructions that process memory sequentially — LODS, STOS, MOVS, SCAS, CMPS — their implicit operands ESI, EDI and DF, the repeat prefixes, XLAT, and the classic string patterns.

10.1 Setting up a string operation

A **string** in assembly is simply a sequence of bytes, words or dwords in memory. The string instructions have **implicit operands**:

- the **source** string is at **DS:ESI** (the one you read);
- the **destination** string is at **ES:EDI** (the one you write or scan);
- the **direction flag** DF decides how ESI/EDI move after each element: DF = 0 (`cld`) → they **increase**; DF = 1 (`std`) → they **decrease**.

So before a string loop you always:

1. put the source offset in **ESI**,
2. put the destination offset in **EDI**,
3. put the number of elements in **ECX**,
4. choose the direction with **`cld`** (or `std`).

> **DS AND ES IN OUR PROGRAMS**
>
> In flat Windows programs DS and ES describe the same memory, so you only set ESI and EDI. You never load DS or ES yourself.

10.2 The string instructions

Instruction	Effect (B = byte, W = word, D = dword; step = 1, 2, 4)
`lodsb` / `lodsw` / `lodsd`	AL/AX/EAX ← [ESI]; ESI ± step
`stosb` / `stosw` / `stosd`	[EDI] ← AL/AX/EAX; EDI ± step
`movsb` / `movsw` / `movsd`	[EDI] ← [ESI]; both ± step (the only `mov` memory → memory!)
`scasb` / `scasw` / `scasd`	`cmp` AL/AX/EAX, [EDI]; EDI ± step
`cmpsb` / `cmpsw` / `cmpsd`	`cmp` [ESI], [EDI]; both ± step

```
segment data use32 class=data
    s dw 1234h, 5678h
    d dw 0, 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    cld
    lodsw                   ; AX = 1234h, ESI = s+2
    stosw                   ; d[0] = 1234h, EDI = d+2
    movsw                   ; d[1] = 5678h, ESI = s+4, EDI = d+4
```

TWO MEMORY OPERANDS

`movs` and `cmps` have **both** operands in memory (implicitly) — the exception to the rule "at most one memory operand". They are the answer to the question "give an instruction whose two operands both come from memory".

LODS/STOS USE THE ACCUMULATOR

`lodsb` always loads into **AL** and `stosb` always stores **AL**. Keep your own data out of AL while a string loop runs, or save it first.

10.3 Repeat prefixes

A **prefix** placed before a string instruction repeats it:

Prefix	Repeats while	Used with
`rep`	ECX ≠ 0	`movs`, `stos`
`repe` / `repz`	ECX ≠ 0 and ZF = 1	`cmps`, `scas` (compare while equal)
`repne` / `repnz`	ECX ≠ 0 and ZF = 0	`scas` (search until found)

Each repetition decrements ECX. If ECX = 0 at the start, the instruction is not executed at all (unlike `loop`).

```
segment data use32 class=data
    s db 'assembly'
    len equ $ - s
    d times len db 0
segment code use32 class=code
start:
    ; copy len bytes from s to d
    mov esi, s
    mov edi, d
    mov ecx, len
    cld
    rep movsb               ; d = 'assembly', ECX = 0
```

10.4 XLAT: table lookup

`xlat` performs **AL ← [EBX + AL]**: it replaces AL by the element of a byte table whose address is in EBX and whose index is AL. A classic use: digit → hexadecimal character.

```
segment data use32 class=data
    hexd db '0123456789ABCDEF'
segment code use32 class=code
start:
    mov ebx, hexd
    mov al, 11
    xlat                    ; AL = 'B'
```

10.5 Typical problems

a) Low and high bytes of a string of words

Words `1234h, 1A2Bh` → B1 = high bytes `12h, 1Ah`, B2 = low bytes `34h, 2Bh`.

```
segment data use32 class=data
    a dw 1234h, 1a2bh
    len_a equ ($ - a) / 2
    B1 times len_a db 0
    B2 times len_a db 0
segment code use32 class=code
start:
    mov esi, a
    mov edi, B2             ; low bytes go here with stosb
    mov ebx, B1
    mov ecx, len_a
    cld
    jecxz done
again:
    lodsw                   ; AX = word, AL = low byte, AH = high byte
    stosb                   ; B2[i] = AL
    mov [ebx], ah           ; B1[i] = AH
    inc ebx
    loop again
done:                       ; B1 = 12h, 1Ah   B2 = 34h, 2Bh
```

b) Mirror a string (reverse copy)

```
segment data use32 class=data
    s db 'abcde'
    lens equ $ - s
    r times lens db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, r + lens - 1   ; last byte of r
    mov ecx, lens
    jecxz done
again:
    cld
    lodsb                   ; forward in s
    std
    stosb                   ; backward in r
    loop again
    cld
done:                       ; r = 'edcba'
```

c) Palindrome check

```
segment data use32 class=data
    t db 'capac'
    lt equ $ - t
    pal db 0
segment code use32 class=code
start:
    mov esi, t
    mov edi, t + lt - 1
    mov ecx, lt / 2
    mov byte [pal], 1
    jecxz done
check:
    mov al, [esi]
    cmp al, [edi]
    jne no
    inc esi
    dec edi
    loop check
    jmp done
no:
    mov byte [pal], 0
done:                       ; pal = 1
```

d) Search a value with repne scasb

```
segment data use32 class=data
    s db 'find x here'
    len equ $ - s
    pos dd -1
segment code use32 class=code
start:
    mov edi, s
    mov ecx, len
    mov al, 'x'
    cld
    repne scasb             ; stops after the first 'x' or when ECX = 0
    jne not_found           ; ZF = 0 -> no 'x'
    dec edi                 ; EDI is one past the match
    sub edi, s              ; position = edi - s
    mov [pos], edi          ; pos = 5
not_found:
```

e) Count a character in a 0-terminated string

```
segment data use32 class=data
    txt db 'banana', 0
    cnt db 0
segment code use32 class=code
start:
    mov esi, txt
    cld
next:
    lodsb
    cmp al, 0
    je done
    cmp al, 'a'
    jne next
    inc byte [cnt]
    jmp next
done:                       ; cnt = 3
```

A 0-terminated string (a "C string") carries no length; the loop runs until it reads the 0 byte.

10.6 Chapter summary

IN SHORT

- Source DS:ESI, destination ES:EDI, count ECX, direction DF (`cld` forward, `std` backward).
- `lods` → accumulator; `stos` ← accumulator; `movs` memory→memory; `scas` compares accumulator with [EDI]; `cmps` compares [ESI] with [EDI].
- `rep` (movs, stos), `repe` / `repz` (compare while equal), `repne` / `repnz` (search until found); each step ECX−1.
- `xlat`: AL ← [EBX + AL].
- After `repne scasb`, EDI is one past the match; after `repe cmpsb`, ZF tells whether the last pair was equal.

Solutions — Chapter 10

Exercise 10.1 — Trace the string registers BASIC

STATEMENT

`s dw 1234h, 5678h, 9ABCh` and `d` is a byte buffer. Give AX (or AL), ESI and EDI (as s+k, d+k) after each line:

```
mov esi, s
mov edi, d
cld
lodsw
stosb
lodsb
std
stosb
```

THEORY USED

Effect and step of each string instruction (10.2); DF (10.1).

STEP BY STEP

line	AX / AL	ESI	EDI
`lodsw`	AX = 1234h	s+2	d
`stosb`	d[0] = 34h (AL)	s+2	d+1
`lodsb`	AL = 78h (low byte of 5678h) → AX = 1278h	s+3	d+1
`std`	DF = 1	s+3	d+1
`stosb`	d[1] = 78h	s+3	d (decreased)

```
segment data use32 class=data
    s dw 1234h, 5678h, 9ABCh
    d db 0, 0, 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    cld
    lodsw                   ; AX = 1234h
    stosb                   ; d[0] = 34h
    lodsb                   ; AL = 78h -> AX = 1278h
    std
    stosb                   ; d[1] = 78h, EDI = d
    cld
```

FINAL RESULT

Final: AX = 1278h, ESI = s+3, EDI = d, d = 34h, 78h.

Exercise 10.2 — Copy a string BASIC

STATEMENT

Copy the string `s db 'Hello, NASM!'` into `d` with one repeated instruction.

THEORY USED

`rep movsb` (10.3).

STEP BY STEP

```
segment data use32 class=data
    s db 'Hello, NASM!'
    len equ $ - s
    d times len db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    mov ecx, len            ; 12
    cld
    rep movsb               ; d = 'Hello, NASM!'
```

FINAL RESULT

`rep movsb` with ESI = s, EDI = d, ECX = len, DF = 0.

Exercise 10.3 — Uppercase copy BASIC

STATEMENT

Build `d` = the uppercase version of `s db 'hello world'` (non-letters unchanged).

THEORY USED

`lodsb` / `stosb` (10.2); 'a' – 'A' = 32 (Chapter 1); unsigned range check (Chapter 9).

STEP BY STEP

```
segment data use32 class=data
    s db 'hello world'
    ls equ $ - s
    d times ls db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    mov ecx, ls
    cld
    jecxz done
next:
    lodsb
    cmp al, 'a'
    jb keep
    cmp al, 'z'
    ja keep
    sub al, 'a' - 'A'       ; lowercase letter -> uppercase
keep:
    stosb
    loop next
done:                       ; d = 'HELLO WORLD'
```

FINAL RESULT

d = 'HELLO WORLD'.

Exercise 10.4 — Compare two strings INTERMEDIATE

STATEMENT

Compare two strings `s1`, `s2` of equal length with `repe cmpsb` and set `eq = 1` if they are equal, 0 otherwise. Try `'abcde'` / `'abcde'` and `'abcde'` / `'abXde'`.

THEORY USED

`repe cmpsb` (10.3); ZF after the last comparison.

STEP BY STEP

```
segment data use32 class=data
    s1 db 'abcde'
    len equ $ - s1
    s2 db 'abXde'
    eq db 0
segment code use32 class=code
start:
    mov esi, s1
    mov edi, s2
    mov ecx, len
    cld
    mov byte [eq], 1
    jecxz same              ; empty strings are equal
    repe cmpsb              ; stops at the first difference or when ECX = 0
    je same                 ; ZF = 1 after the last compare -> all equal
    mov byte [eq], 0
same:                       ; eq = 0 for 'abXde' (1 for 'abcde')
```

Why the `jecxz`: if len = 0, `repe` does nothing and ZF is whatever it was before.

FINAL RESULT

eq = 1 for equal strings, eq = 0 for 'abcde' / 'abXde'.

Exercise 10.5 — Sum of the bytes of each dword INTERMEDIATE

STATEMENT

Given a string of dwords, build a string of bytes where each byte is the sum (mod 256) of the 4 bytes of the matching dword. Use `s dd 01020304h, 0FFh`.

THEORY USED

`lodsd` / `stosb` (10.2); register parts (Chapter 3).

STEP BY STEP

```
segment data use32 class=data
    s dd 01020304h, 0FFh
    ls equ ($ - s) / 4
    d times ls db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    mov ecx, ls
    cld
    jecxz done
next:
    lodsd                   ; EAX = dword
    mov bl, al
    add bl, ah
    shr eax, 16
    add bl, al
    add bl, ah              ; BL = sum of the 4 bytes (mod 256)
    mov al, bl
    stosb
    loop next
done:                       ; d = 0Ah, FFh
```

01020304h → 4 + 3 + 2 + 1 = 10 = 0Ah; 000000FFh → FFh.

FINAL RESULT

d = 0Ah, FFh.

Exercise 10.6 — Keep only the positive elements INTERMEDIATE

STATEMENT

From `s db 3, -2, 0, 7, -9, 12` build `d` with only the strictly positive elements (signed), and count them in `n`.

THEORY USED

`lodsb` / `stosb` (10.2); signed jump `jle` (Chapter 9).

STEP BY STEP

```
segment data use32 class=data
    s db 3, -2, 0, 7, -9, 12
    ls equ $ - s
    d times ls db 0
    n db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, d
    mov ecx, ls
    cld
    jecxz done
next:
    lodsb
    cmp al, 0
    jle skip                ; signed: keep only > 0
    stosb
    inc byte [n]
skip:
    loop next
done:                       ; d = 3, 7, 12   n = 3
```

`stosb` only runs for kept elements, so EDI advances only then and the result is packed.

FINAL RESULT

d = 3, 7, 12; n = 3.

Exercise 10.7 — Concatenate low and high bytes INTERMEDIATE

STATEMENT

Build `r` = the low bytes of the words of `s` followed by the high bytes of the words of `t`, for `s dw 1122h, 3344h` and `t dw 5566h, 7788h, 99AAh`.

THEORY USED

`lodsw`, `stosb` and EDI continuing between loops (10.2).

STEP BY STEP

```
segment data use32 class=data
    s dw 1122h, 3344h
    ls equ ($ - s) / 2
    t dw 5566h, 7788h, 99AAh
    lt equ ($ - t) / 2
    r times ls + lt db 0
segment code use32 class=code
start:
    cld
    mov edi, r
    mov esi, s
    mov ecx, ls
    jecxz part2
loop1:
    lodsw
    stosb                   ; AL = low byte
    loop loop1
part2:
    mov esi, t
    mov ecx, lt
    jecxz done
loop2:
    lodsw
    mov al, ah              ; high byte
    stosb
    loop loop2
done:                       ; r = 22h, 44h, 55h, 77h, 99h
```

EDI is not reset between the two loops, so the second part is appended automatically.

FINAL RESULT

r = 22h, 44h, 55h, 77h, 99h.

Exercise 10.8 — A byte as two hex characters INTERMEDIATE

STATEMENT

Convert the byte `val db 0B7h` into its 2-character hexadecimal text `hx` using `xlat`.

THEORY USED

`xlat` (10.4); nibbles with `shr` and `and` (Chapter 7).

STEP BY STEP

```
segment data use32 class=data
    hexd db '0123456789ABCDEF'
    val db 0B7h
    hx db 0, 0
segment code use32 class=code
start:
    mov ebx, hexd
    mov al, [val]
    shr al, 4               ; high nibble = 0Bh
    xlat
    mov [hx], al            ; 'B'
    mov al, [val]
    and al, 0Fh             ; low nibble = 07h
    xlat
    mov [hx+1], al          ; '7'
```

A VARIABLE CALLED OUT

The name `out` cannot be used for a variable: OUT is an x86 instruction (port output), so NASM rejects `out db 0, 0`. Mnemonics and register names are reserved words.

FINAL RESULT

hx = 'B7'.

Exercise 10.9 — Reverse a string with string instructions INTERMEDIATE

STATEMENT

Reverse `s db 'stack'` into `r` (r = 'kcats') by reading forward with `lodsb` and writing backward with `stosb`.

THEORY USED

DF changes the direction (10.1); pattern b of 10.5.

STEP BY STEP

```
segment data use32 class=data
    s db 'stack'
    lens equ $ - s
    r times lens db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, r + lens - 1   ; last position of r
    mov ecx, lens
    jecxz done
again:
    cld
    lodsb                   ; read forward
    std
    stosb                   ; write backward
    loop again
done:
    cld                     ; leave DF = 0, as C functions expect
                            ; r = 'kcats'
```

FINAL RESULT

r = 'kcats'.

Exercise 10.10 — Split by parity CHALLENGE

STATEMENT

Split `s db 4, 7, 10, 3, 8, 1, 6` into `ev` (even elements) and `od` (odd elements), keeping the order, and store their counts in `ne` and `no`.

THEORY USED

`lodsb`, `stosb` with EDI for one list and a second pointer for the other (10.2); `test` (Chapter 7).

STEP BY STEP

```
segment data use32 class=data
    s db 4, 7, 10, 3, 8, 1, 6
    ls equ $ - s
    ev times ls db 0
    od times ls db 0
    ne db 0
    no db 0
segment code use32 class=code
start:
    mov esi, s
    mov edi, ev             ; even elements with stosb
    mov ebx, od             ; odd elements by hand
    mov ecx, ls
    cld
    jecxz done
next:
    lodsb
    test al, 1
    jnz is_odd
    stosb
    inc byte [ne]
    jmp cont
is_odd:
    mov [ebx], al
    inc ebx
    inc byte [no]
cont:
    loop next
done:                       ; ev = 4, 10, 8, 6   od = 7, 3, 1
```

FINAL RESULT

ev = 4, 10, 8, 6 (ne = 4); od = 7, 3, 1 (no = 3).

Exercise 10.11 — Find a character and its position CHALLENGE

STATEMENT

Find the position (from 0) of the first `'m'` in `s db 'assembly language'` with `repne scasb`; store −1 in `pos` if there is none. Then explain the values of EDI and ECX after the search.

THEORY USED

`repne scasb` (10.3); pattern d of 10.5.

STEP BY STEP

```
segment data use32 class=data
    s db 'assembly language'
    len equ $ - s
    pos dd -1
segment code use32 class=code
start:
    mov edi, s
    mov ecx, len            ; 17
    mov al, 'm'
    cld
    repne scasb
    jne not_found
    mov eax, edi
    dec eax                 ; EDI is one past the match
    sub eax, s
    mov [pos], eax          ; pos = 4
not_found:
```

After the search EDI = s + 5 (one past 'm') and ECX = 17 − 5 = 12: five comparisons were made, each decrementing ECX.

FINAL RESULT

pos = 4; EDI = s + 5, ECX = 12.

Exercise 10.12 — Remove the spaces in place CHALLENGE

STATEMENT

Remove all spaces from the 0-terminated string `t db 'a b c d', 0` **in place** (the result must also end with 0).

THEORY USED

`lodsb` / `stosb` on the same string with ESI ≥ EDI (10.2).

STEP BY STEP

ESI reads every character, EDI writes only the kept ones. Since EDI never overtakes ESI, nothing is overwritten before it is read.

```
segment data use32 class=data
    t db 'a b  c d', 0
segment code use32 class=code
start:
    mov esi, t
    mov edi, t
    cld
next:
    lodsb
    cmp al, ' '
    je next                 ; skip spaces
    stosb                   ; keep (including the final 0)
    cmp al, 0
    jne next
                            ; t = 'abcd', 0
```

FINAL RESULT

t = 'abcd' followed by 0.

PART IV

Procedures and Modules

The stack, CALL/RET, the C library, files, multimodule programs.

CHAPTER 11

The Stack, CALL/RET, cdecl, printf/scanf, Files

How the stack works, how procedures are called and how they return, the cdecl calling convention of C, and how an assembly program uses printf, scanf and the C file functions.

11.1 The stack

The **stack** is a memory area addressed by **SS:ESP**. ESP holds the offset of the **top** element.

- The stack **grows toward smaller addresses**: `push` first decreases ESP, then writes.
- In 32-bit code an element is normally a **dword** (you may also push words, but avoid mixing sizes).
- It is **LIFO**: the last value pushed is the first popped.

Instruction	Effect
`push src`	ESP = ESP − 4, then [ESP] = src (dword)
`pop dst`	dst = [ESP], then ESP = ESP + 4
`push word x` / `pop ax`	the same with 2 bytes
`pushad` / `popad`	push / pop all 8 general registers
`pushf` / `popf`	push / pop EFLAGS

```
push dword [a]          ; a memory operand needs a size
push 15                 ; NASM generates push dword 15
pop eax
```

```
before push eax          after push eax           after pop ebx
|  ...   |              |  ...   |               |  ...   |
|  old   | <- ESP       |  old   |               |  old   | <- ESP
|        |              |  EAX   | <- ESP-4      |  EAX   |  (still there, but free)
```

Combining registers through the stack

The stack is little endian like the rest of memory, which makes it a handy tool for splitting and joining registers:

```
segment code use32 class=code
start:
    mov dx, 1234h
    mov ax, 5678h
    ; DX:AX -> EAX
    push dx
    push ax
    pop eax                 ; EAX = 12345678h (AX is at the lower address)
    ; EAX -> DX:AX
    mov eax, 0AABBCCDDh
    push eax
    pop ax                  ; AX = CCDDh (low word)
    pop dx                  ; DX = AABBh (high word)
```

11.2 CALL and RET

```
call proc       ; push the return address (offset of the next instruction); jump to proc
ret             ; pop EIP -> continue after the call
ret n           ; pop EIP, then ESP += n (removes n bytes of parameters)
```

A procedure in the same program reads its parameters from the stack, above the return address:

```
segment code use32 class=code
; int sum(int a, int b) -> EAX
sum:
    mov eax, [esp + 4]      ; a  ([esp] holds the return address)
    add eax, [esp + 8]      ; b
    ret
start:
    push dword 7            ; b (right to left!)
    push dword 5            ; a
    call sum                ; EAX = 12
    add esp, 4*2            ; the caller cleans the stack
```

The stack frame

If the procedure itself pushes values, `[esp + 4]` no longer finds the first parameter. The standard solution is a **stack frame**: EBP is set once and stays fixed while ESP moves.

```
segment code use32 class=code
; int twice_sum(int a, int b) = 2*(a+b)
twice_sum:
    push ebp
    mov ebp, esp            ; [ebp+4] = return address, [ebp+8] = 1st param
    push ebx                ; saving a register moves ESP, but not EBP
    mov ebx, [ebp + 8]      ; a
    add ebx, [ebp + 12]     ; b
    lea eax, [ebx + ebx]    ; EAX = 2*(a+b)
    pop ebx
    mov esp, ebp
    pop ebp
    ret
start:
    mov ebx, 99
    push dword 4
    push dword 3
    call twice_sum          ; EAX = 14, EBX still 99
    add esp, 8
```

```
[ebp + 12]   2nd parameter
[ebp + 8]    1st parameter
[ebp + 4]    return address
[ebp]        old EBP
[ebp - 4]    saved EBX / local variables
```

11.3 The cdecl convention

C functions (printf, scanf, fopen, ...) follow the **cdecl** convention. When your assembly code calls them — or when C calls your assembly function (Chapter 12) — both sides must respect it:

1. Parameters are pushed **right to left**, each as a **dword**.
2. The result comes back in **EAX** (64-bit results in EDX:EAX).
3. The function may **destroy EAX, ECX, EDX** (volatile registers). EBX, ESI, EDI, EBP are preserved.
4. The **caller** removes the parameters: `add esp, 4 * number_of_parameters`.

A LOOP COUNTER IN ECX AROUND A C CALL

`printf` destroys ECX. A `loop` whose body calls a C function loses its counter. Use EBX, ESI or EDI as the counter (they are preserved), or save ECX on the stack around the call.

11.4 printf and scanf

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    n dd 0
    msg db "n = ", 0                        ; C strings end with 0
    fmt_in db "%d", 0
    fmt_out db "You typed %d (hex %x)", 10, 0  ; 10 = new line

segment code use32 class=code
start:
    push dword msg          ; printf(msg) -> push the ADDRESS of the string
    call [printf]
    add esp, 4*1

    push dword n            ; scanf("%d", &n) -> push the ADDRESS of n
    push dword fmt_in
    call [scanf]
    add esp, 4*2

    push dword [n]          ; printf(fmt_out, n, n) -> push the VALUES of numbers
    push dword [n]
    push dword fmt_out
    call [printf]
    add esp, 4*3

    push dword 0
    call [exit]
```

For the input `42` the program prints `n = You typed 42 (hex 2a)`.

Rules to remember:

- strings (format, text) → push the **address**: `push dword msg`;
- scanf targets → push the **address** of the variable: `push dword n`;
- printf numbers → push the **value** as a dword: `push dword [n]`. A byte or word must be widened to a dword first (`movsx eax, byte [b]` / `push eax`).
- Formats: `%d` signed decimal, `%u` unsigned decimal, `%x` hexadecimal, `%c` character, `%s` string.

Every function used must appear in `extern` and have its own `import … msvcrt.dll` line.

11.5 Files

C function	Use	Returns in EAX
`fopen(name, mode)`	open; mode `"r"`, `"w"`, `"a"`, `"r+"` …	file handle, 0 on error
`fprintf(f, fmt, ...)`	write formatted text	characters written (< 0 on error)
`fscanf(f, fmt, &x...)`	read formatted	items read (EOF = −1)
`fread(buf, size, count, f)`	read raw bytes	number of elements read
`fwrite(buf, size, count, f)`	write raw bytes	number written
`fclose(f)`	close	0

Read a text file and print it

```
bits 32
global start
extern exit, fopen, fread, fclose, printf
import exit msvcrt.dll
import fopen msvcrt.dll
import fread msvcrt.dll
import fclose msvcrt.dll
import printf msvcrt.dll

segment data use32 class=data
    fname db "input.txt", 0
    mode db "r", 0
    handle dd -1
    buf times 101 db 0      ; 100 chars + 0 terminator
    fmt db "%s", 0

segment code use32 class=code
start:
    push dword mode
    push dword fname
    call [fopen]            ; fopen("input.txt", "r")
    add esp, 4*2
    cmp eax, 0
    je final                ; could not open
    mov [handle], eax

    push dword [handle]     ; fread(buf, 1, 100, handle)
    push dword 100
    push dword 1
    push dword buf
    call [fread]
    add esp, 4*4            ; EAX = number of bytes read
    mov byte [buf + eax], 0 ; make it a C string

    push dword buf
    push dword fmt
    call [printf]
    add esp, 4*2
```

```
    push dword [handle]
    call [fclose]
    add esp, 4
final:
    push dword 0
    call [exit]
```

The file name is relative to the folder from which the program runs. For files longer than the buffer, put the `fread` in a loop and stop when it returns 0 (Exercise 11.10).

NOT CHECKING FOPEN

If the file does not exist, `fopen` returns 0. Using 0 as a handle in `fread` crashes the program — always check EAX right after `fopen`.

11.6 Chapter summary

IN SHORT

- `push`: ESP −= 4 then write; `pop`: read then ESP += 4; the stack grows down; LIFO.
- DX:AX → EAX: `push dx`, `push ax`, `pop eax`; EAX → DX:AX: `push eax`, `pop ax`, `pop dx`.
- `call` pushes the return address; `ret` pops it; `ret n` also removes n bytes.
- Stack frame: `push ebp` / `mov ebp, esp`; parameters at [ebp+8], [ebp+12]...; end with `mov esp, ebp` / `pop ebp` / `ret`.
- cdecl: right to left, result in EAX, EAX/ECX/EDX destroyed, caller cleans with `add esp, 4*n`.
- printf/scanf: strings and scanf targets by address, printf numbers by value (as dwords).
- Files: fopen (check 0!), fread/fwrite/fprintf/fscanf, fclose.

Solutions — Chapter 11

Exercise 11.1 — What does this sequence do? BASIC

STATEMENT

`push eax` / `push ebx` / `pop eax` / `pop ebx` — what happened? Check with EAX = 1, EBX = 2.

THEORY USED

The stack is LIFO (11.1).

STEP BY STEP

```
segment code use32 class=code
start:
    mov eax, 1
    mov ebx, 2
    push eax                ; stack: 1
    push ebx                ; stack: 1, 2 (top)
    pop eax                 ; EAX = 2
    pop ebx                 ; EBX = 1
```

The last value pushed (EBX) comes out first, into EAX: the two registers are **swapped**, and ESP is back to its initial value.

FINAL RESULT

EAX and EBX are swapped (EAX = 2, EBX = 1).

Exercise 11.2 — Follow ESP BASIC

STATEMENT

Initially `ESP = 0019FF74h`. Give ESP after each line: `push eax`, `push ax`, `pop bx`, `push dword 7`, `pop ecx`, `pop edx`.

THEORY USED

`push` / `pop` of dwords move ESP by 4, of words by 2 (11.1).

STEP BY STEP

line	change	ESP
start	–	0019FF74h
`push eax`	−4	0019FF70h
`push ax`	−2	0019FF6Eh
`pop bx`	+2	0019FF70h
`push dword 7`	−4	0019FF6Ch
`pop ecx`	+4 (ECX = 7)	0019FF70h
`pop edx`	+4 (EDX = old EAX)	0019FF74h

FINAL RESULT

70h, 6Eh, 70h, 6Ch, 70h, 74h (all 0019FFxxh); ECX = 7, EDX = the old EAX.

Exercise 11.3 — Address or value? BASIC

STATEMENT

Why do we write `push dword msg` for a text but `push dword [n]` for a number in `printf`? And why `push dword n` in `scanf`?

THEORY USED

printf/scanf rules (11.4).

STEP BY STEP

- `printf` needs the **address** of the format string or text (a `char*`) → `push dword msg`.
- For `%d` it needs the **value** of the number (an `int`) → `push dword [n]`.
- `scanf` must **write** into n, so it needs where n is: its **address** (`&n`) → `push dword n`.

FINAL RESULT

Strings and scanf targets are passed by address; printf numbers by value.

Exercise 11.4 — Joining and splitting through the stack BASIC

STATEMENT

Put the word result of `mul` (DX:AX) into EAX using the stack, then split `EBX = 11223344h` into `CX` (high word) and `SI` (low word).

THEORY USED

Combining registers through the stack (11.1).

STEP BY STEP

```
segment code use32 class=code
start:
    mov ax, 1000
    mov bx, 300
    mul bx                  ; DX:AX = 300000 = 0004 93E0h
    push dx
    push ax
    pop eax                 ; EAX = 300000
    mov ebx, 11223344h
    push ebx
    pop si                  ; SI = 3344h (low word, at the lower address)
    pop cx                  ; CX = 1122h
```

FINAL RESULT

EAX = 300 000 (000493E0h); SI = 3344h, CX = 1122h.

Exercise 11.5 — Read two numbers, print (a+b)·(a−b) INTERMEDIATE

STATEMENT

Read two numbers `a` and `b` from the keyboard and print `(a+b)*(a-b)`. What is printed for the input `7 3`?

THEORY USED

scanf/printf (11.4); `imul` (Chapter 5).

STEP BY STEP

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    a dd 0
    b dd 0
    result dd 0
    fin db "%d %d", 0
    fout db "(a+b)*(a-b) = %d", 10, 0

segment code use32 class=code
start:
    push dword b
    push dword a
    push dword fin
    call [scanf]            ; scanf("%d %d", &a, &b)
    add esp, 4*3
    mov ebx, [a]
    add ebx, [b]            ; EBX = a+b
    mov eax, [a]
    sub eax, [b]            ; EAX = a-b
    imul ebx                ; EDX:EAX = (a+b)*(a-b)
    mov [result], eax
    push dword [result]
    push dword fout
    call [printf]
    add esp, 4*2
    push dword 0
    call [exit]
```

For `7 3`: (10)·(4) = 40 → prints `(a+b)*(a-b) = 40`.

FINAL RESULT

Prints `(a+b)*(a-b) = 40` for 7 and 3.

Exercise 11.6 — Read until 0, print the maximum INTERMEDIATE

STATEMENT

Read numbers until 0 is read; then print the maximum of the numbers read (0 excluded). What is printed for `4 -2 17 9 0`?

THEORY USED

scanf in a loop (11.4); signed comparison (Chapter 9).

STEP BY STEP

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    x dd 0
```

```
    max dd 80000000h        ; the smallest signed dword
    fin db "%d", 0
    fout db "max = %d", 10, 0

segment code use32 class=code
start:
read_next:
    push dword x
    push dword fin
    call [scanf]
    add esp, 4*2
    mov eax, [x]
    cmp eax, 0
    je print
    cmp eax, [max]
    jle read_next           ; signed
    mov [max], eax
    jmp read_next
print:
    push dword [max]
    push dword fout
    call [printf]
    add esp, 4*2
    push dword 0
    call [exit]
```

Starting from 80000000h (−2 147 483 648) guarantees that the first number read becomes the maximum. Output: `max = 17`.

FINAL RESULT

Prints `max = 17`.

Exercise 11.7 — Write 1...10 to a file INTERMEDIATE

STATEMENT

Write the numbers 1...10, one per line, to the file `out.txt`.

THEORY USED

fopen/fprintf/fclose (11.5); volatile registers (11.3).

STEP BY STEP

```
bits 32
global start
extern exit, fopen, fprintf, fclose
import exit msvcrt.dll
import fopen msvcrt.dll
import fprintf msvcrt.dll
import fclose msvcrt.dll

segment data use32 class=data
    fname db "out.txt", 0
    mode db "w", 0
    fmt db "%d", 10, 0
    h dd 0

segment code use32 class=code
start:
    push dword mode
    push dword fname
    call [fopen]
```

```
    add esp, 8
    cmp eax, 0
    je final
    mov [h], eax
    mov ebx, 1              ; EBX is preserved by C functions
write_next:
    push ebx
    push dword fmt
    push dword [h]
    call [fprintf]          ; fprintf(h, "%d\n", ebx)
    add esp, 4*3
    inc ebx
    cmp ebx, 10
    jbe write_next
    push dword [h]
    call [fclose]
    add esp, 4
final:
    push dword 0
    call [exit]
```

EBX is the counter, not ECX: ECX is destroyed by `fprintf`.

FINAL RESULT

out.txt contains the lines 1 to 10.

Exercise 11.8 — A procedure with three parameters INTERMEDIATE

STATEMENT

Write a procedure `sum3(a, b, c)` with a stack frame that returns a + b + c in EAX, preserves EBX, and call it with 10, −4, 25 (cdecl).

THEORY USED

CALL/RET and the stack frame (11.2); cdecl (11.3).

STEP BY STEP

```
segment code use32 class=code
sum3:
    push ebp
    mov ebp, esp
    push ebx                ; EBX will be used: save it
    mov ebx, [ebp + 8]      ; a
    add ebx, [ebp + 12]     ; + b
    add ebx, [ebp + 16]     ; + c
    mov eax, ebx            ; result in EAX
    pop ebx
    mov esp, ebp
    pop ebp
    ret
start:
    mov ebx, 777
    push dword 25           ; c
    push dword -4           ; b
    push dword 10           ; a
    call sum3               ; EAX = 31
    add esp, 4*3            ; EBX is still 777, ESP restored
```

FINAL RESULT

EAX = 31; EBX and ESP unchanged.

Exercise 11.9 — Print a byte signed and unsigned INTERMEDIATE

STATEMENT

Print the byte `b db -5` once as a signed number and once as an unsigned number, on one line: `-5 251`.

THEORY USED

printf needs dwords: `movsx` / `movzx` (11.4, Chapter 5).

STEP BY STEP

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    b db -5
    fmt db "%d %d", 10, 0

segment code use32 class=code
start:
    movzx eax, byte [b]     ; 251 (zero extension)
    push eax                ; 2nd %d
    movsx eax, byte [b]     ; -5 (sign extension)
    push eax                ; 1st %d
    push dword fmt
    call [printf]           ; prints "-5 251"
    add esp, 4*3
    push dword 0
    call [exit]
```

`push dword [b]` would push the byte FBh together with the 3 bytes that follow it in memory — a wrong number.

FINAL RESULT

Prints `-5 251`.

Exercise 11.10 — Count the digits in a file CHALLENGE

STATEMENT

Count the decimal digit characters in the file `data.txt` (of any length) by reading it in chunks of 8 bytes with `fread`, and print the count. For a file containing `Room 101, floor 3, code 7741`, what is printed?

THEORY USED

fread in a loop until it returns 0 (11.5); EBX/ESI/EDI preserved across calls (11.3).

STEP BY STEP

```
bits 32
global start
extern exit, fopen, fread, fclose, printf
```

```
import exit msvcrt.dll
import fopen msvcrt.dll
import fread msvcrt.dll
import fclose msvcrt.dll
import printf msvcrt.dll

segment data use32 class=data
    fname db "data.txt", 0
    mode db "r", 0
    h dd 0
    buf times 8 db 0
    fmt db "digits: %d", 10, 0

segment code use32 class=code
start:
    push dword mode
    push dword fname
    call [fopen]
    add esp, 8
    cmp eax, 0
    je final
    mov [h], eax
    mov ebx, 0              ; digit counter (preserved by C calls)
read_chunk:
    push dword [h]
    push dword 8
    push dword 1
    push dword buf
    call [fread]            ; EAX = bytes read (0 at the end)
    add esp, 4*4
    cmp eax, 0
    je finished
    mov esi, 0              ; index in buf
check:
    mov dl, [buf + esi]
    cmp dl, '0'
    jb not_digit
    cmp dl, '9'
    ja not_digit
    inc ebx
not_digit:
    inc esi
    cmp esi, eax            ; EAX = number of valid bytes in buf
    jb check
    jmp read_chunk
finished:
    push ebx
    push dword fmt
    call [printf]
    add esp, 8
    push dword [h]
    call [fclose]
    add esp, 4
final:
    push dword 0
    call [exit]
```

Only the first EAX bytes of the buffer are valid after each `fread`, so the inner loop stops at EAX. The digits of the sample are 1, 0, 1, 3, 7, 7, 4, 1 → prints `digits: 8`.

FINAL RESULT

Prints `digits: 8`.

Exercise 11.11 — A GCD procedure CHALLENGE

STATEMENT

Write a procedure `gcd(a, b)` (Euclid's algorithm with remainders) that returns the greatest common divisor in EAX, and use it from `start` to print `gcd(84, 36) = 12`.

THEORY USED

Procedures and parameters (11.2); `div` (Chapter 5); printf (11.4).

STEP BY STEP

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    fmt db "gcd(%d, %d) = %d", 10, 0

segment code use32 class=code
; unsigned gcd(unsigned a, unsigned b)
gcd:
    push ebp
    mov ebp, esp
    push ebx
    mov eax, [ebp + 8]      ; a
    mov ebx, [ebp + 12]     ; b
.again:
    cmp ebx, 0
    je .done                ; gcd(a, 0) = a
    mov edx, 0
    div ebx                 ; EDX = a mod b
    mov eax, ebx            ; a = b
    mov ebx, edx            ; b = a mod b
    jmp .again
.done:
    pop ebx
    mov esp, ebp
    pop ebp
    ret

start:
    push dword 36
    push dword 84
    call gcd                ; EAX = 12
    add esp, 8
    push eax
    push dword 36
    push dword 84
    push dword fmt
    call [printf]           ; gcd(84, 36) = 12
    add esp, 4*4
    push dword 0
    call [exit]
```

84 mod 36 = 12; 36 mod 12 = 0 → gcd = 12. Labels starting with a dot (`.again`) are **local** to the previous normal label (`gcd`), so another procedure may reuse the same names.

FINAL RESULT

Prints `gcd(84, 36) = 12`.

Exercise 11.12 — Bug hunt in a printing loop CHALLENGE

STATEMENT

The program below should print the squares of 1...5, one per line, but it misbehaves. Find the bugs and give a corrected version.

```
segment data use32 class=data
    fmt db "%d", 10
segment code use32 class=code
start:
    mov ecx, 5
    mov ebx, 1
next:
    mov eax, ebx
    mul ebx
    push dword fmt
    push eax
    call [printf]
    inc ebx
    loop next
    push dword 0
    call [exit]
```

THEORY USED

cdecl (11.3): order of parameters, cleaning the stack, volatile ECX; C strings end with 0 (11.4).

STEP BY STEP

bug	effect	fix
`fmt` has no 0 terminator	printf reads past the string	`fmt db "%d", 10, 0`
parameters pushed in the wrong order	printf receives the number as the format address	push `eax` first, then `fmt`
no `add esp, 8` after the call	the stack grows by 8 bytes per iteration	`add esp, 4*2`
ECX is the `loop` counter	printf destroys ECX → the loop count is lost	use EBX as the counter, `cmp` / `jbe`

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll

segment data use32 class=data
    fmt db "%d", 10, 0

segment code use32 class=code
start:
    mov ebx, 1              ; EBX survives the calls
next:
    mov eax, ebx
    mul ebx                 ; EAX = ebx*ebx
    push eax                ; last parameter first
    push dword fmt
    call [printf]
    add esp, 4*2
    inc ebx
    cmp ebx, 5
    jbe next                ; prints 1 4 9 16 25 (one per line)
    push dword 0
    call [exit]
```

FINAL RESULT

Four bugs: missing 0 terminator, reversed parameters, stack not cleaned, ECX destroyed by printf. The fixed program prints 1, 4, 9, 16, 25.

CHAPTER 12

Multimodule Programming: ASM + ASM and ASM + C

Splitting a program into separately assembled modules: global and extern, linking several object files, calling assembly functions from C, and the call, entry and exit code that make a function call work.

12.1 Why several modules?

A large program is split into **modules** (source files). Each module is assembled or compiled **separately** into an **object file** (`.obj`), and the **linker** joins the object files into one executable. One module contains the entry point; the others **export** functions (and possibly variables).

Two keywords let a symbol cross the border between modules:

- `global name` — "**I define** `name` here; other modules may use it" (export);
- `extern name` — "`name` is **defined in another module**" (import).

The linker matches every `extern` with exactly one `global` of the same name. A missing definition gives an *unresolved external* error at link time.

12.2 ASM + ASM

main.asm — the module with the entry point:

```
bits 32
global start
extern exit, printf
import exit msvcrt.dll
import printf msvcrt.dll
extern sum                      ; defined in sum.asm

segment data use32 class=data
    a dd 10
    b dd 20
    fmt db "%d + %d = %d", 0

segment code use32 class=code
start:
    push dword [b]              ; sum(a, b) - right to left
    push dword [a]
    call sum                    ; NOT call [sum]: sum is a label, not a pointer from a DLL
    add esp, 4*2                ; EAX = 30
    push eax
    push dword [b]
    push dword [a]
    push dword fmt
    call [printf]               ; prints 10 + 20 = 30
    add esp, 4*4
    push dword 0
    call [exit]
```

sum.asm — a module that exports a function:

```
bits 32
segment code use32 public class=code    ; 'public' lets the linker merge code segments
global sum

; int sum(int a, int b)
sum:
    mov eax, [esp + 4]          ; a
    add eax, [esp + 8]          ; b
    ret                         ; result in EAX
```

Build:

```
nasm -f obj main.asm
nasm -f obj sum.asm
alink main.obj sum.obj -oPE -subsys console -entry start
```

CALL SUM VERSUS CALL [PRINTF]

For a function imported from a DLL with `import`, the name designates a **pointer** to the function, so you call `[printf]`. For your own label (in the same module or exported by another module with `global`), the name *is* the address of the code: `call sum`, without brackets.

Variables can be shared too: `global v` in the module that defines `v dd 5`, `extern v` in the module that uses it (Exercise 12.11).

12.3 ASM + C

Now the C module contains `main` and the assembly module provides functions. Three differences from ASM + ASM:

1. Object format **win32** (COFF) instead of obj: `nasm -f win32 mod.asm`.
2. The C compiler adds an **underscore** before names: the C function `sum` is `_sum` in assembly.
3. The **cdecl** convention must be followed exactly (Chapter 11): parameters on the stack right to left, result in EAX, the caller cleans the stack, the function may destroy EAX/ECX/EDX but must preserve EBX, ESI, EDI, EBP.

sum.asm

```
bits 32
segment code use32 public class=code
global _sum

; int sum(int a, int b)
_sum:
    mov eax, [esp + 4]
    add eax, [esp + 8]
    ret
```

main.c (or main.cpp with `extern "C"`)

```
#include <stdio.h>

int sum(int a, int b);        /* in C++: extern "C" int sum(int a, int b); */

int main() {
    int a = 10, b = 20;
    printf("%d + %d = %d\n", a, b, sum(a, b));
    return 0;
}
```

Build (MinGW): `nasm -f win32 sum.asm` then `gcc -m32 sum.obj main.c -o main.exe` (or `g++ -m32 … main.cpp`). In Visual Studio, add the `.asm` file with a NASM custom build step.

12.4 A function with a stack frame and saved registers

```
bits 32
segment code use32 public class=code
global _count_vowels

; int count_vowels(const char* s)
_count_vowels:
    push ebp
    mov ebp, esp
    push ebx                    ; EBX is non-volatile -> save it
    mov ebx, [ebp + 8]          ; s
    mov eax, 0                  ; counter
.next:
    mov cl, [ebx]
    cmp cl, 0
    je .done
    or cl, 20h                  ; lowercase (letters only differ in bit 5)
    cmp cl, 'a'
    je .vowel
    cmp cl, 'e'
    je .vowel
    cmp cl, 'i'
    je .vowel
    cmp cl, 'o'
    je .vowel
    cmp cl, 'u'
    jne .skip
.vowel:
    inc eax
.skip:
    inc ebx
    jmp .next
.done:
    pop ebx                     ; restore
    mov esp, ebp
    pop ebp
    ret
```

`count_vowels("Assembly Is Fun")` returns 4 (A, e, I, u). The stack inside the function, after `push ebp` / `mov ebp, esp`:

address	content
`[ebp + 12]`	2nd parameter
`[ebp + 8]`	1st parameter
`[ebp + 4]`	return address
`[ebp]`	old EBP
`[ebp − 4]` …	saved registers / local variables

12.5 Call code, entry code, exit code

A function call needs three pieces of code:

Call code (in the caller):

1. save the volatile resources it still needs (EAX, ECX, EDX, flags);

2. pass the parameters (push, right to left);
3. save the return address and jump: `call`.

After the call (not part of the call code, but mandatory): remove the parameters (`add esp, …`, cdecl) and restore the volatile resources.

Entry code (at the start of the callee):

1. build the stack frame: `push ebp` / `mov ebp, esp`;
2. allocate local variables: `sub esp, n`;
3. save the non-volatile registers it will modify (`push ebx` …).

Exit code (at the end of the callee) — the reverse:

1. restore the non-volatile registers;
2. free the locals and the frame: `mov esp, ebp` / `pop ebp`;
3. return: `ret` (or `ret n` in the stdcall convention, where the callee also removes the parameters).

Who writes them?

caller → callee	call code	entry code	exit code
C → C	C compiler	C compiler	C compiler
C → ASM	C compiler	ASM programmer	ASM programmer
ASM → C	ASM programmer	C compiler	C compiler
ASM → ASM	ASM programmer (at least the `call`)	nothing mandatory	at least `ret`

The code is **generated** at compile/assembly time (written into the program) and **runs** every time the function is called.

Volatile versus non-volatile

- **Volatile** registers (EAX, ECX, EDX, EFLAGS) belong to the **callee**: it may destroy them; if the caller needs them after the call, the caller saves and restores them.
- **Non-volatile** registers (EBX, ESI, EDI, EBP, ESP) belong to the **caller**: if the callee modifies them, it saves them in the entry code and restores them in the exit code.

12.6 Chapter summary

> **IN SHORT**
>
> - `global` exports a symbol, `extern` imports it; the linker matches them.
> - ASM+ASM: `nasm -f obj` each module, `alink a.obj b.obj -oPE -subsys console -entry start`; code segments `public`.
> - `call [printf]` for DLL imports, `call sum` for your own labels.
> - ASM+C: `nasm -f win32`, names with `_`, cdecl; preserve EBX, ESI, EDI, EBP.
> - Frame: [ebp+8] first parameter, [ebp+4] return address, [ebp] old EBP, [ebp−4] saved registers/locals.
> - Call code (caller), entry code and exit code (callee); who writes them depends on the languages.

Solutions — Chapter 12

Exercise 12.1 — global or extern? BASIC

STATEMENT

`main.asm` calls the function `sum` and reads the variable `limit`; both are defined in `lib.asm`. Which declarations are needed in each module?

THEORY USED

`global` and `extern` (12.1).

STEP BY STEP

- `lib.asm` defines them → `global sum, limit` (or two `global` lines).
- `main.asm` uses them → `extern sum, limit`.
- `main.asm` also has `global start` (the entry point) and the usual `extern` / `import` lines for `exit`.

FINAL RESULT

lib.asm: `global sum` and `global limit`; main.asm: `extern sum` and `extern limit`.

Exercise 12.2 — Brackets or not? BASIC

STATEMENT

Explain the difference between `call sum` and `call [printf]`. What would `call printf` (without brackets) do?

THEORY USED

call sum versus call [printf] (12.2).

STEP BY STEP

- `call sum`: `sum` is a code label — its value is the address of the code; the processor jumps there.
- `call [printf]`: with `import printf msvcrt.dll`, the name `printf` designates a memory location (filled in by the loader) that **contains** the address of the real function; the brackets read that address and call it.
- `call printf` would jump to the memory location itself and execute the bytes of an address as instructions — the program crashes.

FINAL RESULT

Own labels: `call name`; DLL imports: `call [name]` (indirect, through a pointer). `call printf` jumps into data and crashes.

Exercise 12.3 — Names and formats BASIC

STATEMENT

A C program calls `int maxim(int a, int b)` written in assembly. (a) What label must the assembly function have? (b) What NASM format must be used? (c) Give the commands to build with MinGW.

THEORY USED

ASM + C (12.3).

STEP BY STEP

(a) `_maxim`, declared `global _maxim` (the C compiler adds the underscore).

(b) `win32` (COFF), not `obj`.

(c) `nasm -f win32 maxim.asm` then `gcc -m32 maxim.obj main.c -o main.exe`.

FINAL RESULT

`_maxim`; `-f win32`; `nasm -f win32 maxim.asm` + `gcc -m32 maxim.obj main.c -o main.exe`.

Exercise 12.4 — Why save EBX but not ECX? BASIC

STATEMENT

Why must `_count_vowels` (section 12.4) save EBX but not ECX?

THEORY USED

Volatile versus non-volatile registers (12.5).

STEP BY STEP

EBX is **non-volatile** in cdecl: the C caller expects it unchanged after the call, so the callee saves it in the entry code and restores it in the exit code. ECX is **volatile**: the caller does not rely on it, so the function may destroy it freely.

FINAL RESULT

EBX is non-volatile (callee-saved); ECX is volatile (the caller saves it if needed).

Exercise 12.5 — max in assembly, called from C INTERMEDIATE

STATEMENT

Write an assembly module with `int max(int a, int b)` (signed) used from a C `main` that reads two numbers and prints the maximum.

THEORY USED

ASM + C (12.3); signed comparison (Chapter 9).

STEP BY STEP

```
; max.asm -> nasm -f win32 max.asm
bits 32
segment code use32 public class=code
global _max

_max:
    mov eax, [esp + 4]          ; a
    cmp eax, [esp + 8]          ; compare with b (signed)
    jge .done
    mov eax, [esp + 8]
.done:
    ret
```

```
/* main.c -> gcc -m32 max.obj main.c -o main.exe */
#include <stdio.h>

int max(int a, int b);

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    printf("max = %d\n", max(a, b));
    return 0;
}
```

The function uses only EAX (volatile) and the stack, so no register needs saving. max(3, −9) = 3, max(−4, −1) = −1.

FINAL RESULT

`_max` returns the larger (signed) parameter in EAX; e.g. max(−4, −1) = −1.

Exercise 12.6 — Sum of the digits of a C string INTERMEDIATE

STATEMENT

Write `int sum_digits(const char* s)` in assembly: the sum of the digit characters of a 0-terminated string. What does `sum_digits("a1b2c3")` return?

THEORY USED

ASM + C (12.3); character → digit value (Chapter 1).

STEP BY STEP

```
bits 32
segment code use32 public class=code
global _sum_digits

; int sum_digits(const char* s)
_sum_digits:
    mov edx, [esp + 4]          ; EDX = s (volatile, no need to save)
    mov eax, 0                  ; sum
.next:
    movzx ecx, byte [edx]
    cmp ecx, 0
    je .done
    cmp ecx, '0'
    jb .skip
    cmp ecx, '9'
    ja .skip
    sub ecx, '0'                ; character -> digit value
    add eax, ecx
.skip:
    inc edx
    jmp .next
.done:
    ret
```

Only volatile registers (EAX, ECX, EDX) are used, so there is nothing to save. `sum_digits("a1b2c3")` = 1 + 2 + 3 = **6**.

FINAL RESULT

Returns 6.

Exercise 12.7 — Use count_vowels INTERMEDIATE

STATEMENT

Using `_count_vowels` from section 12.4, what are `count_vowels("Assembly Is Fun")`, `count_vowels("xyz")` and `count_vowels("")`? Which registers does the function modify, and which does it restore?

THEORY USED

Section 12.4; cdecl (Chapter 11).

STEP BY STEP

- "Assembly Is Fun": A, e, I, u → **4** (the `or cl, 20h` makes the comparison case-insensitive).
- "xyz": **0**.
- "": the first byte is 0 → **0**.

The function modifies EAX (result) and CL (volatile); it uses EBX and EBP but restores both, and ESP is back to its value at the call.

FINAL RESULT

4, 0, 0; EAX/ECX modified, EBX/EBP saved and restored.

Exercise 12.8 — ASM + ASM with scanf and printf INTERMEDIATE

STATEMENT

Write `main.asm` that reads two numbers, calls `max` from another assembly module and prints `max = …`. Give the module and the build commands. What is printed for `-4 -1`?

THEORY USED

ASM + ASM (12.2); `call max` without brackets.

STEP BY STEP

```
bits 32
global start
extern exit, printf, scanf
import exit msvcrt.dll
import printf msvcrt.dll
import scanf msvcrt.dll
extern max

segment data use32 class=data
    a dd 0
    b dd 0
    fin db "%d %d", 0
    fout db "max = %d", 10, 0

segment code use32 class=code
start:
    push dword b
    push dword a
    push dword fin
    call [scanf]
```

```
    add esp, 4*3
    push dword [b]
    push dword [a]
    call max
    add esp, 4*2
    push eax
    push dword fout
    call [printf]
    add esp, 4*2
    push dword 0
    call [exit]
```

```
; max.asm -> nasm -f obj max.asm
bits 32
segment code use32 public class=code
global max

max:
    mov eax, [esp + 4]
    cmp eax, [esp + 8]
    jge .done
    mov eax, [esp + 8]
.done:
    ret
```

Build: `nasm -f obj main.asm`, `nasm -f obj max.asm`, `alink main.obj max.obj -oPE -subsys console -entry start`. For `-4 -1` it prints `max = -1`.

FINAL RESULT

Prints `max = -1`; label `max` without underscore, format obj, link both objects with ALINK.

Exercise 12.9 — Identify the pieces of a call INTERMEDIATE

STATEMENT

In the code below, mark the call code, the entry code and the exit code, and say who would write each piece if the caller were C.

```
    push ecx
    push dword [n]
    push dword v
    call _f
    add esp, 8
    pop ecx
    ...
_f: push ebp
    mov ebp, esp
    sub esp, 4
    push esi
    ...
    pop esi
    mov esp, ebp
    pop ebp
    ret
```

THEORY USED

Call, entry and exit code (12.5).

STEP BY STEP

- **Call code**: `push ecx` (save a volatile register the caller needs), `push dword [n]`, `push dword v` (parameters, right to left), `call _f`. After it: `add esp, 8` (remove parameters), `pop ecx` (restore).
- **Entry code**: `push ebp`, `mov ebp, esp` (frame), `sub esp, 4` (one local dword), `push esi` (save a non-volatile register).
- **Exit code**: `pop esi`, `mov esp, ebp`, `pop ebp`, `ret`.

With a C caller (C → ASM): the C compiler writes the call code; the assembly programmer writes the entry and exit code of `_f`.

FINAL RESULT

Call code: pushes + `call` (+ `add esp, 8` / `pop ecx` after); entry: frame, locals, `push esi`; exit: `pop esi`, `mov esp, ebp`, `pop ebp`, `ret`. C → ASM: compiler writes the call code, programmer the rest.

Exercise 12.10 — Sum of an int array CHALLENGE

STATEMENT

Write `int sum_array(const int* v, int n)` in assembly with a stack frame, using ESI as the pointer (so it must be saved). What does it return for v = {5, −2, 10, 7} and n = 4? And for n = 0?

THEORY USED

Stack frame and non-volatile registers (12.4, 12.5); `jecxz` (Chapter 9).

STEP BY STEP

```
bits 32
segment code use32 public class=code
global _sum_array

; int sum_array(const int* v, int n)
_sum_array:
    push ebp
    mov ebp, esp
    push esi                    ; non-volatile: save
    mov esi, [ebp + 8]          ; v
    mov ecx, [ebp + 12]         ; n
    mov eax, 0
    jecxz .done                 ; n = 0 -> sum 0
.next:
    add eax, [esi]
    add esi, 4
    loop .next
.done:
    pop esi                     ; restore
    mov esp, ebp
    pop ebp
    ret
```

5 − 2 + 10 + 7 = **20**; for n = 0 the loop is skipped and the result is **0**.

FINAL RESULT

20; 0 for n = 0.

Exercise 12.11 — A variable shared by two modules CHALLENGE

STATEMENT

Module `counter.asm` defines `count dd 0` and a procedure `bump` that adds 1 to it. `main.asm` calls `bump` three times and then prints `count`. Write both modules.

THEORY USED

Sharing variables with `global` / `extern` (12.1, 12.2).

STEP BY STEP

```
; counter.asm
bits 32
global count, bump

segment data use32 public class=data
    count dd 0

segment code use32 public class=code
bump:
    inc dword [count]
    ret
```

```
; main.asm
bits 32
global start
extern exit, printf
import exit msvcrt.dll
import printf msvcrt.dll
extern count, bump

segment data use32 class=data
    fmt db "count = %d", 10, 0

segment code use32 class=code
start:
    call bump
    call bump
    call bump
    push dword [count]          ; the variable of the other module
    push dword fmt
    call [printf]               ; count = 3
    add esp, 8
    push dword 0
    call [exit]
```

The data segment of the module is declared `public` as well, so the linker can merge it with the data of main.

FINAL RESULT

Prints `count = 3`.

Exercise 12.12 — cdecl versus stdcall CHALLENGE

STATEMENT

Rewrite `sum(a, b)` so that the **callee** removes the parameters (stdcall style), and show how the call changes. Why does `printf` have to be cdecl?

THEORY USED

`ret n` (Chapter 11, 12.5).

STEP BY STEP

```
segment code use32 class=code
; int sum(int a, int b) - stdcall: the callee cleans the stack
sum:
    mov eax, [esp + 4]
    add eax, [esp + 8]
    ret 8                       ; pop the return address, then ESP += 8
start:
    push dword 20
    push dword 10
    call sum                    ; EAX = 30
                                ; no add esp, 8: ESP is already restored
```

With `ret 8` the callee removes its 2 parameters, so the caller does not write `add esp, 8`. This works only when the number of parameters is fixed. `printf` has a **variable** number of parameters — only the caller knows how many it pushed — so it must use cdecl, where the caller cleans the stack.

FINAL RESULT

`ret 8` in the callee and no `add esp` in the caller; printf is variadic, so only the caller can clean the stack.

PART V

Inside the Instruction

How an instruction becomes machine-code bytes.

CHAPTER 13

Machine Instruction Representation (Internal Format, ModR/M, SIB, Prefixes)

How the assembler turns an instruction into bytes: the internal format of an x86 instruction, the ModR/M and SIB bytes, displacements and immediates, and the four categories of prefixes.

13.1 The internal format

The main job of an assembler is to generate the **bytes** of each instruction. On x86 an instruction takes between **1 and 15 bytes**:

```
[prefixes] + opcode + [ModR/M] + [SIB] + [displacement] + [immediate]
```

Only the **opcode** is mandatory. You can see these bytes in the second column of OllyDbg's code window.

Field	Size	Role
prefixes	0–4 bytes	change the behaviour of the instruction (repeat, segment, operand/address size)
opcode	1–2 (3) bytes	which operation (and sometimes which register / which size)
ModR/M	1 byte	the operands: register or memory, and how the memory offset is formed
SIB	1 byte	present only if the offset has an index·scale part (or ESP as base)
displacement	1, 2 or 4 bytes	the constant part of a memory offset (direct addressing or `[reg+const]`)
immediate	1, 2 or 4 bytes	a constant operand (`mov eax, 5`)

This maps exactly onto the offset formula of Chapter 8, **[base] + [index·scale] + [constant]**: the base and the index·scale go into ModR/M + SIB, the constant goes into the displacement field.

Multi-byte displacements and immediates are stored **little endian**, like all data: `mov eax, 12345678h` is `B8 78 56 34 12`.

13.2 The ModR/M byte

```
7  6 | 5  4  3    | 2  1  0
Mod  | Reg/Opcode | R/M
```

- **Mod** (2 bits): `11` → R/M is a **register**; `00` → memory `[reg]` with no displacement; `01` → memory `[reg] + disp8`; `10` → memory `[reg] + disp32`. Special case: Mod = `00` and R/M = `101` → direct address `[disp32]`.
- **Reg/Opcode** (3 bits): a register operand, or an **opcode extension** (`/0` ... `/7`) for instructions such as `add r/m, imm` that have only one register/memory operand.
- **R/M** (3 bits): a register, or the base of the memory operand; R/M = `100` means **"a SIB byte follows"**.

Register codes (which register depends on the operand size given by the opcode):

code	32-bit	16-bit	8-bit
000	EAX	AX	AL
001	ECX	CX	CL
010	EDX	DX	DL
011	EBX	BX	BL
100	ESP	SP	AH
101	EBP	BP	CH
110	ESI	SI	DH
111	EDI	DI	BH

THE DIRECTION OF THE OPERANDS

For the same operation there are usually two opcodes: one with the register in Reg as **source** (e.g. `89` = MOV r/m32, r32) and one with the register in Reg as **destination** (`8B` = MOV r32, r/m32). That is why `mov eax, ebx` can be written `89 D8`.

13.3 The SIB byte

```
7  6  | 5  4  3 | 2  1  0
Scale | Index   | Base
```

- **Scale**: `00` = 1, `01` = 2, `10` = 4, `11` = 8.
- **Index**: a register code; `100` means **no index** (that is why ESP can never be an index).
- **Base**: a register code.

The SIB byte is present when ModR/M has R/M = `100`.

[ESP] WITHOUT A SIB

R/M = `100` does not mean "[ESP]" — it means "a SIB byte follows". So `[esp]` can only be expressed with a SIB byte whose base is ESP and whose index is `100` (none): `mov ecx, [esp]` = `8B 0C 24`.

13.4 Decoded examples

Instruction	Bytes	Explanation
`mov ebx, esi`	89 F3	89 = MOV r/m32, r32. F3 = `11 110 011`: Mod 11 = register; Reg 110 = ESI (source); R/M 011 = EBX (destination)
`mov ecx, [ebx]`	8B 0B	8B = MOV r32, r/m32. 0B = `00 001 011`: memory, no displacement; Reg = ECX; R/M = [EBX]
`mov [edx+17], ecx`	89 4A 11	4A = `01 001 010`: Mod 01 = [reg] + disp8; Reg = ECX; R/M = EDX; disp8 = 11h = 17
`mov ecx, [esp+ebx*4]`	8B 0C 9C	0C = `00 001 100`: R/M 100 → SIB. 9C = `10 011 100`: scale 4, index EBX, base ESP
`add eax, [ebx+esi*2+8]`	03 44 73 08	44 = `01 000 100`: disp8, Reg = EAX, SIB follows. 73 = `01 110 011`: ×2, ESI, EBX. disp8 = 08
`mov eax, 12345678h`	B8 78 56 34 12	opcode B8 + register code (EAX = 0); immediate in little endian
`add byte [ebx], 5`	80 03 05	80 = group "op r/m8, imm8"; 03 = `00 000 011`: Reg field 000 = extension /0 = ADD; R/M = [EBX]; immediate 05

How to decode by hand

1. Look up the opcode (it tells you the operation and the operand size).
2. Split ModR/M into Mod | Reg | R/M.
3. If Mod = 11, R/M is a register. Otherwise it is memory: if R/M = 100 read a SIB byte; then read 0, 1 or 4 displacement bytes according to Mod.
4. Read the immediate, if the opcode has one.

13.5 Prefixes

At most **4 prefixes** — one from each group — can appear on one instruction. Four categories:

Given explicitly by the programmer

1. **Repeat prefixes** — `F3h` = REP / REPE / REPZ, `F2h` = REPNE / REPNZ. Example: `rep movsb` → `F3 A4`.
2. **Segment-override prefixes** — `2Eh` CS, `36h` SS, `3Eh` DS, `26h` ES, `64h` FS, `65h` GS. Example: `mov eax, [ss:ebx]` → `36 8B 03`.

Generated implicitly by the assembler

3. **Operand-size prefix** `66h` — the instruction uses a 16-bit operand while the code is 32-bit (`bits 32`): `mov ax, bx` → `66 89 D8`; `cbw` → `66 98` (result in AX) but `cwde` → `98`; `push ax` → `66 50`.
4. **Address-size prefix** `67h` — the address is 16-bit in 32-bit code: `mov eax, [bx]` → `67 8B 07`.

Several at once: in `bits 16` code, `rep push dword [cs:ebx]` → `F3 2E 66 67 FF 33` (all four categories).

> **WHY CBW AND CWDE SHARE AN OPCODE**
>
> `cbw` (AL → AX) and `cwde` (AX → EAX) are the same operation "double the size of the accumulator". In 32-bit code the default size is 32 bits, so the bare opcode `98` means `cwde`; the operand-size prefix `66` switches it to the 16-bit version, `cbw`. The same holds for `cwd` (`66 99`) and `cdq` (`99`).

13.6 Chapter summary

IN SHORT

- Format: [prefixes] opcode [ModR/M] [SIB] [displacement] [immediate]; 1–15 bytes; only the opcode is mandatory.
- ModR/M = Mod | Reg | R/M. Mod 11 register, 00 [reg], 01 [reg]+disp8, 10 [reg]+disp32; Mod 00 + R/M 101 = [disp32]; R/M 100 = SIB follows.
- SIB = Scale | Index | Base; index 100 = none; [esp] always needs a SIB.
- Register codes 000...111: EAX ECX EDX EBX ESP EBP ESI EDI.
- Displacements and immediates are little endian.
- Prefixes: repeat (F3, F2), segment (2E 36 3E 26 64 65), operand size 66, address size 67.

Solutions — Chapter 13

Exercise 13.1 — Decode a ModR/M byte BASIC

STATEMENT

Decode the ModR/M byte `D8h` in the instruction `89 D8` (89 = MOV r/m32, r32).

THEORY USED

ModR/M fields and register codes (13.2).

STEP BY STEP

D8h = `11 011 000`:

- Mod = 11 → R/M is a register;
- Reg = 011 = EBX → the **source** (opcode 89: Reg is the source);
- R/M = 000 = EAX → the **destination**.

FINAL RESULT

`mov eax, ebx`.

Exercise 13.2 — Fields of a direct-address instruction BASIC

STATEMENT

Which fields appear in the encoding of `mov ecx, [v]` (direct addressing)? What changes for `mov eax, [v]` ?

THEORY USED

Internal format (13.1); special case Mod 00, R/M 101 (13.2).

STEP BY STEP

`mov ecx, [v]` = opcode **8B** + ModR/M **0D** = `00 001 101` (Mod 00 + R/M 101 = direct `[disp32]`, Reg 001 = ECX) + a **4-byte displacement** = the offset of `v`. No SIB, no immediate: 6 bytes.

For EAX the processor has a shorter special form: opcode **A1** followed directly by the 4-byte offset (5 bytes); NASM chooses it automatically. `8B 05` + disp32 would also be valid.

FINAL RESULT

Opcode 8B + ModR/M (Mod 00, R/M 101) + disp32; for EAX the short form A1 + disp32.

Exercise 13.3 — Why does [esp] need a SIB? BASIC

STATEMENT

Why does `mov ecx, [esp]` need a SIB byte although there is no index? Give its bytes.

THEORY USED

R/M = 100 means “SIB follows”; index 100 = none (13.3).

STEP BY STEP

The R/M code of ESP, 100, is reserved to announce a SIB byte, so `[esp]` cannot be written in ModR/M alone. It is written with a SIB byte: base = ESP (100), index = 100 (none), scale irrelevant.

`8B 0C 24`: 0C = `00 001 100` (memory, Reg = ECX, SIB follows); 24 = `00 100 100` (scale 1, no index, base ESP).

FINAL RESULT

Because R/M 100 means "SIB follows"; `mov ecx, [esp]` = 8B 0C 24.

Exercise 13.4 — Implicit prefixes BASIC

STATEMENT

Which prefix(es) does NASM generate (in 32-bit code) for `cwd`, `cdq`, `mov ax, [esi]`, `es lodsb`? Give the bytes.

THEORY USED

Prefix categories (13.5).

STEP BY STEP

instruction	bytes	prefix
`cwd`	66 99	operand size (16-bit result DX:AX)
`cdq`	99	none (32-bit)
`mov ax, [esi]`	66 8B 06	operand size
`es lodsb`	26 AC	segment override ES

FINAL RESULT

cwd 66 99; cdq 99; mov ax,[esi] 66 8B 06; es lodsb 26 AC.

Exercise 13.5 — The four prefix categories INTERMEDIATE

STATEMENT

Name the four categories of prefixes, say which ones the programmer writes explicitly, and give one example of each with its bytes.

THEORY USED

Prefixes (13.5).

STEP BY STEP

category	written by	example	bytes
repeat	programmer	`rep movsb`	F3 A4
segment override	programmer	`mov eax, [es:ebx]`	26 8B 03
operand size	assembler (implicit)	`mov ax, bx`	66 89 D8
address size	assembler (implicit)	`mov eax, [bx]`	67 8B 07

FINAL RESULT

Repeat and segment override (explicit); operand size 66 and address size 67 (implicit).

Exercise 13.6 — Decode with a displacement INTERMEDIATE

STATEMENT

Decode `8B 4B 08` (8B = MOV r32, r/m32).

THEORY USED

ModR/M with Mod 01 (13.2).

STEP BY STEP

4Bh = `01 001 011`: Mod 01 → memory with an 8-bit displacement; Reg 001 = ECX (destination); R/M 011 = EBX (base). The next byte 08 is the displacement.

FINAL RESULT

`mov ecx, [ebx+8]`.

Exercise 13.7 — Decode with a SIB byte INTERMEDIATE

STATEMENT

Decode `8B 44 B3 10`.

THEORY USED

ModR/M and SIB (13.2, 13.3).

STEP BY STEP

- 44h = `01 000 100`: Mod 01 → disp8 follows; Reg 000 = EAX; R/M 100 → SIB follows.
- B3h = `10 110 011`: scale 10 = 4; index 110 = ESI; base 011 = EBX.
- disp8 = 10h.

FINAL RESULT

`mov eax, [ebx+esi*4+10h]`.

Exercise 13.8 — Encode two instructions INTERMEDIATE

STATEMENT

Encode `mov edx, [eax]` (opcode 8B = MOV r32, r/m32) and `add ecx, edx` (opcode 01 = ADD r/m32, r32).

THEORY USED

ModR/M (13.2); direction of the operands.

STEP BY STEP

- `mov edx, [eax]`: Reg = EDX (010) is the destination; R/M = [EAX]: Mod 00, R/M 000. ModR/M = `00 010 000` = 10h → **8B 10**.
- `add ecx, edx`: with opcode 01, R/M is the destination (ECX, register → Mod 11, R/M 001) and Reg is the source (EDX = 010). ModR/M = `11 010 001` = D1h → **01 D1**.

FINAL RESULT

8B 10 and 01 D1.

Exercise 13.9 — Byte-sized moves INTERMEDIATE

STATEMENT

Given the opcodes 88 = MOV r/m8, r8 and 8A = MOV r8, r/m8, encode `mov [ebx], al` and `mov al, [ebx]`.

THEORY USED

Register codes for 8-bit registers (13.2).

STEP BY STEP

Both use ModR/M = `00 000 011` = 03h (Mod 00 = memory without displacement, Reg 000 = AL, R/M 011 = [EBX]). Only the opcode tells the direction:

- `mov [ebx], al` → **88 03**
- `mov al, [ebx]` → **8A 03**

FINAL RESULT

88 03 and 8A 03.

Exercise 13.10 — Instruction lengths CHALLENGE

STATEMENT

How many bytes does each instruction take, and which fields does it have? `mov eax, 12345678h`, `mov ecx, [ebx+esi*4+100h]`, `add byte [ebx], 5`, `mov ax, bx`, `rep movsb`.

THEORY USED

The internal format (13.1).

STEP BY STEP

instruction	bytes	fields	length
`mov eax, 12345678h`	B8 78 56 34 12	opcode (+reg) + imm32	5
`mov ecx, [ebx+esi*4+100h]`	8B 8C B3 00 01 00 00	opcode + ModR/M + SIB + disp32	7
`add byte [ebx], 5`	80 03 05	opcode + ModR/M + imm8	3
`mov ax, bx`	66 89 D8	prefix + opcode + ModR/M	3
`rep movsb`	F3 A4	prefix + opcode	2

The displacement 100h does not fit in a signed byte, so Mod = 10 (disp32) is used: 8C = `10 001 100`.

FINAL RESULT

5, 7, 3, 3, 2 bytes.

Exercise 13.11 — A 32-bit displacement CHALLENGE

STATEMENT

Decode `8B 8B 00 01 00 00`.

THEORY USED

Mod 10 → disp32 (13.2); little endian.

STEP BY STEP

- 8Bh (second byte) = `10 001 011`: Mod 10 → 32-bit displacement; Reg 001 = ECX; R/M 011 = EBX.
- disp32 = bytes 00 01 00 00 → **00000100h** = 256.

Note that the first 8B is the opcode and the second 8B is a ModR/M byte that happens to have the same value.

FINAL RESULT

`mov ecx, [ebx+100h]`.

Exercise 13.12 — All four prefixes at once CHALLENGE

STATEMENT

Explain every byte of `F3 2E 66 67 FF 33`, the encoding of `rep push dword [cs:ebx]` in `bits 16` code. Why do the 66 and 67 prefixes appear here, although they would not in 32-bit code?

THEORY USED

Prefixes (13.5); ModR/M with an opcode extension (13.2).

STEP BY STEP

byte	meaning
F3	repeat prefix (REP)
2E	segment override: CS
66	operand size: a **32-bit** operand (dword) in 16-bit code
67	address size: a **32-bit** address ([EBX]) in 16-bit code
FF	opcode group (INC/DEC/CALL/JMP/PUSH r/m)
33	ModR/M `00 110 011`: Mod 00, Reg 110 = extension /6 = PUSH, R/M 011 = [EBX]

The size prefixes always mean "the **other** size than the default". In 16-bit code the default operand and address size is 16 bits, so a dword operand and a 32-bit register address both need a prefix. In 32-bit code they are the default and no prefix would be generated.

FINAL RESULT

F3 rep, 2E CS, 66 operand size, 67 address size, FF opcode, 33 ModR/M (/6 = PUSH, [EBX]).